

C++ for SWE Interviews

Types · Memory · Move Semantics · Smart Pointers
OOP · Templates · Strings · Ranges · I/O

Created by → **Sahil Raj**
Codeforces → [After_Dark](#)

Contents

1	Preprocessor, Headers, and the Compilation Model	9
1.1	How C++ Compilation Works	9
1.2	Header Guards and <code>#pragma once</code>	9
1.3	Forward Declarations	10
1.4	Preprocessor Macros	10
1.5	What Goes in Headers vs. Source Files	11
2	The Basics: Types, Namespaces, and <code>const</code>	13
2.1	Integer Types: <code>int</code> vs. <code>int32_t</code> and Fixed-Width Types	13
2.1.1	The Problem with Built-in Integer Types	13
2.1.2	Fixed-Width Integer Types (<code><cstdint></code>)	13
2.1.3	What is <code>size_t</code> and <code>ptrdiff_t</code> ?	13
2.2	Namespaces and Scope Resolution	13
2.2.1	Why Namespaces Exist	14
2.2.2	The Scope Resolution Operator <code>::</code>	14
2.2.3	Nested Namespaces (C++17 Shorthand)	14
2.2.4	<code>using</code> Declarations vs. Directives	14
2.2.5	Anonymous Namespaces (Internal Linkage)	14
2.2.6	Key <code>std</code> Sub-namespaces	15
2.3	<code>const</code> vs. <code>constexpr</code> : When and Why	15
2.3.1	The Core Difference	15
2.3.2	When to Use <code>const</code>	15
2.3.3	When to Use <code>constexpr</code>	15
2.3.4	Combining <code>const</code> and <code>constexpr</code> on Member Functions	16
2.3.5	Worked Example: All Four Combinations	16
2.3.6	The Blank-Filling Game: <code>const/constexpr/empty</code> on Free Functions	17
2.4	If <code>constexpr</code> : Compile-Time Conditional Branching	19
2.4.1	The Solution: <code>if constexpr</code>	20
2.4.2	Rules for Usage	20
3	Literals, Type Aliases, Type Deduction, and <code>auto</code>	21
3.1	Literals and Magic Numbers	21
3.1.1	Integer Literal Suffixes	21
3.1.2	Floating-Point Literal Suffixes	21
3.1.3	Character and String Literals	22
3.2	Type Aliases: <code>using</code> and <code>typedef</code>	22
3.3	<code>auto</code> : Let the Compiler Deduce Types	22
3.3.1	<code>decltype</code> : Query a Type Without Evaluating	23
4	Operators In Depth	25
4.1	Arithmetic Operators and Integer Division	25
4.2	Increment / Decrement: Pre vs. Post	25
4.3	Bitwise Operators	26

4.3.1	Common Bit Manipulation Tricks	26
4.4	The Conditional Operator <code>?:</code>	26
4.5	The Comma Operator	27
4.6	<code>sizeof</code> Operator	27
5	Control Flow: If, Switch, Loops, and Halts	29
5.1	If, Else, and If-Initializer	29
5.2	Switch Statements	30
5.3	Loops	30
5.3.1	While Loop	30
5.3.2	Do-While Loop	30
5.3.3	For Loop	31
5.3.4	Range-Based For Loop	31
5.3.5	Break, Continue, and Return	31
5.4	Program Termination	32
6	Functions In Depth	33
6.1	Default Arguments	33
6.2	Function Overloading	33
6.3	Inline Functions	34
6.4	Assertions: Catching Bugs Early	34
6.5	Recursive Functions	34
6.6	Function Pointers	35
6.7	<code>std::function</code> : Type-Erased Callable	35
7	I/O, Streams, and Fast I/O	37
7.1	I/O in C++: <code>cin/cout</code> , Streams, and <code>FastIO</code>	37
7.1.1	The C++ I/O Stack: What Actually Happens on <code>cout << x</code>	37
7.1.2	<code>std::ios::sync_with_stdio(false)</code>	37
7.1.3	<code>std::cin.tie(nullptr)</code>	38
7.1.4	The <code>istream</code> / <code>ostream</code> Class Hierarchy	38
7.1.5	Stream State Bits	39
7.1.6	String Streams	39
7.1.7	File I/O	40
7.1.8	Why <code>FastIO</code> ? The Remaining Bottleneck	41
7.1.9	<code>FastIO</code> : Complete Deep Dive	41
8	Strings, <code>string_view</code>, and C++20 Ranges	45
8.1	<code>std::string</code> : The Essentials	45
8.1.1	String Conversion	45
8.1.2	Trimming Whitespace (No Standard Function)	46
8.1.3	String Streams	46
8.2	<code>std::string_view</code> : Zero-Cost Slicing	46
8.3	C++20 Ranges and Views	47
8.3.1	Ranges Algorithms: No More <code>.begin()/end()</code>	47
8.3.2	Views: Lazy Composable Pipelines	48
8.3.3	Common Views Reference	48
9	Scope, Storage Duration, and Lifetime	49
9.1	Variable Scope and Storage Duration	49
9.2	Static Variables	49
9.3	<code>thread_local</code> : Per-Thread Storage	50
10	References: Everything You Need to Know	51
10.1	Lvalue References	51

10.1.1	Const References Extend Temporary Lifetime	51
10.2	Reference vs. Pointer: When to Use Which	51
11	Memory, Pointers, and Smart Pointers	53
11.1	Pointers & Dereferencing	53
11.1.1	The Map Analogy	53
11.1.2	Code Example	53
11.1.3	The Trap: Nullptr Dereferencing	53
11.2	Random Number Generation: <code>rand()</code> , Mersenne Twister & Distributions	53
11.2.1	Three Reasons <code>rand()</code> is Broken	53
11.2.2	The Solution: Mersenne Twister (<code>std::mt19937</code>)	54
11.2.3	Engine + Distribution Architecture	54
11.2.4	Is a Mersenne Number Always Prime?	54
11.2.5	Two Mathematical Criteria	54
11.2.6	Connection to <code>std::mt19937</code>	54
11.2.7	Code Example	55
11.2.8	Standard Library Distribution Overview	55
11.3	Linkage Rules: <code>const</code> , <code>inline</code> , and ODR	55
11.3.1	Why Global <code>const</code> Defaults to Internal Linkage	55
11.3.2	What is <code>inline</code> in Modern C++?	55
11.3.3	C++17 Inline Variables	56
11.3.4	Global Variables: The Four Flavours and Their Linkage	56
11.4	Implicit Conversion Ranking, <code>= delete</code> , and Ambiguity Fixes	58
11.4.1	The <code>printint</code> Overload Problem	58
11.4.2	Case 1: <code>printint('a')</code> — Exact Match on a Deleted Overload	58
11.4.3	Case 2: <code>printint(5LL)</code> — Ambiguity Crash	58
11.4.4	The Clean Fix: C++20 Concepts	58
12	Type Conversion and Type Safety	59
12.1	Implicit (Automatic) Conversions	59
12.2	Explicit Conversions with <code>static_cast</code>	59
12.3	<code>std::optional</code> : A Value or Nothing	60
13	Classes and OOP	61
13.1	Classes & OOP	61
13.1.1	Enums & Scoped <code>enum class</code>	61
13.1.2	Runtime Overhead: The Hidden Paperwork Tax	61
13.1.3	Cache-Locality Architecture: AoS vs. SoA	62
13.1.4	Copy Constructors & Memory Safety	68
13.1.5	Copy Elision: RVO & NRVO	70
13.1.6	Common Constructor Errors	71
13.1.7	The <code>this</code> Pointer & Method Chaining	72
13.1.8	Static Members	73
13.1.9	Ref-Qualifiers	73
13.1.10	Constexpr Classes	75
13.1.11	The <code>const</code> After a Function Name	76
14	Operators, Containers, and Object Relations	79
14.1	Operator Overloading	79
14.1.1	Basic Example: <code>vector2D</code>	79
14.1.2	Returning by Value vs. by Reference in Arithmetic Operators	79
14.1.3	I/O Operators: Why <code>operator<<</code> Needs <code>friend</code>	80
14.1.4	The Spaceship Operator <code><=></code> and <code>operator==</code>	83
14.1.5	Custom Container Classes	83
14.2	Containers: <code>std::vector</code> , <code>std::array</code> and Safe Iteration	86

14.2.1	<code>emplace_back()</code> vs <code>push_back()</code>	86
14.2.2	<code>reserve()</code> , <code>capacity()</code> , <code>size()</code> and <code>resize()</code>	87
14.2.3	<code>std::array</code> vs C-Style Array	88
14.2.4	Erasing from <code>std::vector</code> Inside a Loop	88
14.3	Memory: Stack, Heap, <code>new/delete</code> and Function Pointers	90
14.3.1	Stack vs. Heap	90
14.3.2	Handling <code>new</code> Failures	91
14.3.3	Function Pointers	92
14.4	Lambda Functions	93
14.4.1	Why Lambdas are Superior to Function Pointers	93
14.4.2	Lambdas are Immutable by Default (<code>mutable</code>)	94
14.4.3	IIFE: Immediately Invoked Function Expression	94
14.4.4	The Capture Clause: All 6 Forms	95
14.4.5	Init-Capture / Generalized Capture (C++14)	97
14.4.6	Captures and Value Categories: The Golden Rule	98
14.4.7	What is <code>std::function</code> ?	98
14.4.8	Recursive Lambdas: <code>std::function</code> vs. <code>auto&& self</code>	99
14.4.9	<code>std::function</code> vs. <code>auto</code> : A Detailed Comparison	100
14.4.10	Type Erasure: The Hidden Cost of <code>std::function</code>	101
14.5	Object Relations: Composition, Aggregation, Association & Inheritance	101
14.5.1	Composition — “Has-a” with Ownership	102
14.5.2	Aggregation — “Has-a” without Ownership	102
14.5.3	Association — “Uses-a” with No Storage	103
14.5.4	Inheritance — Tightest Coupling	104
14.5.5	<code>std::initializer_list</code> for Custom Containers	105
15	Advanced OOP: Virtual Dispatch and Inheritance Patterns	107
15.1	Advanced OOP: <code>virtual</code> , Vtables, Inheritance Patterns & Operator Overloading	107
15.1.1	What is Polymorphism?	107
15.1.2	1. What <code>virtual</code> does here (The Runtime Detour)	108
15.1.3	2. Why you <i>had</i> to change <code>Animal::speak()</code> to <code>const</code>	109
15.1.4	3. What <code>override</code> does (The Compiler Bodyguard)	109
15.1.5	4. What the Virtual Destructor (<code>virtual Animal() = default;</code>) does here	109
15.1.6	The Silent Bug: What happens <i>without</i> <code>override</code> ?	109
15.1.7	How <code>override</code> Acts as Your Bodyguard	110
15.1.8	3 Sneaky Mistakes <code>override</code> Will Catch	110
15.1.9	Rule of Thumb	110
15.1.10	The <code>final</code> Keyword	111
15.1.11	How Vtables Work Under the Hood	111
15.1.12	1. The Blueprint: What is a Vtable?	111
15.1.13	2. The Anchor: What is a <code>vptr</code> ?	112
15.1.14	3. Step-by-Step Runtime Dispatch Trace	112
15.1.15	How the Vtables Are Built (Slot-by-Slot)	113
15.1.16	Tracing <code>ref.hello()</code> at Runtime	113
15.1.17	Why <code>override</code> Is Mandatory for Safety	113
15.1.18	Virtual Calls vs. Inlining	114
15.1.19	Four Rules of Safe Polymorphic Code	115
15.1.20	Abstract Classes vs. Interface Classes	117
15.1.21	Multiple Inheritance and the Diamond Problem	119
15.1.22	1. What it looks like in Memory	120
15.1.23	2. The Fatal Flaw: The Diamond Problem	120
15.1.24	The Broken Code Layout	121
15.1.25	Why does this fail?	121
15.1.26	How Virtual Inheritance Works Under the Hood	122

15.1.275. Modern Best Practice: Mixin Interfaces	122
15.1.281. Side-by-Side Comparison of the Hidden Pointers	122
15.1.292. The Functional Differences	123
15.1.30 Summary Rule of Thumb	123
15.1.311. The Core Syntax and How It Works	123
15.1.322. Pointers vs. References: The Error Handling Catch	124
15.1.333. How does it check types at runtime? (RTTI)	124
15.1.344. The Golden Rules of <code>dynamic_cast</code>	125
15.1.351. The Code Blueprint: A Polymorphic Factory	125
15.1.362. The Payoff: Eliminating Type Slicing and Casts	126
15.1.374. The Strict Constraints of Covariance	126
15.1.381. The Core Trap: The Assignment Operator (<code>operator=</code>)	127
15.1.39 The Broken Way (Data Loss)	127
15.1.40 The Correct Way: Explicit Base Invocation	127
15.1.412. Stream Insertion (<code>operator<<</code>) and Slicing	127
15.1.42 The Problem: Static Binding of Free Functions	128
15.1.43 The Elegant Solution: The Virtual Helper Pattern	128
15.1.443. Comparison Operators (<code>operator==</code> , <code>operator<</code>)	129
15.1.45 Design Best Practice	129
16 <code>std::pair</code>, <code>std::tuple</code>, and <code>std::variant</code>	131
16.1 <code>std::pair</code>	131
16.2 <code>std::tuple</code> : N-ary Pair	131
16.3 <code>std::variant</code> : Type-Safe Union	132
17 STL Containers	133
17.1 Container Overview	133
17.2 <code>std::map</code> and <code>std::unordered_map</code>	134
17.3 <code>std::set</code> and <code>std::unordered_set</code>	134
17.4 <code>std::stack</code> , <code>std::queue</code> , <code>std::priority_queue</code>	135
17.5 <code>std::deque</code> : Double-Ended Queue	135
18 Iterators and Algorithms	137
18.1 Iterator Categories	137
18.2 <code>std::algorithm</code> : The 20 You Need to Know	138
19 Error Handling and Exceptions	141
19.1 Why Exceptions?	141
19.2 <code>try / catch / throw</code>	142
19.3 The Standard Exception Hierarchy	142
19.4 Custom Exceptions	143
19.5 <code>noexcept</code> : Promising No Exceptions	143
19.6 Exception Safety Guarantees	143
19.7 RAII: The Connection Between Exceptions and Resource Safety	144
19.8 <code>std::expected</code> (C++23): Typed Error Returns	144
20 Move Semantics and Value Categories	145
20.1 Value Categories: Lvalues, Rvalues, and References	145
20.1.1 Lvalues vs. Rvalues	145
20.1.2 Lvalue References (&) vs. Rvalue References (&&)	146
20.1.3 Converting Lvalues to Rvalues: <code>std::move</code>	146
20.1.4 Reference Collapsing and Forwarding References	146
21 Templates and Compile-Time Programming	149
21.1 Template Processing: Compilation Mechanics & Metaprogramming	149

21.1.1	From Blueprint to Instance	149
21.1.2	Trade-Offs	149
21.1.3	Template Metaprogramming (TMP)	149
21.2	Polymorphism: Compile-Time vs. Runtime (vtable)	149
21.2.1	Compile-Time Polymorphism (Static Binding)	150
21.2.2	Runtime Polymorphism (Dynamic Binding)	150
21.2.3	Mechanics of the Virtual Lookup Table (vtable)	150
21.3	Templates + <code>if constexpr</code> : Compile-Time Specialisation	150
21.3.1	Example: Iterator Advancement	150
21.4	<code>constexpr</code> : Immediate Functions & Zero-Runtime Footprints	151
21.4.1	<code>constexpr</code> vs. <code>constexpr</code>	151
21.5	The Casting Nuclear Options: <code>const_cast</code> & <code>reinterpret_cast</code>	151
21.5.1	<code>const_cast</code> — Stripping the Constness Label	151
21.5.2	<code>reinterpret_cast</code> — Raw Bit Reinterpretation	153
21.6	Undefined Behaviour & Constant Folding	154
21.6.1	Why Does It Not Crash at <code>-00</code> ?	155
21.6.2	The Trap: Constant Folding under <code>-02</code>	155
21.7	Compiler Optimisation Tiers: <code>-00</code> through <code>-03</code>	155
21.7.1	<code>-00</code> : Zero Optimisation (Debug)	155
21.7.2	<code>-02</code> : Production Level	155
21.7.3	<code>-03</code> : Aggressive Speed Performance	155
21.7.4	Trade-offs of <code>-03</code>	155
22	Timing, Formatting, and Advanced Class Features	157
22.1	Basic Timing with <code>std::chrono</code>	157
22.2	<code>std::format</code> (C++20): Type-Safe String Formatting	157
22.3	Class Template Argument Deduction (CTAD, C++17)	157
22.4	The Rule of Zero	158
22.5	Aggregate Types and Aggregate Initialization	158
22.6	RAII Scope Guard: Cleanup on Any Exit	158
23	C++ Attributes	161
23.1	Attributes You Should Actually Use	161
24	C++20 Features: Concepts and More	163
24.1	Concepts: Constrained Templates	163
24.2	<code>std::span</code> : Non-Owning View over Contiguous Data	163
24.3	Three-Way Comparison: <code><=></code> (Spaceship)	164
24.4	Designated Initializers (C++20)	164
25	Multithreading and Concurrency	165
25.1	<code>std::thread</code> : Basic Threading	165
25.2	Mutexes and Locks	165
25.3	<code>std::atomic</code> : Lock-Free Shared Variables	166
25.4	<code>std::async</code> and <code>std::future</code>	166
25.5	<code>std::condition_variable</code>	166
26	Common Interview Patterns and Idioms	169
26.1	Copy-and-Swap for Assignment Operators	169
26.2	Structured Bindings: Full Reference	169
26.3	Useful <code>std::algorithm</code> Patterns for Interviews	170

Chapter 1

Preprocessor, Headers, and the Compilation Model

1.1 How C++ Compilation Works

Every `.cpp` file is a **translation unit** (TU). The compiler processes each TU independently, then the linker stitches them together.

Concept / Pattern

```
1 // Three stages:
2 // 1. Preprocessor: handles #include, #define, #if -- purely text substitution
3 // 2. Compiler:      translates each .cpp TU into a .o object file
4 // 3. Linker:        combines .o files, resolves references, produces the binary
5
6 // Compile steps manually:
7 // g++ -c math.cpp -o math.o      <- compile math.cpp
8 // g++ -c main.cpp -o main.o      <- compile main.cpp
9 // g++ math.o main.o -o program   <- link together
```

1.2 Header Guards and `#pragma once`

Headers can be included multiple times. Without guards you get redefinition errors.

```
1 // Traditional include guard (portable, always works)
2 #ifndef MY_MATH_H
3 #define MY_MATH_H
4
5 double square(double x);
6
7 #endif // MY_MATH_H
8
9 // Modern shorthand -- preferred in new code
10 #pragma once
11
12 double square(double x);
```

Key Insight

Prefer `#pragma once`. It's shorter, less error-prone (no typo risk in the macro name), and compilers can skip re-reading the file entirely.

1.3 Forward Declarations

Declare something without defining it, so other files know it exists without pulling in a full header.

```

1 // Instead of #include "BigClass.h" in a header, forward-declare:
2 class BigClass;           // compiler knows the name, not the internals
3 void process(BigClass* obj); // OK: pointer/reference -- size not needed
4
5 // You CAN forward-declare when:
6 //   - you only use pointers or references to the type
7 //   - you don't call methods or access members in the header
8
9 // You CANNOT forward-declare when:
10 //   - you create an object by value (compiler needs the full size)
11 //   - you call methods or access fields

```

1.4 Preprocessor Macros

```

1 // Object-like macros -- prefer constexpr instead
2 #define MAX_SIZE 100
3 #define PI 3.14159
4
5 // Function-like macros -- prefer inline functions or templates
6 #define SQUARE(x) ((x) * (x)) // parentheses are critical!
7 SQUARE(3 + 1) // becomes (3+1)*(3+1) = 16; without parens: 3+1*3+1 = 7
8
9 // Conditional compilation -- this is what macros are genuinely good for
10 #ifdef NDEBUG
11     // release-only code
12 #endif
13
14 #if defined(_WIN32)
15     // Windows-specific
16 #elif defined(__linux__)
17     // Linux-specific
18 #endif
19
20 // Useful predefined macros
21 __FILE__ // current filename as a string
22 __LINE__ // current line number as int
23 __func__ // current function name (C++11)
24 __cplusplus // 202002L for C++20, 201703L for C++17

```

Common Mistake

Macros are dumb text substitution – they don't respect scope, types, or namespaces and can't be debugged in a meaningful way. Use `constexpr` for constants and `inline/template` for functions. Reserve macros for conditional compilation and platform detection.

1.5 What Goes in Headers vs. Source Files

Performance / Detail

In the header (.h)	In the source (.cpp)
Function declarations	Function definitions (bodies)
Class / struct declarations	Member function bodies
<code>inline</code> function bodies	Regular function bodies
<code>template</code> full definitions	Explicit template instantiations
<code>constexpr</code> variables	<code>extern</code> variable definitions
Type aliases, <code>using</code>	Global variable definitions
Templates must be fully defined in headers because the compiler needs the full body at every instantiation site in every TU.	

Chapter 2

The Basics: Types, Namespaces, and `const`

2.1 Integer Types: `int` vs. `int32_t` and Fixed-Width Types

In C++, fundamental integer types like `int`, `long`, and `long long` do not have guaranteed fixed sizes across all platforms.

2.1.1 The Problem with Built-in Integer Types

Concept / Pattern

The Size Guarantees of Standard Integers

- `int`: At least 16 bits (usually 32 on modern hardware).
- `long`: At least 32 bits. On 64-bit Windows: 32 bits; on Linux/macOS: 64 bits!
- `long long`: At least 64 bits.

2.1.2 Fixed-Width Integer Types (`<stdint.h>`)

```
1 #include <stdint.h>
2 int8_t    a; // Exactly 8 bits, signed
3 uint8_t   b; // Exactly 8 bits, unsigned
4 int32_t   e; // Exactly 32 bits, signed (safe replacement for int)
5 int64_t   g; // Exactly 64 bits, signed (safe replacement for long long)
6 uint64_t  h; // Exactly 64 bits, unsigned
```

Key Insight

Best Practice

- Use `int32_t`/`int64_t` when exact size matters.
- Use `size_t` for memory sizes and array indexing.
- Alias: using `i32 = int32_t`; and using `i64 = int64_t`;

2.1.3 What is `size_t` and `ptrdiff_t`?

- `size_t`: Unsigned, 64-bit on 64-bit systems. Never mix with signed integers without casting.
- `ptrdiff_t`: Signed counterpart. Represents the difference between two pointers.

2.2 Namespaces and Scope Resolution

A **namespace** is a named declarative region that provides a scope to avoid *name collisions*.

2.2.1 Why Namespaces Exist

```
1 namespace LibA { void draw() {} }
2 namespace LibB { void draw() {} }
3 LibA::draw();    // Unambiguous
4 LibB::draw();    // Unambiguous
```

2.2.2 The Scope Resolution Operator `::`

Concept / Pattern

Three Uses of `::`

- Namespace access: `std::cout`
- Class member access: `MyClass::staticMethod()`
- Global scope: `::value` — bypasses local shadowing

```
1 int x = 100;
2 void demo() {
3     int x = 5;
4     std::cout << x;    // 5 (local)
5     std::cout << ::x;  // 100 (global)
6 }
```

2.2.3 Nested Namespaces (C++17 Shorthand)

```
1 namespace engine::render::gl { void init() {} }
2 engine::render::gl::init();
```

2.2.4 `using` Declarations vs. Directives

```
1 using std::cout;    // Declaration: imports ONE name
2 using namespace std; // Directive: imports ALL names
```

Common Mistake

`using namespace std;` in Headers is Dangerous

Never put `using namespace std;` in a header — it pollutes every including file. Prefer fully-qualified names in headers.

2.2.5 Anonymous Namespaces (Internal Linkage)

```
1 namespace {
2     int helper = 0;    // Only visible in this .cpp file
3     void helper_fn() {} // Cannot be called from other files
4 }
```

Concept / Pattern

Anonymous Namespace vs. `static`

Feature	<code>static</code>	Anonymous Namespace
Functions	Yes	Yes
Types/classes	No	Yes
Variables	Yes	Yes
Modern C++ preferred	No	Yes

2.2.6 Key `std` Sub-namespaces

- `std::chrono`: `steady_clock`, `duration`.
- `std::filesystem`: `path`, `exists`.
- `std::literals`: `"hello"s`, `1h`, `200ms`.
- `std::placeholders`: `_1`, `_2`, ...
- `__gnu_pbds`: GCC ordered set with order statistics.

2.3 `const` vs. `constexpr`: When and Why

`const` and `constexpr` are both tools for expressing immutability in C++, but they operate at *different stages of the program's lifecycle* and carry very different compiler guarantees.

2.3.1 The Core Difference

Concept / Pattern

`const` vs. `constexpr` at a Glance

Property	<code>const</code>	<code>constexpr</code>
When evaluated	Compile-time <i>or</i> runtime	Compile-time only
Value known at compile-time?	No — can be runtime input	Yes — must be a constant expression
Applied to functions	Member qualifier (read-only this)	Enables compile-time evaluation
Applied to variables	Yes (read-only alias)	Yes (compile-time constant)
Placed in <code>.rodata</code>	Only if initialised with a literal	Always (if <code>constexpr</code> object)

2.3.2 When to Use `const`

Use `const` when a value should not change *after it is set*, but its value might only be known *at runtime*:

```

1 void process(const i32 limit) { // caller decides limit at runtime
2     // limit = 10; // ERROR: cannot modify a const
3     for (i32 i = 0; i < limit; ++i) { ... }
4 }
5
6 const i32 user_input = get_value_from_cin(); // runtime const

```

`const` on a **member function** means “this function does not modify the object’s state”. It changes the hidden **this** pointer from `T*` to `const T*`:

```

1 class Wallet {
2     i32 m_balance;
3 public:
4     i32 getBalance() const { return m_balance; } // read-only: safe to call on const objects
5     void deposit(i32 amount) { m_balance += amount; } // mutating: not const
6 };

```

2.3.3 When to Use `constexpr`

Use `constexpr` when a value or computation is *fully known at compile-time* and you want it baked directly into the binary:

```

1 constexpr i32 MAX_SIZE = 1024; // compile-time constant
2 constexpr double PI = 3.14159265358979; // compile-time constant
3
4 // constexpr function: CAN run at compile-time if args are constants
5 constexpr i32 square(i32 x) { return x * x; }

```

```

6 constexpr i32 s1 = square(5);    // evaluated at compile-time: s1 = 25 (literal)
7 i32 n; std::cin >> n;
8 i32 s2 = square(n);              // evaluated at runtime: n is not a constant

```

Concept / Pattern

`constexpr` Functions are Dual-Mode

A `constexpr` function is not *only* a compile-time function. It is a function that **can** run at compile-time (when given constant arguments) and **falls back** to runtime silently when given non-constant arguments. Use `constexpr` instead if you want to *forbid* the runtime fallback entirely.

2.3.4 Combining `const` and `constexpr` on Member Functions

These two qualifiers serve completely different purposes and can be combined freely. Here is what each position in a member function signature means:

```

1 constexpr i32 getX() const { return m_x; }
2 // ^
3 // |
4 // |      |__ const MEMBER QUALIFIER:
5 // |      "this function does not modify the object"
6 // |__ constexpr SPECIFIER:
7 // "this function CAN be evaluated at compile-time"

```

Common Mistake

These are Completely Independent Axes

- `const` (member qualifier) controls **mutability of the object**.
- `constexpr` (specifier) controls **when the function can run**.
- They do not conflict. You can have one, both, or neither, depending on what you need.

2.3.5 Worked Example: All Four Combinations

The class below Shows every valid combination of `const` and `constexpr` on member functions:

```

1 #include <iostream>
2 using i32 = int32_t;
3
4 class Point {
5     i32 m_x, m_y;
6 public:
7     constexpr Point(i32 x, i32 y) : m_x{x}, m_y{y} {}
8
9     // Combination 1: constexpr + const
10    // Can run at compile-time. Does NOT modify the object.
11    // Most common pattern for getters in constexpr classes.
12    constexpr i32 getX() const { return m_x; }
13    constexpr i32 getY() const { return m_y; }
14
15    // Combination 2: constexpr (no const)
16    // Can run at compile-time. DOES modify the object (C++14+).
17    // Only callable on non-const objects.
18    constexpr void scale(i32 factor) {
19        m_x *= factor;
20        m_y *= factor;
21    }
22
23    // Combination 3: const (no constexpr)

```



```

24 // Runtime only. Does NOT modify the object.
25 // Use when the function body cannot be constexpr (e.g. I/O).
26 void print() const {
27     std::cout << "(" << m_x << ", " << m_y << ")\n";
28 }
29
30 // Combination 4: neither
31 // Runtime only. DOES modify the object.
32 void reset() { m_x = 0; m_y = 0; }
33
34 // Bonus: constexpr + const returning a new object
35 // Pure functional style computes a new Point without mutation.
36 constexpr Point translated(i32 dx, i32 dy) const {
37     return Point{m_x + dx, m_y + dy};
38 }
39 };
40
41 void solve() {
42     // Compile-time usage
43     constexpr Point cp{3, 4};
44     constexpr i32 x = cp.getX(); // constexpr+const: compile-time OK
45     constexpr Point cp2 = cp.translated(1, 1); // constexpr+const: compile-time OK
46
47     // cp.scale(2); // ERROR: scale() modifies the object, cp is const
48
49     // Runtime usage
50     Point p{10, 20};
51     p.scale(3); // constexpr (no const): runtime, mutates p -> (30, 60)
52     p.print(); // const (no constexpr): runtime, read-only -> (30, 60)
53     p.reset(); // neither: runtime, mutates p -> (0, 0)
54     p.print();
55 }

```

Key Insight

Decision Tree for Member Functions

- Does the function **modify this**?
 - **No** $\Rightarrow$ add `const` qualifier.
 - **Yes** $\Rightarrow$ no `const` qualifier.
- Can the function body be evaluated with **only compile-time inputs**?
 - **Yes** (no I/O, no runtime allocations, pure logic) $\Rightarrow$ add `constexpr`.
 - **No** $\Rightarrow$ omit `constexpr`.
- Do you want to **forbid the runtime fallback** of a `constexpr` function?
 - $\Rightarrow$ Use `constexpr` instead.

Common Mistake

`constexpr` Implies `inline`

Every `constexpr` function is implicitly `inline`. This means it can be defined in a header file and included in multiple translation units without violating the One Definition Rule (ODR). This is *not* the case for regular functions.

2.3.6 The Blank-Filling Game: `const/constexpr/empty` on Free Functions

Consider this code skeleton with two blanks to fill:

```

1 ----- int f() {
2     return 42;           // some pure computation
3 }
4 int main() {

```

```

5   ----- auto x = f();
6   }

```

Each blank can be `constexpr`, `const`, or empty ($\emptyset$). This gives **nine combinations**. The table below shows exactly what the compiler does with each one:

Concept / Pattern

All Nine Combinations at a Glance

f() specifier	call-site	Compiles?	When evaluated?	x is...
<code>constexpr</code>	<code>constexpr</code>	✓	Compile-time	CT constant
<code>constexpr</code>	<code>const</code>	✓	Runtime (fallback)	Runtime const
<code>constexpr</code>	$\emptyset$	✓	Runtime (fallback)	Mutable
<code>const</code>	<code>constexpr</code>	ERROR	—	—
<code>const</code>	<code>const</code>	✓	Runtime	Runtime const
<code>const</code>	$\emptyset$	✓	Runtime	Mutable
$\emptyset$	<code>constexpr</code>	ERROR	—	—
$\emptyset$	<code>const</code>	✓	Runtime	Runtime const
$\emptyset$	$\emptyset$	✓	Runtime	Mutable

Group 1 — Function is `constexpr`

```

1  constexpr int f() { return 42; }
2
3  // (A) constexpr call-site: forced compile-time evaluation
4  constexpr auto x = f();    // x == 42, baked into binary as a literal
5
6  // (B) const call-site: constexpr falls back to RUNTIME
7  const auto x = f();        // f() called at runtime, result stored as const int
8
9  // (C) empty call-site: same runtime fallback, x is mutable
10 auto x = f();              // f() called at runtime, x is a plain int

```

Concept / Pattern

Why `constexpr` Falls Back

Declaring `f` as `constexpr` does NOT force it to run at compile-time. It *permits* compile-time evaluation. The compiler is only *forced* to evaluate at compile-time when the result is used in a `constexpr` context (e.g. initialising a `constexpr` variable, a template argument, or an array size). Otherwise it silently generates a normal runtime call. Use `constexpr` to forbid this.

Group 2 — Function has `const` on the Return Type

```

1  const int f() { return 42; } // "const" on a value return type is IGNORED
2
3  // (D) ERROR: f is not constexpr, cannot use result in a constexpr context
4  constexpr auto x = f();    // Compile error: f() is not a constant expression
5
6  // (E) Runtime call, const result -- compiles, but the "const" on f() is redundant
7  const auto x = f();        // x is const int; the "const" on f() did nothing extra
8
9  // (F) Mutable result -- compiles, the "const" on f() is silently discarded
10 auto x = f();              // x is int; compiler may warn about discarding top-level
    const

```

Common Mistake**`const` on a Value Return Type is Useless**

`const int f()` and `int f()` are identical for any caller. A returned *value* (not a reference) is a temporary; you cannot take its address or mutate it anyway, so the `const` qualifier is stripped by the compiler and produces no effect. Most compilers emit a warning like: `warning: type qualifiers ignored on function return type`

Group 3 — Plain Function (no specifier)

```

1 int f() { return 42; }
2
3 // (G) ERROR: f is not constexpr, cannot appear in a constexpr expression
4 constexpr auto x = f(); // Compile error: call to non-constexpr function 'int f()'
5
6 // (H) Runtime const
7 const auto x = f(); // OK: runtime call, x is immutable
8
9 // (I) Plain runtime call
10 auto x = f(); // OK: runtime call, x is mutable

```

Compiler Processing Summary

1. **Parsing `f()`:** The compiler checks whether `f` is marked `constexpr/constexpr`. If not, it is a plain runtime function. `const` on the return value type is stripped immediately.
2. **Parsing the call site:** If `constexpr` appears on `x`, the compiler enters *constant-expression evaluation mode* and demands that `f()` can be evaluated at compile-time. If `f` is not `constexpr`, this is a **hard compile error**.
3. **Forced compile-time evaluation:** Only happens when the call result is bound to a `constexpr` variable, used as a template argument, or used as an array dimension.
4. **Runtime fallback:** A `constexpr` function called without a `constexpr` context is silently compiled as a normal function call — the compiler generates runtime assembly for it.

Key Insight**Quick Decision Rule**

- Want **guaranteed compile-time**? $\Rightarrow$ `constexpr` on *both* the function and the call-site variable.
- Want the function to be **optionally** compile-time? $\Rightarrow$ `constexpr` on the function, call-site as needed.
- Want to **forbid runtime fallback** entirely? $\Rightarrow$ use `constexpr` on the function.
- `const` on a free function's **return type** (not a reference/pointer) $\Rightarrow$ useless, remove it.

2.4 If Cpp: Compile-Time Conditional Branching

Introduced in C++17, `if constexpr` is called "compile-time if". It allows your program to make logical branching decisions while compiling, rather than while executing. The compiler evaluates the expression, retains the true branch, and completely discards the false branch from the final generated machine binary.

The Problem with Regular 'if':

A standard execution-time `if` statement requires BOTH paths to compile successfully, even if one path is logically unreachable for certain data types.

Example of broken regular code:

```

1 template <typename T>

```

```

2 void print_value(T x) {
3     if (std::is_pointer_v<T>) {
4         std::cout << *x << "\n"; // Error if T is a normal int! You can't dereference an int
5     } else {
6         std::cout << x << "\n";
7     }
8     // Result: Fails to compile entirely for print_value(10).
9 }

```

2.4.1 The Solution: `if constexpr`

```

1 template <typename T>
2 void print_value(T x) {
3     if constexpr (std::is_pointer_v<T>) {
4         std::cout << *x << "\n"; // Kept ONLY if T is a pointer
5     } else {
6         std::cout << x << "\n"; // Kept ONLY if T is not a pointer
7     }
8 }

```

If `T` is an `int`, the compiler evaluates `if constexpr` (`false`), vaporizes the pointer branch instantly, and only compiles the `else` block.

2.4.2 Rules for Usage

1. The condition must be a true compile-time constant expression (e.g., `sizeof(T) > 4`).
2. Discarded branches must still be syntactically correct (no random text or broken symbols).

Concept / Pattern

`if constexpr` Quick Rules

- The condition **must** be a compile-time constant (`constexpr bool`, `sizeof`, type traits).
- The discarded branch is **not instantiated** but must still be syntactically valid.
- Can be nested: `if constexpr (...) { if constexpr (...) { ... } }`.
- Cannot be used outside a template context unless the condition is already a constant.

Chapter 3

Literals, Type Aliases, Type Deduction, and auto

3.1 Literals and Magic Numbers

A **literal** is a fixed value written directly in source code. Using raw numbers scattered through code is called **magic numbers** — avoid them. Name everything.

Concept / Pattern

```
1 // BAD: magic numbers -- what do 3.14159 and 86400 mean?
2 double area = 3.14159 * r * r;
3 int seconds = 86400;
4
5 // GOOD: named constants
6 constexpr double PI = 3.14159265358979;
7 constexpr int SECS_PER_DAY = 86400;
8 double area = PI * r * r;
9 int seconds = SECS_PER_DAY;
```

3.1.1 Integer Literal Suffixes

```
1 int a = 42; // plain int
2 long b = 42L; // long
3 long long c = 42LL; // long long
4 unsigned d = 42u; // unsigned int
5 unsigned long long e = 42ULL;
6
7 // Different bases
8 int hex = 0xFF; // hexadecimal (255)
9 int oct = 0777; // octal (511)
10 int bin = 0b1010; // binary (C++14): 10
11 int big = 1'000'000; // digit separators (C++14): one million
```

3.1.2 Floating-Point Literal Suffixes

```
1 double a = 3.14; // double (default)
2 float b = 3.14f; // float
3 long double c = 3.14L; // long double
4
5 double sci = 6.02e23; // 6.02 * 10^23 (scientific notation)
6 double neg = 1.5e-4; // 0.00015
```

3.1.3 Character and String Literals

```

1 char    c = 'A';           // single char, single quotes
2 char    nl = '\n';         // escape sequence: newline
3 char    tab = '\t';        // tab
4 char    bs = '\\';         // backslash
5 char    qt = '\'';         // single quote
6
7 // String literals (double quotes) -- C-style, null terminated
8 const char* s = "hello"; // type: const char[6] including '\0'
9
10 // std::string literal (C++14): auto becomes std::string, not const char*
11 using namespace std::string_literals;
12 auto str = "hello"s;       // std::string
13
14 // std::string_view literal (C++17)
15 using namespace std::string_view_literals;
16 auto sv = "hello"sv;      // std::string_view
17
18 // Raw string literals: no escapes needed
19 const char* path = R"(C:\Users\sahil\Desktop)"; // backslashes literal
20 const char* multi = R"(
21     line 1
22     line 2
23 )"; // multi-line, no \n needed

```

3.2 Type Aliases: using and typedef

Type aliases give a type a new name. Makes code more readable and easier to change.

Concept / Pattern

```

1 // Modern C++17 way: using
2 using Score = int;
3 using StringVec = std::vector<std::string>;
4 using Callback = std::function<void(int)>;
5
6 Score s = 100;
7 StringVec names = {"Alice", "Bob"};
8
9 // Old C way: typedef (avoid in new code)
10 typedef int Score; // same as above
11 typedef void (*FuncPtr)(int, double); // function pointer type alias
12
13 // Template aliases (only possible with 'using')
14 template <typename T>
15 using Vec = std::vector<T>;
16 Vec<int> numbers;
17 Vec<double> prices;

```

3.3 auto: Let the Compiler Deduce Types

auto tells the compiler to figure out the type from the initializer. It's not dynamic typing — the type is fixed at compile time, you just don't write it out.

Concept / Pattern

```

1 auto x = 5;           // int
2 auto y = 3.14;        // double
3 auto z = 3.14f;       // float
4 auto s = std::string("hello"); // std::string
5 auto v = std::vector<int>{1, 2, 3}; // std::vector<int>
6
7 // For loop: avoid writing the full iterator type
8 for (auto& elem : my_vector) { /* ... */ }
9 for (const auto& [key, val] : my_map) { /* ... */ }
10
11 // Function return type deduction (C++14)
12 auto add(int a, int b) { return a + b; } // deduced as int
13
14 // Trailing return type (required when return type depends on params)
15 auto divide(double a, double b) -> double { return a / b; }

```

Common Mistake

auto drops const and references by default. If you want to keep them, write them explicitly:

```

1 const std::string& get_name();
2
3 auto      name1 = get_name(); // std::string (copy! const& dropped)
4 const auto& name2 = get_name(); // const std::string& (correct)
5 auto&&      name3 = get_name(); // forwarding ref: binds to anything

```

3.3.1 decltype: Query a Type Without Evaluating

```

1 int x = 5;
2 decltype(x) y = 10; // y is int (same type as x)
3 decltype(x + 3.0) z = 0.0; // z is double (type of x+3.0 expression)
4
5 // Useful in templates when you don't know the return type
6 template <typename A, typename B>
7 auto multiply(A a, B b) -> decltype(a * b) {
8     return a * b;
9 }

```


Chapter 4

Operators In Depth

4.1 Arithmetic Operators and Integer Division

Concept / Pattern

```
1 // Integer division truncates towards zero
2 7 / 2 == 3    // NOT 3.5
3 -7 / 2 == -3  // NOT -4
4
5 // Modulo: sign follows the dividend (left operand) in C++
6 7 % 2 == 1
7 -7 % 2 == -1  // NOT 1
8 7 % -2 == 1
9
10 // To get floating-point division from integers, cast first
11 double result = static_cast<double>(7) / 2; // 3.5
12
13 // Overflow: signed integers wrap with UNDEFINED BEHAVIOUR
14 // Unsigned integers wrap with well-defined behaviour
15 int x = INT_MAX;
16 x = x + 1;           // UB! do NOT do this
17 unsigned u = UINT_MAX;
18 u = u + 1;           // 0 (well-defined)
```

4.2 Increment / Decrement: Pre vs. Post

```
1 int a = 5;
2 int b = ++a;    // pre-increment: a becomes 6, THEN b = 6
3 int c = a++;    // post-increment: c = 6 (old value), THEN a becomes 7
4
5 // Prefer pre-increment (++i) for non-trivial types like iterators
6 // Post-increment must create a temporary copy, pre-increment doesn't
7 for (auto it = v.begin(); it != v.end(); ++it) { /* ... */ }
```

4.3 Bitwise Operators

Concept / Pattern

```

1 // & bitwise AND    | bitwise OR    ^ bitwise XOR    ~ bitwise NOT
2 // << left shift    >> right shift
3
4 unsigned a = 0b1010; // 10
5 unsigned b = 0b1100; // 12
6
7 a & b // 0b1000 = 8    (AND: both bits must be 1)
8 a | b // 0b1110 = 14  (OR: at least one bit is 1)
9 a ^ b // 0b0110 = 6    (XOR: bits differ)
10 ~a    // 0b...10101    (NOT: flip all bits, result is ~10 = -11 for signed)
11 a << 1 // 0b10100 = 20 (left shift: multiply by 2)
12 a >> 1 // 0b0101 = 5   (right shift: divide by 2, but implementation-defined for
    negative signed)

```

4.3.1 Common Bit Manipulation Tricks

```

1 unsigned flags = 0;
2 unsigned BIT0 = 1 << 0; // 0001
3 unsigned BIT1 = 1 << 1; // 0010
4 unsigned BIT3 = 1 << 3; // 1000
5
6 flags |= BIT0;           // SET bit 0
7 flags &= ~BIT1;          // CLEAR bit 1
8 flags ^= BIT3;           // TOGGLE bit 3
9 bool is_set = (flags & BIT0) != 0; // CHECK bit 0
10
11 // Check if a number is a power of 2
12 bool pow2 = (n > 0) && (n & (n - 1)) == 0;
13
14 // Get lowest set bit
15 int lowest = n & (-n);
16
17 // Clear lowest set bit
18 n &= (n - 1);

```

Performance / Detail

Use `std::bitset<N>` from `<bitset>` when you need to work with a fixed set of bits but want a nicer interface:

```

1 #include <bitset>
2 std::bitset<8> b("11001010"); // from string
3 b.set(0);           // set bit 0
4 b.reset(3);         // clear bit 3
5 b.flip(7);          // toggle bit 7
6 b.test(5);          // bool: is bit 5 set?
7 b.count();          // number of set bits
8 b.to_ulong();       // convert to unsigned long
9 std::cout << b;     // print as "11001010"

```

4.4 The Conditional Operator ?:

```

1 int x = 5;
2 // condition ? value_if_true : value_if_false
3 int abs_x = (x >= 0) ? x : -x;
4 std::cout << ((x > 0) ? "positive" : "not positive");
5

```

```
6 // Works inline in expressions -- great for simple cases
7 // Avoid nesting ternaries: a ? b : c ? d : e is unreadable
```

4.5 The Comma Operator

```
1 // Evaluates left then right, returns right. Rarely needed.
2 int x = (++a, a + 1); // a is incremented, then x = a + 1
3 // You'll see it most in for-loop multiple-update expressions:
4 for (int i = 0, j = 10; i < j; ++i, --j) { /* ... */ }
```

4.6 sizeof Operator

```
1 sizeof(int)      // 4 bytes (on most platforms)
2 sizeof(double)   // 8 bytes
3 sizeof(char)     // 1 byte (guaranteed by the standard)
4 sizeof(int[5])   // 20 bytes (5 * 4)
5
6 int arr[10];
7 size_t count = sizeof(arr) / sizeof(arr[0]); // 10 (element count)
8 // WARNING: sizeof on a pointer gives pointer size (8 on 64-bit), NOT array size!
9 void broken(int* arr) {
10     sizeof(arr); // 8: size of the pointer, not the array!
11 }
```


Chapter 5

Control Flow: If, Switch, Loops, and Halts

5.1 If, Else, and If-Initializer

Concept / Pattern

```
1 // Basic if/else
2 if (x > 0) { /* ... */ }
3 else if (x < 0) { /* ... */ }
4 else { /* ... */ }
5
6 // C++17: if with initializer -- scopes the variable to the if block
7 if (auto it = map.find("key"); it != map.end()) {
8     // it is only available inside this if/else block
9     std::cout << it->second;
10 }
11
12 // constexpr if -- compile-time branching (see template chapter)
13 if constexpr (sizeof(T) > 4) { /* ... */ }
```

Common Mistake

Watch out for assignment in conditions:

```
1 int x = 0;
2 if (x = 5) { /* ... */ } // assigns 5, ALWAYS true -- probably a bug!
3 if (x == 5) { /* ... */ } // comparison -- what you meant
4 // Turn on -Wall to get warned about this.
```

5.2 Switch Statements

Concept / Pattern

```

1 int choice = 2;
2
3 switch (choice) {           // expression must be integral or enum
4     case 1:
5         std::cout << "one\n";
6         break;              // MUST break or fall through to next case!
7     case 2:
8         std::cout << "two\n";
9         break;
10    case 3:
11    case 4:                  // cases 3 and 4 share the same body
12        std::cout << "three or four\n";
13        break;
14    default:                // optional catch-all
15        std::cout << "other\n";
16        break;
17 }

```

Common Mistake

Forgetting a `break` causes **fallthrough** to the next case. This is occasionally intentional but usually a bug. If you want intentional fallthrough, annotate it:

```

1 case 1:
2     do_something();
3     [[fallthrough]];       // C++17: tells compiler/reader this is intentional
4 case 2:
5     do_more();
6     break;

```

Variables can't be initialized in case labels without a block:

```

1 case 1:
2     int x = 5; // ERROR: jump over initialization of x
3     break;
4 case 2: {         // FIXED: wrap in a block
5     int x = 5;
6     break;
7 }

```

5.3 Loops

5.3.1 While Loop

```

1 int i = 0;
2 while (i < 10) {           // condition checked BEFORE each iteration
3     std::cout << i++;
4 }
5
6 // Infinite loop pattern
7 while (true) {
8     if (done) break;
9 }

```

5.3.2 Do-While Loop

```

1 int x;
2 do {

```

```

3     std::cout << "Enter a positive number: ";
4     std::cin >> x;
5 } while (x <= 0);    // condition checked AFTER first iteration
6 // Guaranteed to execute at least once

```

5.3.3 For Loop

```

1 // init; condition; update
2 for (int i = 0; i < 10; ++i) {
3     std::cout << i << ' ';
4 }
5
6 // Multiple variables
7 for (int i = 0, j = 10; i < j; ++i, --j) { }
8
9 // Can omit any part (empty condition = infinite)
10 for (;;) { /* infinite loop */ }
11
12 // For with index and value together (C++17)
13 for (size_t i = 0; i < v.size(); ++i) {
14     std::cout << i << ": " << v[i] << '\n';
15 }

```

5.3.4 Range-Based For Loop

```

1 std::vector<int> nums = {1, 2, 3, 4, 5};
2
3 for (int x : nums) { }           // copy of each element
4 for (const int& x : nums) { }    // const reference (read-only, no copy)
5 for (int& x : nums) { x *= 2; }  // reference: modifies the original
6 for (auto& x : nums) { }         // auto&: let compiler deduce type
7
8 // Works on any range: arrays, strings, maps, initializer lists
9 for (char c : "hello") { }
10 for (const auto& [k, v] : my_map) { }

```

5.3.5 Break, Continue, and Return

```

1 for (int i = 0; i < 10; ++i) {
2     if (i == 3) continue;    // skip this iteration, go to i=4
3     if (i == 7) break;       // exit the loop entirely
4     std::cout << i << ' ';   // prints: 0 1 2 4 5 6
5 }
6
7 // Return exits the entire function (not just the loop)
8 int find_first(const std::vector<int>& v, int target) {
9     for (int i = 0; i < (int)v.size(); ++i)
10         if (v[i] == target) return i;
11     return -1; // not found
12 }

```

5.4 Program Termination

Performance / Detail

Function	What it does
<code>std::exit(code)</code>	Normal exit. Calls static destructors, flushes streams. Returns <code>code</code> to OS (0 = success).
<code>std::quick_exit(code)</code>	Exits without calling destructors or flushing. Calls <code>at_quick_exit</code> handlers.
<code>std::abort()</code>	Abnormal termination. No cleanup at all. Used to signal a bug.
<code>std::terminate()</code>	Called when an exception is unhandled. Default behaviour: calls <code>abort()</code> .

Common Mistake

`std::abort()` is not the same as `assert(false)`. `assert()` only fires in debug builds (disabled when `NDEBUG` is defined). `std::abort()` always fires.

Chapter 6

Functions In Depth

6.1 Default Arguments

Concept / Pattern

```
1 // Default values must go at the END of the parameter list
2 void print(std::string_view msg, int times = 1, char sep = '\n') {
3     for (int i = 0; i < times; ++i)
4         std::cout << msg << sep;
5 }
6
7 print("hello");           // times=1, sep='\n'
8 print("hello", 3);        // times=3, sep='\n'
9 print("hello", 3, ' ');   // all specified
10
11 // Defaults go in the DECLARATION (header), not the definition (source file)
12 // Don't repeat them in both -- that's a redeclaration error.
```

6.2 Function Overloading

Same function name, different parameter types. The compiler picks the best match at the call site.

```
1 void print(int x)          { std::cout << "int: " << x; }
2 void print(double x)      { std::cout << "double: " << x; }
3 void print(std::string x) { std::cout << "string: " << x; }
4 void print(int x, int y)  { std::cout << x << ", " << y; }
5
6 print(5);                 // print(int)
7 print(3.14);              // print(double)
8 print("hi");              // print(string) -- "hi" converts to std::string
9 print(1, 2);              // print(int, int)
```

Common Mistake

Overload resolution is done purely on **parameters**, not return type. Two functions that differ only in return type are NOT valid overloads — they're a redefinition error.

```
1 int get() { return 1; }
2 double get() { return 1.0; } // COMPILER ERROR: redefinition
```

6.3 Inline Functions

Performance / Detail

`inline` on a function primarily means: “the linker should merge multiple identical definitions of this, not error”. It’s how you safely define functions in header files.

The compiler decides whether to actually inline the function body at the call site — `inline` is just a hint, not a command. Modern compilers often inline functions without the keyword, and ignore it when it’s present.

```
1 // Safe to put in a header file
2 inline int square(int x) { return x * x; }
3
4 // In C++17, inline variables in headers are also safe
5 inline constexpr double PI = 3.14159265358979;
```

6.4 Assertions: Catching Bugs Early

Concept / Pattern

```
1 #include <cassert>
2
3 // assert(expr): if expr is false, prints a message and calls std::abort()
4 // ONLY active when NDEBUG is NOT defined (i.e., debug builds)
5 assert(ptr != nullptr);
6 assert(index >= 0 && index < size);
7
8 // static_assert(expr, msg): compile-time assertion, always active
9 static_assert(sizeof(int) == 4, "Need 32-bit ints");
10 static_assert(std::is_trivially_copyable_v<MyStruct>);
11
12 // In release builds (-DNDEBUG), asserts compile to nothing (zero cost)
13 // Use asserts to document preconditions and catch programming errors, NOT
14 // runtime errors (like bad user input). Use exceptions for those.
```

6.5 Recursive Functions

A function that calls itself. Every recursive function needs a **base case** to stop recursion, otherwise you get a stack overflow.

```
1 // Factorial: n! = n * (n-1)!
2 int factorial(int n) {
3     if (n <= 1) return 1; // BASE CASE: stop recursion
4     return n * factorial(n - 1); // RECURSIVE CASE
5 }
6 factorial(5); // 5 * 4 * 3 * 2 * 1 = 120
7
8 // Fibonacci
9 int fib(int n) {
10     if (n <= 1) return n;
11     return fib(n-1) + fib(n-2); // WARNING: exponential time! Use DP.
12 }
13
14 // Power of 2: check via recursion
15 bool is_power_of_2(int n) {
16     if (n == 1) return true;
17     if (n % 2 != 0 || n <= 0) return false;
18     return is_power_of_2(n / 2);
19 }
```

Key Insight

Recursive functions are elegant for tree/graph problems. But deep recursion (thousands of levels) can blow the stack. Convert deep recursion to iteration with an explicit stack (`std::stack`) when needed. **Memoization** (caching results): convert recursive Fibonacci from $O(2^n)$ to $O(n)$ by remembering past results.

```

1 std::map<int, long long> memo;
2 long long fib_memo(int n) {
3     if (n <= 1) return n;
4     if (memo.count(n)) return memo[n];
5     return memo[n] = fib_memo(n-1) + fib_memo(n-2);
6 }

```

6.6 Function Pointers

A function pointer stores the address of a function. Lets you pass functions as arguments and call them later.

```

1 // Declare: return_type (*name)(param_types...)
2 int add(int a, int b) { return a + b; }
3 int mul(int a, int b) { return a * b; }
4
5 int (*op)(int, int); // function pointer named 'op'
6 op = add;           // point to add
7 op(3, 4);           // calls add(3, 4) = 7
8 op = mul;           // now points to mul
9 op(3, 4);           // calls mul(3, 4) = 12
10
11 // Pass as argument
12 void apply_twice(int x, int (*func)(int, int), int y) {
13     std::cout << func(x, y) << '\n';
14 }
15 apply_twice(3, add, 4); // 7
16
17 // Type alias makes it readable
18 using BinaryOp = int(*)(int, int);
19 BinaryOp f = mul;

```

6.7 std::function: Type-Erased Callable

`std::function<Ret(Args...)>` can hold any callable: function pointer, lambda, functor, member function via `std::bind`. More flexible than raw function pointers, but has runtime overhead (heap allocation, virtual dispatch internally).

```

1 #include <functional>
2
3 std::function<int(int, int)> op;
4
5 op = add; // function pointer
6 op = [](int a, int b) { return a - b; }; // lambda
7 op = std::multiplies<int>{}; // functor
8
9 // Great for callbacks and storing heterogeneous callables
10 std::vector<std::function<void()>> callbacks;
11 callbacks.push_back([]{ std::cout << "A\n"; });
12 callbacks.push_back([]{ std::cout << "B\n"; });
13 for (auto& cb : callbacks) cb();

```

When to Use

Prefer raw function pointers or templates when performance matters. Use `std::function` when you need to store callables in containers, pass them across translation units, or the exact type isn't known at compile time.

Chapter 7

I/O, Streams, and Fast I/O

7.1 I/O in C++: `cin/cout`, Streams, and `FastIO`

7.1.1 The C++ I/O Stack: What Actually Happens on `cout << x`

Every `cout << x` you write passes through **four layers** before a single byte reaches the terminal:

1. **Formatting layer** — `std::ostream` converts `x` to text (locale, width, fill...).
2. **Stream buffer** (`std::streambuf`) — accumulates characters in a memory buffer; calls the OS only when the buffer is full or explicitly flushed.
3. **C `stdio` sync** — by default the C++ stream buffer delegates writes through `printf`'s buffer so that `cout` and `printf` can be safely mixed.
4. **OS syscall** (`write(1, ...)`) — the actual kernel call that moves data from user-space to the terminal/pipe.

Performance / Detail

The Bottleneck

The OS syscall is **orders of magnitude slower** than copying bytes in user-space. Every call crosses the kernel boundary, which takes hundreds of nanoseconds. Good I/O design minimizes the number of syscalls by maximizing the amount of data sent per call — i.e., buffering.

7.1.2 `std::ios::sync_with_stdio(false)`

What Synchronization Means

C originally used `printf/scanf`. C++ added its own stream layer. By default the two **share the same underlying buffer** so that code mixing both styles produces consistent output:

```
1 printf("Hello ");
2 std::cout << "World\n"; // safe to mix -- both go through stdio's buffer
```

Maintaining this guarantee forces `std::cout` to **flush its own buffer into `stdio`'s buffer before every operation**. That is the synchronization cost.

Disabling Synchronization

```
1 std::ios::sync_with_stdio(false);
```

This one call tells the C++ runtime: “stop delegating through `stdio`; use your own independent buffer.” The C++ stream now accumulates data in its own buffer and only calls `write()` when that buffer fills — far fewer syscalls.

Common Mistake**The Trade-off**

After calling `sync_with_stdio(false)`, **never mix** `printf/scanf` with `cout/cin` in the same program. The two buffers are now independent; interleaving them produces garbled output. In competitive programming this is fine — you use one or the other exclusively.

Concept / Pattern**What it Does NOT Do**

`sync_with_stdio(false)` does **not** remove the formatting overhead of `operator<<`. It only removes the stdio delegation cost. `cout` still parses locale, handles fill/width, and maintains stream state on every call.

7.1.3 `std::cin.tie(nullptr)`

By default `std::cin` is **tied** to `std::cout`. Before every `cin` read, the runtime automatically calls `cout.flush()` to ensure any pending output (like a prompt) is visible before the user types input:

```
1 std::cout << "Enter a number: "; // buffered, not yet printed
2 i32 x; std::cin >> x;           // tie causes cout to flush first -- prompt appears
```

For competitive programming there are no interactive prompts, so this auto-flush is pure waste:

```
1 std::cin.tie(nullptr); // untie: cin no longer forces cout to flush
```

Key Insight**The Standard CP Macro**

```
1 #define After_Dark std::ios::sync_with_stdio(false); std::cin.tie(NULL)
```

These two lines together make `cin/cout` roughly **5–10x faster** than their defaults. They are the minimum setup for any competitive programming submission.

7.1.4 The `istream` / `ostream` Class Hierarchy**Performance / Detail****The Inheritance Chain**

```
1 std::ios_base           // flags, state bits, locale -- no read/write
2   std::basic_ios       // stream state, tie, fill
3     std::istream       // operator>> overloads for all built-in types
4       std::ifstream    // file input
5       std::istringstream // string input
6     std::ostream       // operator<< overloads for all built-in types
7       std::ofstream    // file output
8       std::ostringstream // string output
9     std::iostream      // inherits both istream and ostream
10       std::fstream     // file input + output
11       std::stringstream // string input + output
```

`std::cin` is a global instance of `std::istream` connected to file descriptor 0 (stdin).

`std::cout` is a global instance of `std::ostream` connected to file descriptor 1 (stdout).

`std::cerr` is an **unbuffered** `std::ostream` on fd 2 — every write flushes immediately.
`std::clog` is a **buffered** `std::ostream` on fd 2 — more efficient for logging.

How operator» Works Internally

When you write `cin >> x` for `int x`, the stream:

1. Skips whitespace (space, tab, newline) via the `skipws` flag (on by default).
2. Reads characters until a non-digit is encountered.
3. Converts the character sequence to an integer using the current locale.
4. Sets `failbit` if no valid integer was found.

7.1.5 Stream State Bits

Every stream carries a set of error flags:

Flag	Check with	Meaning
<code>goodbit</code>	<code>s.good()</code>	No errors, I/O can proceed
<code>eofbit</code>	<code>s.eof()</code>	End of stream reached
<code>failbit</code>	<code>s.fail()</code>	Logical error (e.g. reading “abc” into an <code>int</code>)
<code>badbit</code>	<code>s.bad()</code>	Unrecoverable I/O error (e.g. disk failure)

```

1  i32 x;
2  std::cin >> x;
3  if (std::cin.fail()) {
4      std::cin.clear();           // reset the failbit
5      std::cin.ignore(1000, '\n'); // discard the bad input line
6  }
7
8  // Read until EOF:
9  while (std::cin >> x) { // operator>> returns the stream; falsy when !good()
10     process(x);
11 }
```

7.1.6 String Streams

String streams let you use the same `>>/<<` interface on an in-memory string instead of `stdin/stdout` — great for parsing and formatting.

`std::istringstream` — Parse a String Like `cin`

```

1  #include <sstream>
2
3  std::string line = "42 3.14 hello";
4  std::istringstream iss(line);
5
6  i32 a; double b; std::string c;
7  iss >> a >> b >> c;
8  // a = 42, b = 3.14, c = "hello"
```

Common pattern: read a whole line then parse it token by token:

```

1  std::string line;
2  std::getline(std::cin, line);
3  std::istringstream iss(line);
4  i32 x;
5  while (iss >> x) v.push_back(x); // splits the line into integers
```

std::ostringstream — Build a String Like cout

```

1 std::ostringstream oss;
2 oss << "Player " << name << " scored " << score << " points.";
3 std::string result = oss.str(); // extract the built string

```

std::stringstream — Read and Write

```

1 std::stringstream ss;
2 ss << 100 << " " << 3.14; // write
3 i32 a; double b;
4 ss >> a >> b; // read back

```

Key Insight**Fast int-to-string / string-to-int (C++11+)**

Prefer `std::to_string(x)` and `std::stoi(s)` over `stringstream` for simple conversions — they are shorter and faster. Use `ostringstream` only when you need formatted/multi-field output.

7.1.7 File I/O

```

1 #include <fstream>
2
3 // --- Reading from a file ---
4 std::ifstream fin("input.txt");
5 if (!fin.is_open()) { /* handle error */ }
6
7 i32 x;
8 while (fin >> x) process(x); // reads until EOF
9 fin.close(); // optional -- closes on destruction (RAII)
10
11 // --- Writing to a file ---
12 std::ofstream fout("output.txt");
13 fout << "Answer: " << 42 << '\n';
14 // fout destructor flushes and closes automatically
15
16 // --- Redirect cin/cout to files (CP trick) ---
17 #ifndef ONLINE_JUDGE
18     freopen("input.txt", "r", stdin);
19     freopen("output.txt", "w", stdout);
20 #endif
21 // Now cin/cout read from/write to the files transparently

```

Open Modes

Mode Flag	Meaning
<code>ios::in</code>	Open for reading (default for <code>ifstream</code>)
<code>ios::out</code>	Open for writing (default for <code>ofstream</code>); truncates file
<code>ios::app</code>	Always append; never truncate
<code>ios::trunc</code>	Truncate to zero length on open
<code>ios::binary</code>	Raw binary mode; no newline translation
<code>ios::ate</code>	Seek to end immediately after open

```

1 // Append to an existing log file
2 std::ofstream log("log.txt", std::ios::app);
3 log << "[INFO] starting\n";
4
5 // Binary read/write
6 std::fstream bin("data.bin", std::ios::in | std::ios::out | std::ios::binary);

```


7.1.8 Why FastIO? The Remaining Bottleneck

Even with `sync_with_stdio(false)` and `cin.tie(nullptr)`, `cin/cout` still:

- Run through a **locale** layer on every character (internationalisation support you never use).
- Go through `streambuf::sgetc()/sputc()` virtual function calls — one virtual dispatch per character.
- Maintain **stream state flags** (`failbit`, `eofbit`) checked on every operation.

FastIO bypasses all of this by talking directly to the OS:

- **Input:** one `read(0, buf, 64KB)` syscall fills 64 KB; subsequent reads are just array indexing.
- **Output:** bytes are written into a local array; one `write(1, buf, 64KB)` syscall when full.
- **No locale, no virtual calls, no state flags.**

Concept / Pattern

When Does It Matter?

For problems with 10^6 – 10^7 integers to read/write, FastIO can be **3–8x faster** than optimised `cin/cout`. For problems with small I/O ($< 10^5$ values), the difference is invisible.

7.1.9 FastIO: Complete Deep Dive

Buffer Layout

```
1 static const size_t BS = 1 << 16; // 65536 bytes = 64 KB
2 char ibuf[BS]; // input buffer -- filled in one syscall, drained byte-by-byte
3 char obuf[BS]; // output buffer -- filled byte-by-byte, flushed in one syscall
4 size_t iptr = 0; // next unread byte in ibuf
5 size_t icnt = 0; // how many bytes are currently valid in ibuf
6 size_t optr = 0; // next free slot in obuf
```

Performance / Detail

Why 64 KB?

Most OS kernels use 4 KB memory pages. 64 KB = 16 pages, which is large enough to amortise the syscall overhead across tens of thousands of characters, but small enough to sit comfortably in L2 cache (typically 256 KB – 1 MB). Larger buffers do not help much and waste cache.

`next_char()`: Refilling the Input Buffer

```
1 inline char next_char() {
2     if (iptr >= icnt) { // buffer is exhausted
3         icnt = read(0, ibuf, BS); // syscall: fd 0 = stdin, read up to 64 KB
4         iptr = 0;
5         if (icnt == 0) return EOF; // EOF or error
6     }
7     return ibuf[iptr++]; // return next byte, advance pointer
8 }
```

`read(0, ibuf, BS)` is a raw POSIX syscall. It blocks until data is available and returns the number of bytes actually read (which may be less than BS at the last read). `icnt == 0` means end of file.

`push_char()`: Writing to the Output Buffer

```
1 inline void push_char(char c) {
2     if (optr >= BS) { // buffer is full
3         size_t w = write(1, obuf, optr); // syscall: fd 1 = stdout, flush all
4         (void)w; // suppress unused-variable warning
5         optr = 0; // reset write pointer
6     }
```

```

7   obuf[optr++] = c;           // store byte, advance pointer
8 }

```

The `(void)w` is a deliberate no-op. In production code you would check `w` for write errors, but in competitive programming it is ignored to keep the code simple.

Destructor: Flushing the Remaining Output

```

1 ~FastIO() {
2     if (optr > 0) {
3         size_t w = write(1, obuf, optr); // flush whatever is left
4         (void)w;
5     }
6 }

```

Because `fio` is a global object, its destructor runs **after `main()` returns**. Any output not yet flushed (because the buffer never filled up) is sent here. Without this, the last chunk of output would be silently lost.

`operator>>(T& num): Reading an Integer`

```

1 template <typename T>
2 inline FastIO& operator>>(T& num) {
3     num = 0;
4     char c = next_char();
5     bool neg = false;
6     // Phase 1: skip non-digits (whitespace, newlines), detect '-'
7     while (c != EOF && (c < '0' || c > '9')) {
8         if (c == '-') neg = true;
9         c = next_char();
10    }
11    // Phase 2: accumulate digits
12    while (c >= '0' && c <= '9') {
13        num = num * 10 + (c - '0');
14        c = next_char();
15    }
16    if (neg) num = -num;
17    return *this; // enables chaining: fio >> a >> b
18 }

```

Concept / Pattern

Why `c - '0'` Works

In ASCII, the digits `'0'` through `'9'` have consecutive values 48–57. Subtracting `'0'` (48) converts the character to its integer value: `'3' - '0' == 3`. This works without any library call.

`operator>>(std::string&): Reading a Token`

```

1 inline FastIO& operator>>(std::string& s) {
2     s.clear();
3     char c = next_char();
4     // skip leading whitespace (space, tab, newline, carriage return)
5     while (c != EOF && (c <= ' ' || c == '\n' || c == '\r')) c = next_char();
6     // accumulate until whitespace
7     while (c != EOF && c > ' ') {
8         s.push_back(c);
9         c = next_char();
10    }
11    return *this;
12 }

```

The condition `c > ' '` is a compact way of checking “printable non-space character”: all printable ASCII characters have values `> 32` (space).

operator<<(T num): Writing an Integer

```

1 template <typename T>
2 inline FastIO& operator<<(T num) {
3     if (num == 0) { push_char('0'); return *this; }
4     if (num < 0) { push_char('-'); num = -num; }
5     char temp[32]; size_t idx = 0;
6     // Build digits in reverse (least significant first)
7     while (num > 0) {
8         temp[idx++] = (num % 10) + '0';
9         num /= 10;
10    }
11    // Emit in correct order (most significant first)
12    while (idx > 0) push_char(temp[--idx]);
13    return *this;
14 }

```

Dividing by 10 and taking the remainder naturally produces digits from least significant to most significant. The 32-byte temp array holds the reversed digits; they are then emitted in reverse order to produce the correct output.

Overloads for const char* and char

```

1 inline FastIO& operator<<(const char* s) {
2     while (*s) push_char(*s++); // null-terminated: loop until '\0'
3     return *this;
4 }
5 inline FastIO& operator<<(char c) {
6     push_char(c);
7     return *this;
8 }

```

Complete Annotated Class

```

1 class FastIO {
2 private:
3     static const size_t BS = 1 << 16; // 64 KB buffers
4     char ibuf[BS], obuf[BS];
5     size_t iptr = 0, icnt = 0, optr = 0;
6
7     inline char next_char() {
8         if (iptr >= icnt) {
9             icnt = read(0, ibuf, BS); // one syscall per 64 KB
10            iptr = 0;
11            if (icnt == 0) return EOF;
12        }
13        return ibuf[iptr++];
14    }
15    inline void push_char(char c) {
16        if (optr >= BS) {
17            size_t w = write(1, obuf, optr); (void)w; // flush when full
18            optr = 0;
19        }
20        obuf[optr++] = c;
21    }
22 public:
23     ~FastIO() {
24         if (optr > 0) { size_t w = write(1, obuf, optr); (void)w; }
25     }
26     template <typename T> inline FastIO& operator>>(T& num) {
27         num = 0; char c = next_char(); bool neg = false;
28         while (c != EOF && (c < '0' || c > '9')) { if (c == '-') neg = true; c = next_char(); }
29         while (c >= '0' && c <= '9') { num = num * 10 + (c - '0'); c = next_char(); }
30         if (neg) num = -num; return *this;

```

```

31 }
32 inline FastIO& operator>>(std::string& s) {
33     s.clear(); char c = next_char();
34     while (c != EOF && (c <= ' ' || c == '\n' || c == '\r')) c = next_char();
35     while (c != EOF && c > ' ') { s.push_back(c); c = next_char(); }
36     return *this;
37 }
38 template <typename T> inline FastIO& operator<<(T num) {
39     if (num == 0) { push_char('0'); return *this; }
40     if (num < 0) { push_char('-'); num = -num; }
41     char temp[32]; size_t idx = 0;
42     while (num > 0) { temp[idx++] = (num % 10) + '0'; num /= 10; }
43     while (idx > 0) push_char(temp[--idx]);
44     return *this;
45 }
46 inline FastIO& operator<<(const char* s) { while (*s) push_char(*s++); return *this; }
47 inline FastIO& operator<<(char c) { push_char(c); return *this; }
48 } fio; // global instance -- destructor runs after main() exits

```

Usage

```

1 // Replace cin/cout with fio:
2 i32 n; fio >> n;
3 customVector1D<i32> a{n};
4 for (i32 i = 0; i < n; i++) fio >> a[i];
5
6 i64 ans = solve(a);
7 fio << ans << '\n';
8
9 // Chaining works exactly like cin/cout:
10 i32 x, y; fio >> x >> y;
11 fio << x + y << '\n';

```

Common Mistake

FastIO Does Not Support double/float

The integer `operator>>` template will produce garbage for floating-point types because it reads digits only and ignores the decimal point. Do not use `fio >> d` where `d` is a double. Fall back to `scanf("%lf", &d)` or `cin >> d` for floating-point input.

Concept / Pattern

FastIO vs cin/cout: Decision Rule

- $\leq 10^5$ values: `cin/cout` with `After_Dark` is fine.
- 10^6 – 10^7 integers, tight time limit: switch to `FastIO`.
- Floating-point or formatted output (e.g. `setprecision`): use `printf/scanf` or `cin/cout`.

Chapter 8

Strings, string_view, and C++20 Ranges

8.1 std::string: The Essentials

`std::string` is a heap-allocated, dynamically-sized character sequence that manages its own memory. It's your go-to for any mutable string work.

Concept / Pattern

```
1 #include <string>
2 std::string s = "Hello, World!";
3 s.size();           // 13 (length, no null terminator counted)
4 s.empty();          // false
5 s[0];               // 'H' -- no bounds check, fast but risky
6 s.at(0);            // 'H' -- throws std::out_of_range if bad index
7 s.front(); s.back(); // 'H', '!'
8
9 // Concatenation
10 s += " C++";        // append in place (efficient)
11 std::string t = s + "!"; // creates a new string (slow: allocates)
12
13 // Searching
14 auto pos = s.find("World"); // returns index, or std::string::npos if absent
15 if (pos != std::string::npos) { /* found */ }
16 s.contains("C++");        // C++23: returns bool directly
17
18 // Substrings
19 s.substr(7, 5);           // "World" (start index, length -- COPIES)
20
21 // Modification
22 s.insert(5, " there");    // inserts " there" at position 5
23 s.erase(5, 6);           // removes 6 chars starting at position 5
24 s.replace(0, 5, "Hi");    // replaces first 5 chars with "Hi"
25 s.clear();               // empties the string
26
27 // Case conversion (no stdlib one-liner -- use <algorithm>)
28 #include <algorithm>
29 #include <cctype>
30 std::transform(s.begin(), s.end(), s.begin(), ::tolower);
```

8.1.1 String Conversion

```
1 // To string
2 std::string s1 = std::to_string(42);    // "42"
3 std::string s2 = std::to_string(3.14);  // "3.140000"
4
5 // From string
```

```

6 int    x = std::stoi("123");
7 long   y = std::stol("99999");
8 double z = std::stod("3.14");

```

8.1.2 Trimming Whitespace (No Standard Function)

```

1 // Trim leading
2 s.erase(0, s.find_first_not_of(" \t\n\r"));
3 // Trim trailing
4 s.erase(s.find_last_not_of(" \t\n\r") + 1);

```

8.1.3 String Streams

```

1 #include <sstream>
2
3 // Build a string piece by piece
4 std::ostringstream oss;
5 oss << "Player " << 5 << " scored " << 100;
6 std::string result = oss.str(); // "Player 5 scored 100"
7
8 // Parse a string like cin
9 std::istringstream iss("10 20 30");
10 int a, b, c;
11 iss >> a >> b >> c; // a=10, b=20, c=30
12
13 // Tokenize by line
14 std::istringstream stream("line1\nline2\nline3");
15 std::string line;
16 while (std::getline(stream, line)) {
17     std::cout << line << '\n';
18 }

```

Performance / Detail

SSO (Small String Optimization): Most standard library implementations store short strings (≤ 15 chars on most platforms) directly inside the `std::string` object itself – no heap allocation. Longer strings go on the heap. This makes working with short strings essentially free.

8.2 std::string_view: Zero-Cost Slicing

Passing a `std::string` by value copies the entire heap buffer. Passing by `const std::string&` is better, but still requires a temporary heap allocation if you call it with a string literal. `std::string_view` (C++17) is the fix: it's just a **(pointer, length)** pair pointing into existing memory.

Concept / Pattern

```

1 #include <string_view>
2
3 // Always prefer string_view for read-only string parameters
4 void log(std::string_view msg) { // accepts literals, std::string, substrings
5     std::cout << msg;           // zero allocation, zero copy
6 }
7
8 log("raw literal");             // 0 allocations
9 std::string s = "hello world";
10 log(s);                        // 0 allocations (view into s's buffer)
11 log(std::string_view(s).substr(6, 5)); // view of "world", 0 allocations
12
13 // Operations (all O(1), no copying)
14 std::string_view sv = "hello world";
15 sv.size();                     // 11
16 sv.starts_with("hello");       // true (C++20)
17 sv.ends_with("world");         // true (C++20)
18 sv.find("world");              // 6
19 sv.remove_prefix(6);            // sv now points at "world"
20 sv.remove_suffix(5);            // trim from back

```

Common Mistake

`string_view` does NOT own memory. If the underlying string is destroyed or reallocated, the view becomes a dangling pointer. Never store a `string_view` as a class member unless you can guarantee the source outlives the object.

```

1 std::string_view get_view() {
2     std::string s = "temporary";
3     return s; // DANGER: s is destroyed when function returns!
4 }             // view now points at freed memory -- crash

```

Also, `string_view` is NOT guaranteed to be null-terminated (it's a substring view). Never pass it to C functions that expect `char*`.

When to Use

Use `string_view` when the function only reads the string and doesn't store it beyond the call. Use `const std::string&` when you need null-termination or must store the string. Use `std::string` when you need to own or modify the string.

8.3 C++20 Ranges and Views

`std::ranges` brings functional-style, lazy, composable sequence operations. Pipe (l) operations together to build pipelines. Views compute nothing until you iterate — no intermediate vectors, O(1) memory overhead.

8.3.1 Ranges Algorithms: No More `.begin()/.end()`

```

1 #include <algorithm>
2 #include <ranges>
3 #include <vector>
4 std::vector<int> v = {5, 3, 1, 4, 2};
5
6 std::ranges::sort(v);           // whole container, no iterators needed
7 std::ranges::reverse(v);
8 bool found = std::ranges::contains(v, 3); // C++23
9 auto it    = std::ranges::find(v, 4);
10 std::ranges::sort(v, std::greater<>{}); // sort descending

```

8.3.2 Views: Lazy Composable Pipelines

Concept / Pattern

```

1 std::vector<int> nums = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
2
3 // Filter evens, square them, take first 3 -- zero intermediate vectors
4 auto result = nums
5     | std::views::filter([](int n) { return n % 2 == 0; })
6     | std::views::transform([](int n) { return n * n; })
7     | std::views::take(3);
8
9 for (int x : result) std::cout << x << ' '; // 4 16 36
10 // Nothing was computed until the for-loop ran!

```

8.3.3 Common Views Reference

Performance / Detail

View	What it does
<code>views::filter(pred)</code>	Keep elements where <code>pred</code> is true
<code>views::transform(func)</code>	Apply <code>func</code> to each element
<code>views::take(n)</code>	Keep only first <code>n</code> elements
<code>views::drop(n)</code>	Skip first <code>n</code> elements
<code>views::reverse</code>	Iterate in reverse
<code>views::iota(start, end)</code>	Generate sequence <code>[start, end)</code> like Python <code>range()</code>
<code>views::keys / views::values</code>	First/second of pairs (great for maps)
<code>views::enumerate (C++23)</code>	Yields (index, element) pairs
<code>views::zip(a, b) (C++23)</code>	Pairs up two ranges element-by-element
<code>views::chunk(n) (C++23)</code>	Splits into sub-ranges of size <code>n</code>
<code>views::split(delim)</code>	Splits range by delimiter

```

1 // Generate: 0 to 99, filter odds, square them
2 auto sq_odds = std::views::iota(0, 100)
3     | std::views::filter([](int n){ return n % 2 != 0; })
4     | std::views::transform([](int n){ return n * n; });
5
6 // Iterate map keys/values cleanly
7 std::map<std::string, int> scores = {"Alice", 90}, {"Bob", 80};
8 for (auto& key : scores | std::views::keys) std::cout << key << '\n';
9 for (auto& val : scores | std::views::values) std::cout << val << '\n';
10
11 // C++23 enumerate (like Python's enumerate)
12 std::vector<std::string> names = {"Alice", "Bob", "Carol"};
13 for (auto [i, name] : std::views::enumerate(names))
14     std::cout << i << ": " << name << '\n';

```

Common Mistake

Views are lazy but not free. Some combinations (like `filter + reverse`) may still traverse the whole range. An infinite view like `views::iota(0)` will loop forever without a `take`. Also, views don't store data — if the underlying container is modified while you're iterating a view over it, that is undefined behavior.

Chapter 9

Scope, Storage Duration, and Lifetime

9.1 Variable Scope and Storage Duration

Scope = where in code you can refer to a variable. **Storage duration** = how long its memory lives.

Concept / Pattern

```
1 int g_global = 10;           // global scope, static storage (whole program)
2
3 void foo() {
4     int local = 5;           // block scope, automatic storage (created/destroyed with
5                               // foo)
6
7     static int s_counter = 0; // block scope, but STATIC storage
8     ++s_counter;              // keeps value between calls! Initialized exactly once.
9
10    {
11        int inner = 3;        // inner block scope
12    }
13    // inner is destroyed here; local and s_counter still live
14 }
```

9.2 Static Variables

```
1 // 1. Static local: initialized once, survives between calls (thread-safe in C++11)
2 int next_id() {
3     static int id = 0;
4     return ++id; // returns 1, 2, 3, ... on successive calls
5 }
6
7 // 2. Static member: shared across all instances of a class
8 class Counter {
9 public:
10     static int count;
11     Counter() { ++count; }
12 };
13 int Counter::count = 0; // define in .cpp
14
15 // 3. Internal linkage: static free variable/function not visible outside the TU
16 static int helper_only_here = 42;
17 // Prefer anonymous namespaces (more idiomatic):
18 namespace {
19     int also_internal = 99;
20     void private_helper() { }
21 }
```

9.3 `thread_local`: Per-Thread Storage

```
1 // Each thread gets its OWN copy of a thread_local variable
2 thread_local int tl_error = 0;
3
4 void worker() {
5     tl_error = 42; // only affects THIS thread's copy
6 }
7
8 // Great for per-thread RNG engines (no mutex needed)
9 thread_local std::mt19937 rng{std::random_device{}()};
```

Chapter 10

References: Everything You Need to Know

10.1 Lvalue References

Concept / Pattern

```
1 int x = 5;
2 int& ref = x;      // ref IS x -- same memory address
3 ref = 10;          // x is now 10
4 &ref == &x;        // true
5
6 // References must be initialized; they can't be reseated (re-pointed)
7 int& bad;           // ERROR: uninitialized reference
8 int y = 20;
9 ref = y;            // does NOT reseat ref -- this copies y's VALUE into x
10
11 // Classic use: avoid copying large objects
12 void process(const std::vector<int>& v) { /* no copy */ }
13 void modify(std::vector<int>& v)      { v.push_back(99); }
```

10.1.1 Const References Extend Temporary Lifetime

```
1 // A const lvalue ref can bind to a temporary, extending its lifetime to match
2 const std::string& s = std::string("temporary"); // lives as long as s
3 // Non-const ref CANNOT bind to temporaries:
4 std::string& bad = std::string("hello"); // COMPILE ERROR
```

10.2 Reference vs. Pointer: When to Use Which

Performance / Detail

Use a reference when...	Use a pointer when...
The argument is always valid (non-null)	It may be <code>nullptr</code> (optional argument)
You don't need to change what you point to	You need to point to different objects
Passing to algorithms, range-for	Dynamic allocation, ownership semantics
Returning a member for chaining	Returning a "not found" signal
Rule of thumb: prefer references. Use pointers only when you need nullability or rebinding.	

Chapter 11

Memory, Pointers, and Smart Pointers

11.1 Pointers & Dereferencing

Dereferencing means using a pointer to look up the specific memory address it holds, traveling to that spot in physical RAM, and extracting or modifying the actual value stored there.

11.1.1 The Map Analogy

A pointer variable is like a treasure map—it's simply a piece of paper with an address written on it. Dereferencing is the act of physically traveling to that coordinate, digging, and opening up the treasure box.

11.1.2 Code Example

```
1 int treasure = 42;
2 int* map = &treasure; // Pointer storing address of 'treasure'
3
4 std::cout << map;      // Prints raw address (e.g., 0x7ffeefbff56c)
5 std::cout << *map;     // Dereferencing! Prints the actual value: 42
6
7 *map = 99;             // Dereference modification changes 'treasure' to 99
```

11.1.3 The Trap: Nullptr Dereferencing

If you attempt to dereference a pointer set to `nullptr` (address 0x0), the CPU is ordered to read a protected segment of memory that doesn't exist for your app. The operating system terminates the execution immediately, throwing a Segmentation Fault.

11.2 Random Number Generation: `rand()`, Mersenne Twister & Distributions

Modern C++ heavily discourages using the legacy C-style `rand()` function due to deep structural deficiencies.

11.2.1 Three Reasons `rand()` is Broken

1. Poor Statistical Randomness: Most versions use a Linear Congruential Generator (LCG). The lowest-order bits can repeat rapidly, leading to predictable patterns (e.g., cycling perfectly between even and odd outputs).
2. Modulo Bias: Mapping `rand() % 10` forces the range into values where certain bins are mathematically favored over others if the maximum random threshold (`RAND_MAX`) doesn't divide cleanly by the range.
3. Hidden Global State: `rand()` relies on a single background state, making it unsafe for multi-threaded applications without introducing performance-killing locks.

11.2.2 The Solution: Mersenne Twister (`std::mt19937`)

Introduced in C++11, the Mersenne Twister solves all three issues:

- Incredible Repetition Period: It runs for $2^{19937} - 1$ iterations before ever repeating its pattern.
- Multi-Dimensional Distribution: Passes demanding statistical test suites (Dieharder).
- Object Scope Isolation: It's defined as a local engine instance, completely safe for parallel thread distribution.

11.2.3 Engine + Distribution Architecture

```

1 #include <random>
2
3 std::random_device rd;           // Hardware random seed source
4 std::mt19937 gen(rd());          // Engine (creates random bitstreams)
5 std::uniform_int_distribution<int> distrib(1, 6); // Shaper (eliminates modulo bias)
6
7 int roll = distrib(gen);         // Perfect, balanced distribution

```

Common Mistake

CRITICAL NOTE:

The Mersenne Twister is NOT cryptographically secure. Since its sequence generation is algebraic, observing 624 consecutive numbers allows an attacker to predict every future output. Do not use for security tokens/passwords.

Mersenne Numbers & the Mathematical Basis of `mt19937`

A Mersenne number is any number conforming to the algebraic form: $M_k =$

$$2^k - 1$$

11.2.4 Is a Mersenne Number Always Prime?

No, it is absolutely not always prime.

11.2.5 Two Mathematical Criteria

1. For $2^k - 1$ to have any mathematical chance of being prime, the exponent k MUST be a prime number itself. If k is composite, the entire result is composite. Example ($k=4$): $2^4 - 1 = 15$ (Composite: 3×5).
2. Even if k is prime, the result is still NOT guaranteed to be prime. The condition breaks at $k=11$. Example ($k=11$): $2^{11} - 1 = 2047$ (Composite: 23×89).

11.2.6 Connection to `std::mt19937`

The number 19937 is explicitly chosen because it is a prime exponent that yields a valid Mersenne Prime ($2^{19937} - 1$). This ensures that the state matrix of the algorithm transitions uniformly across its massive mathematical loop without collapsing early.

Gamma & Other `<random>` Distributions

The C++ `<random>` library cleanly decouples raw bit engines (`std::mt19937`) from mathematical distributions. It includes full support for `std::gamma_distribution`, parameterized by alpha (shape) and beta (scale).

11.2.7 Code Example

```
1 std::mt19937 gen(1337);
2 double alpha = 2.0, beta = 2.0;
3 std::gamma_distribution<double> dist(alpha, beta);
4
5 double result = dist(gen); // Generates continuous gamma value
```

Key Insight

Math Trick:

If you configure `alpha = 1.0`, the Gamma distribution simplifies directly into an Exponential Distribution (`std::exponential_distribution`).

11.2.8 Standard Library Distribution Overview

1. Uniform: `uniform_int_distribution`, `uniform_real_distribution` (Equal odds).
2. Bernoulli: `bernoulli_distribution` (Coin flip), `binomial_distribution`, `geometric_distribution`, `negative_binomial_distribution`.
3. Poisson (Arrival Rates/Time): `poisson_distribution` (Event count), `exponential_distribution` (Interval spacing), `gamma_distribution`, `weibull_distribution`, `extreme_value_distribution`.
4. Normal (Bell Curves): `normal_distribution` (Gaussian), `lognormal_distribution`, `chi_squared_distribution`, `cauchy_distribution`, `fisher_f_distribution`, `student_t_distribution`.
5. Sampling (Custom Mappings): `discrete_distribution` (Custom integer weights), `piecewise_constant_distribution`, `piecewise_linear_distribution`.

11.3 Linkage Rules: `const`, `inline`, and ODR

The behavior of scopes in C++ depends entirely on the One Definition Rule (ODR), which mandates that an entity can only have one definition across a complete program.

11.3.1 Why Global `const` Defaults to Internal Linkage

Standard global namespace variables default to *external linkage*, meaning they are visible across all compiled files. However, global `const` variables default to *internal linkage*. They are localized strictly within the single object file (.cpp) they are compiled in.

This design choice allows you to declare global configuration values inside header files (`constants.h`) and include them in multiple translation units safely:

```
1 // constants.h
2 const double PI = 3.14159;
```

Because of internal linkage, every single .cpp file that includes `constants.h` receives its own isolated, private literal copy of `PI`. The linker never detects duplicate global symbols, entirely avoiding ODR collisions.

11.3.2 What is `inline` in Modern C++?

Historically, `inline` was an optimization advice telling the compiler to copy-paste a function's contents directly to bypass function-call speed costs. Modern compilers handle this choice automatically.

Today, `inline` acts as a Linker Directive designed to relax the One Definition Rule. It promises the linker: "This item will be duplicated across files. Do not panic about clashing definitions; drop the extras and match everyone to one shared memory address."

11.3.3 C++17 Inline Variables

Before C++17, global shared variables required split files (header `extern` declaration, plus `.cpp` instantiation). Now, you can declare and initialize a single true global variable directly in a shared header:

```
1 // global_settings.h
2 inline std::string g_app_title = "Production System v4"; // Safely shared project-wide
```

11.3.4 Global Variables: The Four Flavours and Their Linkage

Not all global variables behave the same. The combination of `const`, `constexpr`, and `inline` completely changes how the linker treats them.

Concept / Pattern

Global Variable Linkage Reference Table

Declaration	Linkage	Copies per TU	Safe in headers?
<code>int g_x = 5;</code>	External	1 (shared)	No — ODR violation
<code>const int g_x = 5;</code>	Internal	One per TU	Yes
<code>constexpr int g_x = 5;</code>	Internal	One per TU	Yes
<code>inline int g_x = 5;</code>	External (shared)	1 (shared)	Yes (C++17)
<code>inline constexpr int g_x = 5;</code>	External (shared)	1 (shared)	Yes (C++17, preferred)

Plain Global Variables (`int g_x`)

Regular global variables have **external linkage** by default. Including the same definition in two `.cpp` files causes the linker to scream about a duplicate symbol:

```
1 // shared.h
2 int g_counter = 0; // DANGER: external linkage
3
4 // file_a.cpp
5 #include "shared.h" // defines g_counter
6
7 // file_b.cpp
8 #include "shared.h" // defines g_counter AGAIN -> linker error!
```

Common Mistake

ODR Violation: Multiple Definition Error

The linker error for this looks like:

```
ld: error: duplicate symbol 'g_counter' in file_a.o and file_b.o
clang: error: linker command failed with exit code 1
```

The fix: *never* define (initialise) a non-const, non-inline global in a header. Put the definition in one `.cpp` file and use `extern int g_counter;` in the header.

`const` Global Variables

When you add `const`, the variable's linkage silently flips to **internal**. Each translation unit gets its own private copy, and the linker never sees a conflict:

```
1 // constants.h
2 const int MAX_NODES = 1000; // internal linkage -- safe to include everywhere
3 const double GRAVITY = 9.81; // each .cpp gets its own copy of GRAVITY
```


This is fine for compile-time constants because the compiler replaces every use of `MAX_NODES` with the literal 1000 at compile-time (constant folding), so the duplicate copies never even end up in the binary.

Common Mistake

Each TU Gets Its Own Copy — Address May Differ

Because every `.cpp` gets a separate copy of a `const` global, taking its address (`&MAX_NODES`) in two different files may yield two *different* addresses. Never compare addresses of `const` globals across translation units.

constexpr Global Variables

`constexpr` globals behave *identically* to `const` globals with respect to linkage — they are also **internal** by default. But they carry the additional guarantee that the value is *always* available at compile-time:

```
1 // constants.h
2 constexpr int CACHE_LINE_SIZE = 64;    // internal linkage, compile-time constant
3 constexpr int BUFFER_SIZE      = 1 << 16; // expressions are fine
4
5 // You can use them in compile-time contexts that const cannot satisfy:
6 std::array<int, BUFFER_SIZE> buf;      // OK: BUFFER_SIZE is constexpr
7 template <int N> struct Foo {};
8 Foo<CACHE_LINE_SIZE> f;                // OK: CACHE_LINE_SIZE is constexpr
```

Concept / Pattern

`constexpr` $\supset$ `const`

Every `constexpr` variable is implicitly `const`. You never need to write `const constexpr` — it is redundant. `constexpr` is strictly stronger: it requires a compile-time initialiser *and* guarantees the value is constant.

inline constexpr: The Modern Header-Safe Global (C++17)

The problem with `const/constexpr` globals in headers is that each TU gets its own copy. For *large* objects (e.g. a `constexpr std::array` or a `constexpr std::string_view` lookup table) this wastes binary space.

Adding `inline` promotes the variable back to **external linkage** but tells the linker to collapse all copies into one shared address:

```
1 // constants.h (included in 10 .cpp files -- only ONE copy in the binary)
2 inline constexpr int VERSION      = 42;
3 inline constexpr std::string_view APP_NAME = "AfterDark";
4
5 // lookup table: without inline, 10 copies. With inline: 1 copy.
6 inline constexpr std::array<int, 4> PRIMES = {2, 3, 5, 7};
```

Key Insight

The Modern Best-Practice Rule for Global Constants

- **Small arithmetic constants** (integers, floats): `constexpr int N = 100;` in a header is fine — constant folding means there's effectively no binary copy at all.
- **Large or non-trivial constants** (`std::array`, `std::string_view`): Use `inline constexpr` so all TUs share one copy.
- **Mutable shared state**: Define in one `.cpp`, declare `extern` in the header. Never `inline` a mutable global unless you intentionally want all TUs to see the same changing value.

11.4 Implicit Conversion Ranking, = delete, and Ambiguity Fixes

When resolving an overloaded function call, the compiler builds a candidate list and ranks choices based on implicit type-conversion hierarchy rules.

11.4.1 The printint Overload Problem

```
1 void printint(int32_t val);           // Option A
2 void printint(char) = delete;        // Option B (Explicitly disabled)
3 void printint(int64_t) = delete;     // Option C (Explicitly disabled)
```

11.4.2 Case 1: printint('a') — Exact Match on a Deleted Overload

The compiler searches for an exact match before evaluating promotions. Since 'a' is a `char`, it selects Option B as the perfect match. Only AFTER choosing it does the compiler notice it is explicitly marked `= delete`. It generates a compilation failure. This is the correct, intended way to stop characters from slipping into integer slots.

11.4.3 Case 2: printint(5LL) — Ambiguity Crash

5LL creates a 64-bit long long integer. To map this parameter down to the signatures:

- Conversion from `long long` to `int32_t` (Narrowing Standard Conversion).
- Conversion from `long long` to `int64_t` (Standard Conversion). Both target choices share an identical conversion rank level in the language specification. The compiler cannot select one over the other. Crucially, the compiler evaluates ambiguity BEFORE checking if a function is marked `= delete`. It gives up and throws an "ambiguous call" error.

11.4.4 The Clean Fix: C++20 Concepts

Instead of writing endless `= delete` overloads to lock out types, create a single template constrained by a concept rule to enforce absolute type equivalence:

```
1 #include <concepts>
2
3 template<typename T>
4 requires std::same_as<T, int32_t>
5 void printint(T val) {
6     std::cout << val << '\n';
7 } // Blocks every other type instantly during template parsing, avoiding ranking deadlocks.
```

Chapter 12

Type Conversion and Type Safety

12.1 Implicit (Automatic) Conversions

C++ automatically converts between types in certain situations. The compiler applies a ranked set of conversions to make types match.

Performance / Detail

Conversion ranking (best match first):

1. **Exact match:** same type, or trivial adjustments (e.g., `int& → int`).
2. **Promotion:** small types promoted to larger (e.g., `char → int`, `float → double`).
3. **Standard conversion:** numeric conversions, `nullptr → pointer`, `derived* → base*`.
4. **User-defined conversion:** via constructor or operator `Type()`.

Common Mistake

Narrowing conversions (losing data) happen silently with `=` initialization but are errors with `{}` brace initialization:

```
1 int x = 3.14;           // OK (silently truncates to 3)
2 int y{3.14};           // COMPILE ERROR: narrowing conversion
3 double d = 1e18;
4 int z = d;             // OK (silently becomes garbage)
5 int w{d};             // COMPILE ERROR: narrowing
6
7 // Use {}-initialization (uniform initialization) everywhere to catch this!
```

12.2 Explicit Conversions with `static_cast`

```
1 // int to double for floating-point division
2 int a = 7, b = 2;
3 double result = static_cast<double>(a) / b; // 3.5
4
5 // Truncate: double to int explicitly
6 int rounded = static_cast<int>(3.9); // 3
7
8 // Enum class to underlying type
9 enum class Color { Red, Green, Blue };
10 int c = static_cast<int>(Color::Green); // 1
11
12 // Downcast in a known-safe polymorphic hierarchy
13 // (use dynamic_cast if you're not 100% sure)
14 Base* b = new Derived();
15 Derived* d = static_cast<Derived*>(b); // no runtime check!
```

12.3 `std::optional`: A Value or Nothing

Already covered in the templates chapter, but critical for type-safe error handling:

```
1 #include <optional>
2
3 std::optional<int> divide(int a, int b) {
4     if (b == 0) return std::nullopt; // no value
5     return a / b;
6 }
7
8 if (auto result = divide(10, 3)) {
9     std::cout << *result;           // 3
10 } else {
11     std::cout << "division by zero";
12 }
13
14 // value_or: default if empty
15 int x = divide(10, 0).value_or(-1); // -1
```

Chapter 13

Classes and OOP

13.1 Classes & OOP

13.1.1 Enums & Scoped enum class

An enum (enumeration) creates a specialized data type constrained to a rigid menu list of meaningful options, replacing dangerous "magic numbers".

Old-Style enum: Name Leaking Flaw

```
1 enum Direction { UP, DOWN };
2 enum Status { UP, RUNNING }; // COMPILER ERROR! Name clash on 'UP'
```

Traditional enums leak their item names directly into the surrounding namespace scope.

Modern enum class (Scoped Enums)

Introduced in C++11, scoped enums isolate parameters entirely inside their type boundary:

```
1 enum class Direction { UP, DOWN };
2 enum class Status { UP, RUNNING }; // Perfectly legal
3
4 Direction dir = Direction::UP; // Requires explicit scope resolution
```

Scoped enums offer true type-safety. They refuse to implicitly degrade into integers or cross-compare against separate enums without explicit casting (`static_cast<int>`).

Custom Integer Mappings

By default, the internal registry starts counting sequentially from 0. You can assign arbitrary numbers anywhere:

```
1 enum class Architecture {
2     BASE = 10, // Explicitly 10
3     NEXT, // Automatically 11 (10 + 1)
4     OVERRIDE = 40, // Explicitly 40
5     FOLLOW // Automatically 41 (40 + 1)
6 };
```

13.1.2 Runtime Overhead: The Hidden Paperwork Tax

Runtime overhead represents the time, memory, or CPU processing cycles spent managing background system tasks, rather than running your core algorithms. It is the "administrative paperwork tax" of execution.

Primary Sources of Runtime Overhead

1. Dynamic Allocation (`new/delete`): Halts execution to communicate with the OS kernel, scanning system pools to locate an open memory segment on the heap, tracking fragmentation, and returning the reference. Stack tracking, by contrast, has zero overhead.
2. Virtual Table Lookups (virtual functions): Prevents direct code jumps. The CPU must pull the instance address, resolve its `_vptr`, read the class `vtable` directory array, extract the index target pointer, and then branch. This process can stall pipelines or cause cache misses.
3. Exception Stack Unwinding (`try/catch`): When an anomaly triggers a throw, execution stops while the system unwinds step-by-step through the call stack chain to safely execute destructors for intermediate objects before landing on the catch handler.

13.1.3 Cache-Locality Architecture: AoS vs. SoA

High-performance optimization requires arranging data layouts to match how hardware CPU caches interact with system RAM.

1. Array of Structures (AoS)

The standard Object-Oriented paradigm. You design a single entity structure, then place multiple copies inside a container array.

```
1 struct Particle { float position[3]; int id; char code[16]; };
2 std::vector<Particle> dataset(10000);
```

- RAM Layout: [Pos, ID, Code] [Pos, ID, Code] [Pos, ID, Code]
- Cache Impact: If a loop runs simply to update vector coordinates (`position`), the CPU fetches data in 64-byte blocks called Cache Lines. The cache fills up with adjacent `id` and `code` bytes which are completely unused by the current loop. Wasting space this way causes continuous cache misses.

2. Structure of Arrays (SoA)

The Data-Oriented Design (DOD) model. You instantiate a single system structure containing isolated arrays dedicated to each individual attribute.

```
1 struct ParticleSystem {
2     std::vector<float[3]> positions;
3     std::vector<int> ids;
4     std::vector<char[16]> codes;
5 };
```

- RAM Layout: [Pos, Pos, Pos...] ... [ID, ID, ID...] ... [Code, Code...]
- Cache Impact: When looping over `positions`, 100% of every fetched 64-byte cache line is packed with raw position data. Cache efficiency hits maximum density, instruction pipelines stay full, and the clean data alignment allows compilers under `-O3` to trigger SIMD vector processing.

In C++, the technical difference between a `class` and a `struct` is famously tiny — literally just **one keyword change regarding default visibility**. However, the *stylistic* and architectural reasons for choosing one over the other are deeply ingrained in C++ culture.

Here is why you would use a `class` instead of a `struct` in C++.

The Literal Technical Difference

In C++, both `class` and `struct` can have constructors, destructors, member functions, virtual functions, and use inheritance. The *only* two differences mandated by the compiler are:

- **Default Access Specifier:** Members of a `class` are `private` by default. Members of a `struct` are `public` by default.

- **Default Inheritance Specifier:** A `class` inherits from a base class as `private` by default. A `struct` inherits as `public` by default.

```

1 struct MyStruct {
2     int x; // Public by default
3 };
4
5 class MyClass {
6     int x; // Private by default
7 };

```

Invariant Enforcement and Encapsulation

Because a `class` defaults to `private`, it is the tool of choice when you want to enforce **encapsulation**.

If your data structure has rules (invariants) that must never be broken, you use a `class`. You hide the raw data behind private access and force the user to interact with it via public methods that validate the data.

```

1 // Bad candidate for a struct, because a user could set 'month' to 99.
2 class Date {
3 private:
4     int day;
5     int month;
6     int year;
7
8 public:
9     void setMonth(int m) {
10         if (m >= 1 && m <= 12) month = m; // Enforces the invariant
11     }
12 };

```

If you used a `struct` here, you would constantly have to type `private:` at the top, which defeats the semantic purpose of a `struct` in C++.

The Conceptual Divide (C++ Convention)

Because C++ evolved from C, developers use the two keywords to signal intent to anyone reading the code.

Use a `struct` for Plain Old Data (POD)

A `struct` is used when the type is just a passive bundle of data with no complex internal logic.

- It usually has no user-defined constructor (or just a simple aggregate one).
- It has no private data.
- It doesn't use virtual functions or polymorphism.

```

1 struct Point3D {
2     double x;
3     double y;
4     double z;
5 }; // Just data. No invariants to protect.

```

Use a `class` for an Object

A `class` is used when you are building a smart component with behavior.

- It manages resources (like memory, file handles, or network sockets).
- It has complex internal state.
- It uses inheritance or polymorphism.

Standard Library Expectations

The C++ Standard Library and major frameworks follow these exact semantic cues. For example, standard containers like `std::vector` or `std::string` are implemented as classes because they manage heap memory and have strict internal rules.

Conversely, traits or configuration bundles (like `std::hash` or custom configuration options passed to a library) are almost always written as structs because they are meant to be open and universally accessible.

Summary Guidelines for C++

- Use **struct** when you want a "dumb" container where making all members **public** makes sense, and you don't care if the user manipulates the internal variables directly.
- Use **class** when you are building a "smart" object that needs to protect its internal state, abstract away implementation details, or participate in a class hierarchy.

```

1 class c1 {
2     i32 a;
3     i32 b;
4 public:
5     c1() : b{5}, a{b} {}
6 };
7
8 void solve() {
9     c1();
10 }
```

Common Mistake

Uninitialized Member Trap

The code above compiles without any errors, but it contains a dangerous, hidden bug known as an **uninitialized member initialization trap**. While the code compiles completely fine, `a` will not end up being 5. Instead, `a` will be filled with raw memory garbage.

Here is the exact breakdown of why the compiler allows this, why it fails at runtime, and how to fix it.

1. The Core C++ Rule: Order of Declaration Dominates

In C++, class members are initialized in the exact order they are **declared inside the class body**, completely ignoring the order you list them in your constructor's initialization list.

Look at the class body:

```

1 class c1 {
2     i32 a; // 1. Declared first!
3     i32 b; // 2. Declared second!
4     // ...
```

Because `a` is declared before `b`, the compiler generates execution instructions that initialize `a` first, and `b` second.

2. What Happens Step-by-Step at Runtime

When you call `c1()`, the constructor executes its initialization list in the strict order of declaration:

- **Phase 1 (`a{b}` runs first):** The compiler attempts to initialize `a` using the value of `b`. However, because `b` hasn't been initialized yet, it contains whatever random data or "garbage" was sitting in that memory slot. `a` copies this garbage value.
- **Phase 2 (`b{5}` runs second):** Now, `b` is safely initialized to 5.

By the end of the constructor execution, `b` is correctly equal to 5, but `a` is trapped holding an unpredictable garbage value (e.g., -12837462 or 0 depending on the state of your RAM).

3. Why Doesn't the Compiler Throw an Error?

The compiler considers this valid syntax because `b` technically exists in the scope of memory allocated for the object, even if it hasn't been given a value yet.

However, modern compilers (like GCC or Clang) will usually throw a severe warning if you turn on warning flags (`-Wall -Wextra`):

```
1 warning: member 'c1::b' is used uninitialized in this constructor [-Wuninitialized]
2   c1() : b{5}, a{b} {}
3         ^
```

4. How to Fix It Safely

Fix A: Reorder the Class Declarations (Implicit Fix)

If you change the declaration order inside the class block, the constructor code will instantly execute safely in the order you intended:

```
1 class c1 {
2     i32 b; // 1. Initialized first (gets 5)
3     i32 a; // 2. Initialized second (gets b, which is 5)
4 public:
5     c1() : b{5}, a{b} {} // Safe now!
6 };
```

Key Insight

Avoid Dependencies in the Initializer List

The safest approach especially in competitive programming is to avoid making members depend on each other during the initialization stage altogether:

```
1 class c1 {
2     i32 a;
3     i32 b;
4 public:
5     // Both explicitly read directly from literal values
6     c1() : a{5}, b{5} {}
7 };
```

Constructor Execution Chronology

A common misconception is that default constructors run *before* the actual constructor call. This is incorrect. Nothing runs before you call the constructor. Everything happens during the execution of the constructor call.

When you write `c1();` in your `solve()` function, you are issuing a single command to create an object. The moment that line executes, the constructor springs to life and performs its steps in a highly specific sequential order.

The Timeline of `c1();`

- **Step 1: Memory Allocation**

The system instantly allocates a tiny block of memory on the stack large enough to hold two integers (`a` and `b`). At this exact microsecond, both memory slots contain raw, uninitialized storage garbage.

- **Step 2: Entering the Initialization Phase (The `:` Section)**

The constructor looks at the variables in the order they were declared in the class header block, not the order you wrote them in the code list.

- It sees `i32 a;` is declared first. It executes the initialization instruction for `a`, which is `{b}`. It copies the current value of `b` (which is currently garbage) into `a`.

- It sees `i32 b`; is declared second. It executes the initialization instruction for `b`, which is `{5}`. The garbage inside `b` is overwritten with 5.

- **Step 3: Entering the Constructor Body (The `{}` Section)**

The program finally enters the empty curly braces `{}`. Since there is no code written inside them, the constructor finishes its execution path completely.

Concept / Pattern

The Assembly Line Analogy

Think of the constructor initialization list as an assembly line layout. The machine slots are bolted to the floor in a fixed order: Slot A comes first, then Slot B. When you start the factory call (`c1()`), the conveyor belt moves through Slot A first. Because Slot A demands to look at whatever is sitting in Slot B before the belt has even reached Slot B's assembly machine, it pulls a blank, unpainted component.

The Constructor Matching Process

When you instantiate a parameterized object like `c1 a(5);`, the compiler completely bypasses the default constructor and routes execution straight to the parameterized constructor `c1(i32 val)`. C++ objects are only ever initialized by one constructor per instantiation call.

```
1 c1 v1;      // 0 arguments -> Matches c1()      (Default Constructor)
2 c1 v2(5);   // 1 argument  -> Matches c1(i32 val) (Parameterized Constructor)
```

Because you passed the integer 5, the default constructor code is completely ignored.

Direct Construction vs Assignment

Does the compiler assign the value *after* initializing? It depends entirely on how you write the constructor body:

Case A: Using the Member Initializer List (Direct Construction)

```
1 class c1 {
2     i32 v;
3 public:
4     c1(i32 val) : v(val) {} // The Colon syntax
5 };
```

How it executes: The variable `v` is constructed directly with the value 5 loaded into its memory slot on the very first pass. There is no intermediate "blank" or "default" state.

Case B: Using the Constructor Body (Default + Assignment)

```
1 class c1 {
2     i32 v;
3 public:
4     c1(i32 val) {
5         v = val; // The Assignment syntax
6     }
7 };
```

How it executes:

1. Because `v` is not listed after a colon, the compiler must initialize it before entering the curly braces. For a primitive type like `i32`, it initializes it with garbage memory. (If `v` were a `std::string`, it would call its default constructor here).
2. The code enters the body `{}` and hits `v = val;`.
3. The garbage value inside `v` is overwritten with 5.

Key Insight

Maximum Efficiency

When you call `cl a(5);`, only the parameterized constructor is invoked. To keep your code running at maximum efficiency, always use the colon initializer list syntax (`: v(val)`) so the variable skips the garbage/default allocation step entirely and gets its value immediately.

Brace Initialization vs. Copy Initialization

In modern C++, brace initialization (`a{5}`) is *strongly preferred* over copy initialization (`a = 5`) or parenthesis initialization (`a(5)`) inside constructors because it completely blocks accidental data loss via **narrowing conversion prevention**.

The Danger of Copy Initialization

```

1 class cl {
2     i32 a;
3 public:
4     // Silent data loss: 5.99 becomes 5!
5     cl(double val) : a(val) {}
6 };
7
8 void solve() {
9     cl obj(5.99); // No warning, a == 5
10 }
```

Common Mistake

Silent Truncation

Copy and parenthesis initialization silently truncate data when types don't match. No error, no warning your program runs with corrupted values.

The Safety of Brace Initialization

```

1 class cl {
2     i32 a;
3 public:
4     cl(double val) : a{val} {} // COMPILE ERROR immediately!
5 };
```

The compiler halts with: `error: narrowing conversion of 'val' from 'double' to 'i32' inside { }`

Uniform Initialization

Brace initialization works identically across every C++ context primitives, containers, and custom classes:

```

1 i32 x{5}; // Primitives
2 std::vector<i32> v{1, 2, 3, 4}; // Containers
3 cl obj{5}; // Custom Classes
```

Key Insight

Always Use Braces in Initializer Lists

Use `: a{val}, b{5}` in your constructor initializer lists. It enforces type safety and prevents silent data truncation in competitive programming runs.

13.1.4 Copy Constructors & Memory Safety

A **copy constructor** is a special constructor that creates a brand-new object as an exact duplicate of an existing object of the same class.

Syntax

```

1 class cl {
2     i32 data;
3 public:
4     cl(i32 val) : data{val} {}
5
6     // Copy constructor takes a const reference to the source
7     cl(const cl& source) : data{source.data} {}
8 };

```

When is the Copy Constructor Triggered?

- **Scenario A: Direct initialization from another object**

```

1 cl obj1(10);
2 cl obj2 = obj1; // Copy constructor
3 cl obj3{obj1};  // Also copy constructor
4

```

- **Scenario B: Passing by value to a function**

```

1 void process(cl local_obj) { ... } // copy made on entry
2
3 int main() {
4     cl main_obj(20);
5     process(main_obj);
6 }
7

```

- **Scenario C: Returning by value from a function**

```

1 cl create_object() {
2     cl temp(30);
3     return temp; // copy may run (unless NRVO applies)
4 }
5

```

Shallow Copy vs. Deep Copy

If you don't write a custom copy constructor, the compiler generates a **shallow copy** it copies each member verbatim, including raw pointer addresses.

Common Mistake**Double Free Error — Shallow Copy Disaster**

When a class holds a raw pointer, the default shallow copy makes two objects point to the same heap memory. When both destructors run, the same address is freed twice, causing a `free()`: double free detected crash.

```

1 class BadClass {
2     i32* ptr;
3 public:
4     BadClass() { ptr = new i32(5); }
5     ~BadClass() { delete ptr; }
6 };
7
8 void solve() {
9     BadClass obj1;
10    BadClass obj2 = obj1; // obj2.ptr == obj1.ptr !
11 } // CRASH: both destructors delete the same address

```

Three Ways to Fix the Double Free**Fix 1 — Pass by Reference (Competitive Programming Default)**

```

1 // Crashes: shallow copy triggers double free
2 void process(cl local_obj) { ... }
3
4 // Fixed: no copy is made at all
5 void process(const cl& local_obj) { ... }

```

Fix 2 — Use `std::vector` Instead of Raw Pointers

```

1 // Dangerous: manual memory management
2 class BadClass {
3     i32* arr;
4 public:
5     BadClass(i32 size) { arr = new i32[size]; }
6     ~BadClass() { delete[] arr; }
7 };
8
9 // Safe: std::vector has a built-in deep-copy constructor
10 class SafeClass {
11     std::vector<i32> arr;
12 public:
13     SafeClass(i32 size) : arr(size) {}
14     // No destructor needed
15 };

```

Fix 3 — Rule of Three: Write All Three Manually

```

1 class cl {
2     i32* ptr;
3 public:
4     cl() : ptr{new i32{0}} {}
5     ~cl() { delete ptr; }
6
7     // Deep copy constructor
8     cl(const cl& source) {
9         ptr = new i32(*(source.ptr));
10    }
11
12    // Deep copy assignment operator
13    cl& operator=(const cl& source) {
14        if (this == &source) return *this; // self-assignment guard
15        delete ptr;

```

```

16         ptr = new i32(*(source.ptr));
17         return *this;
18     }
19 };

```

13.1.5 Copy Elision: RVO & NRVO

Copy Elision is a compiler optimization where the compiler skips creating a temporary copy and instead constructs the object *directly in its final destination* memory slot. Under -O2/-O3, it happens automatically.

How It Works Under the Hood

The compiler secretly rewrites the function to accept a hidden pointer to the destination:

```

1  // Your code:
2  std::vector<i32> get_vector() {
3      std::vector<i32> temp = {1, 2, 3};
4      return temp;
5  }
6  void solve() {
7      std::vector<i32> v = get_vector();
8  }
9
10 // What the compiler actually does (conceptually):
11 void get_vector(std::vector<i32>* __dest) {
12     new (__dest) std::vector<i32>({1, 2, 3}); // built directly at v's address
13 }
14 void solve() {
15     std::vector<i32> v; // reserve memory
16     get_vector(&v);    // pass its address in
17 }

```

Zero data is moved. `temp` is never on its own stack frame it is `v`.

RVO vs. NRVO

- **RVO (Return Value Optimization):** Triggered when you return an *anonymous* temporary directly on the return line. **Guaranteed** since C++17.

```

1  c1 create() { return c1(5); } // RVO: zero copies, guaranteed
2

```

- **NRVO (Named Return Value Optimization):** Triggered when you return a *named* local variable. **Not guaranteed** by the standard but applied by all modern compilers under optimization.

```

1  c1 create() {
2      c1 temp(5); // named local
3      return temp; // NRVO: compiler tries to elide the copy
4  }
5

```

std::move on Return is an Anti-Pattern

Common Mistake

Never return `std::move(x)`;

Using `std::move` on a return value explicitly disables copy elision. The compiler is forced to construct the object locally and then execute a move constructor to transfer it — more work than zero-overhead elision.

```

1 // WRONG: kills copy elision
2 std::vector<i32> get_data() {
3     std::vector<i32> local_vec = {1, 2, 3};
4     return std::move(local_vec); // forces move constructor
5 }
6
7 // CORRECT: elision gives zero-cost construction
8 std::vector<i32> get_data() {
9     std::vector<i32> local_vec = {1, 2, 3};
10    return local_vec; // compiler builds it directly in the caller
11 }

```

The Three Return Directions Compared

- **Direction A:** `return local_vec;` $\Rightarrow$ Copy Elision. **Zero overhead.**
- **Direction B:** `return std::move(local_vec);` $\Rightarrow$ Move Constructor. Fast, but adds pointer-swap overhead.
- **Direction C:** Manual copy $\Rightarrow$ Copy Constructor. Allocates a new heap block. **Very slow.**

Key Insight

The `std::move` Rule

Use `std::move()` freely when *transferring* data between variables inside the same scope (e.g., pushing into a container). **Never** use it on a bare `return` statement — the plain return always wins.

13.1.6 Common Constructor Errors

Error: Brace Init Inside Constructor Body

```

1 class dollars {
2     i32 m_yen;
3 public:
4     dollars(i32 val) {
5         m_yen{val}; // ERROR: brace-init only valid at declaration or in : list
6         std::cout << "step1\n";
7     }
8     ~dollars() {
9         delete m_yen; // ERROR: m_yen is not a pointer
10        std::cout << "step2\n";
11    }
12 };

```

Common Mistake

Two Compile Errors

- `m_yen{val};` is invalid inside a function body. Use `m_yen = val;` or move initialization to the `:` list.
- `delete m_yen;` is illegal on a non-pointer primitive. `i32` lives on the stack and cleans itself up.

The Correct Version:

```

1 class dollars {
2     i32 m_yen;
3 public:
4     dollars(i32 val) : m_yen{val} { // initializer list
5         std::cout << "step1\n";
6     }
7     ~dollars() { // no delete needed for primitives
8         std::cout << "step2\n";
9     }
10 };
11
12 void solve() {
13     dollars d(100); // prints "step1"
14 } // prints "step2" as d goes out of scope

```

When Would You Actually Use delete?

Only when the member is a *raw pointer* allocated with `new`:

```

1 class dollars_ptr {
2     i32* m_yen;
3 public:
4     dollars_ptr(i32 val) { m_yen = new i32{val}; }
5     ~dollars_ptr() { delete m_yen; } // legal: m_yen is a pointer
6 };

```

Common Mistake**Never delete a Stack Variable**

```

1 void solve() {
2     i32 val{6};
3     i32* ptr = &val;
4     delete ptr; // UNDEFINED BEHAVIOUR: val is on the stack, not the heap
5 }

```

The heap manager finds no record of `val`'s address and the OS throws a segmentation fault.

13.1.7 The this Pointer & Method Chaining

`this` is a hidden pointer available inside every member function. It points to *the object that called the function*, letting you operate on the object's memory directly and return it by reference.

Method Chaining via `return *this;`

```

1 class cl {
2     i32 a;
3 public:
4     cl(i32 val) : a{val} {}
5
6     cl& add(i32 val) { a += val; return *this; }
7     cl& mult(i32 val) { a *= val; return *this; }
8     i32 res() { return a; }
9 };
10
11 void solve() {
12     cl obj{45};
13     i32 ans = obj.add(45).mult(2).add(20).res();
14     std::cout << ans << '\n'; // ((45+45)*2)+20 = 200
15 }

```


Concept / Pattern**How Chaining Works**

Each method modifies `a` in place and returns `*this` — a reference to the same object at the same memory address. The next call in the chain operates on the same object rather than a copy.

`obj.add(45) → returns ref to obj → .mult(2) runs on obj → returns ref to obj → and so on.`

Key Insight

Without `return *this;`

If `add()` returned `void`, the expression `obj.add(45).mult(2)` would fail to compile: you cannot call `.mult()` on `void`.

Three Roles of `this`

- **Name disambiguation:** When a parameter and a member share the same name, `this->x = x;` makes the intent explicit.
- **Method chaining:** `return *this;` enables fluent interfaces.
- **Self-assignment guard:** In copy-assignment operators, if `(this == &source) return *this;` prevents a class from destroying its own data before copying.

13.1.8 Static Members

Static members belong to the *class itself*, not to any individual object. All instances share a single copy.

```

1 class Counter {
2     static i32 count; // shared across all instances
3 public:
4     Counter() { ++count; }
5     static i32 getCount() { return count; }
6 };
7
8 // Access via class name, not object
9 Counter::getCount();

```

Nested Types: Enums & Structs Inside a Class

```

1 class Fruit {
2 public:
3     enum Type { apple, banana, kiwi };
4     struct Nutrition { i32 calories; double fiber; };
5
6 private:
7     Type t;
8     Nutrition n;
9
10 public:
11     // Both constructor forms are equivalent:
12     Fruit(Type t, Nutrition n) : t{t}, n{n} {}
13 };
14
15 void solve() {
16     Fruit f{Fruit::apple, {45, 5.5}};
17     Fruit::Type t = Fruit::banana;
18 }

```

13.1.9 Ref-Qualifiers

Ref-qualifiers let you restrict which *value category* of object can call a given member function. Placing `&` or `&&` after the function declaration (where `const` usually sits) routes the call based on whether the caller is a

permanent object or a temporary.

Concept / Pattern

Value Category Quick Reference

- **& (Lvalue Reference):** A nickname or alias for an object with a permanent name and a fixed address in memory.
- **&& (Rvalue Reference):** A reference type designed to bind *only* to temporary, anonymous objects that are about to be destroyed.

The Setup: Class with Ref-Qualified Getters

```

1 class cl {
2     std::string cl_str;
3 public:
4     cl(std::string s) : cl_str{s} {}
5
6     // Called on permanent (lvalue) objects returns a read-only reference
7     const std::string& get() const& { return cl_str; }
8
9     // Called on temporary (rvalue) objects steals the internal data
10    std::string get() && { return std::move(cl_str); }
11 };
12
13 void solve() {
14     cl a{"Hello"};
15     cl b = a.get();           // Calls const& version (a is a permanent lvalue)
16     cl c = cl{"Hello"}.get(); // Calls && version (temporary rvalue)
17 }
```

The Four Getter Variants Compared

Common Mistake

Mismatched Qualifier Trap

Writing `std::string& get() const&` causes a compile error you cannot return a non-const reference from a const-qualified function. Either make the return type `const std::string&`, or remove `const` from the qualifier.

Variant 1 — Read-Only Reference (Default Getter)

```

1 const std::string& get() const& { return cl_str; }
```

- **What it does:** Returns a read-only memory link (reference) directly to the internal string of a permanent object. No copy is made.
- **Performance:** *Fastest* — zero copies.
- **Use Case:** Default getter when someone only needs to read, print, or inspect the value.

Variant 2 — Modifiable Reference

```

1 std::string& get() & { return cl_str; }
```

- **What it does:** Returns a direct, modifiable reference to the internal string (only callable on non-const lvalue objects).
- **Performance:** *Fast* — zero copies, but grants the caller full write access.
- **Use Case:** When you want the caller to modify the field directly: `my_obj.get() = "New Value";`.

Variant 3 — Value Copy

```
1 std::string get() & { return cl_str; }
```

- **What it does:** Deliberately duplicates the data by allocating a fresh copy on the heap and returning it.
- **Performance:** *Slow* — full heap allocation on every call.
- **Use Case:** Only when the caller needs an independent, isolated copy they can freely mutate.

Variant 4 — Rvalue Steal (Best for Temporaries)

```
1 std::string get() && { return std::move(cl_str); }
```

- **What it does:** Activates *only* when the parent object is a nameless temporary about to be destroyed. It steals the internal data pointer rather than copying.
- **Performance:** *Fastest* — a lightning-fast pointer swap, zero allocation.
- **Use Case:** High-performance harvesting of resources from expiring temporaries.

13.1.10 constexpr Classes

A `constexpr` class has its constructor and methods evaluated *at compile time* when given constant inputs. The resulting object can live entirely in read-only memory.

Full Example: Point2D

```
1 #include <iostream>
2 using i32 = int32_t;
3
4 class Point2D {
5 private:
6     i32 m_x;
7     i32 m_y;
8
9 public:
10    // 1. constexpr constructor initialises immediately at compile time
11    constexpr Point2D(i32 x, i32 y) : m_x{x}, m_y{y} {}
12
13    // 2. constexpr getters (implicitly const)
14    constexpr i32 getX() const { return m_x; }
15    constexpr i32 getY() const { return m_y; }
16
17    // 3. constexpr method returning a new object
18    constexpr Point2D translate(i32 dx, i32 dy) const {
19        return Point2D{m_x + dx, m_y + dy};
20    }
21
22    // 4. C++14+: constexpr methods can mutate state too
23    constexpr void invert() {
24        m_x = -m_x;
25        m_y = -m_y;
26    }
27 };
```

Runtime vs Compile-Time Evaluation

```
1 void solve() {
2     // RUNTIME   computed when the program runs
3     Point2D p1{10, 20};
4     Point2D p2 = p1.translate(5, 5);
5
6     // COMPILE-TIME   computed entirely by the compiler
7     constexpr Point2D cp1{10, 20};
8     constexpr Point2D cp2 = cp1.translate(5, 5);
9 }
```

```

9
10 // cp2.getX() is baked in as the literal constant 15
11 std::cout << cp2.getX() << "\n";
12 }

```

Why Use constexpr Classes?

- **A. Zero Runtime Cost:** All calculations happen during compilation, so the app boots instantly and uses fewer CPU cycles.
- **B. Read-Only Memory (ROM):** The compiler can place `constexpr` objects directly into the executable's `.rodata` section, making them physically unmodifiable at the hardware level.
- **C. Compile-Time Contexts:** Their values can drive template arguments and fixed array bounds:

```

1 constexpr Point2D array_bounds{5, 10};
2 // Valid! Array sizes must be compile-time constants.
3 i32 game_grid[array_bounds.getX()][array_bounds.getY()];

```

13.1.11 The `const` After a Function Name

Placing `const` right after a member function's name is a *promise* to the compiler: “**This function will not modify any of the object's internal variables.**”

Internally, it changes the hidden `this` pointer from `T const this` (pointer to modifiable object) to `const T const this` (pointer to read-only object).

Core Syntax

```

1 class Wallet {
2     int m_money = 100;
3 public:
4     // Valid: read-only getter
5     int getMoney() const { return m_money; }
6
7     // Compile Error: promised const but tried to mutate!
8     void stealMoney() const { m_money = 0; }
9 };

```

Common Mistake

Why const-correctness matters

A `const` object can *only* call `const`-qualified member functions. Forgetting `const` on a getter will cause a compile error whenever the object is passed as a `const&` reference — even if the function doesn't actually modify anything.

```

1 class Player {
2     std::string m_name;
3 public:
4     Player(std::string name) : m_name{name} {}
5     std::string getName() { return m_name; } // Missing const!
6 };
7
8 void printPlayerInfo(const Player& p) {
9     std::cout << p.getName(); // ERROR: cannot call non-const on const object
10 }

```

The mutable Escape Hatch

If a member variable doesn't affect the logical state of the object (e.g., a call-count cache), mark it `mutable`. It can then be modified even inside `const` functions.

```
1 class Database {  
2     mutable int m_read_count = 0; // Exempt from const restriction  
3 public:  
4     std::string fetchData() const {  
5         m_read_count++; // Legal because of mutable  
6         return "Secure Data";  
7     }  
8 };
```


Chapter 14

Operators, Containers, and Object Relations

14.1 Operator Overloading

14.1.1 Basic Example: vector2D

```
1 class vector2D {
2     i32 x, y;
3 public:
4     vector2D(i32 x, i32 y) : x{x}, y{y} {}
5
6     vector2D operator+(const vector2D& other) const {
7         return {x + other.x, y + other.y};
8     }
9
10    void print() const {
11        std::cout << this->x << " " << this->y << '\n';
12    }
13 };
14
15 void solve() {
16     vector2D v1{1, 2}, v2{2, 3};
17     vector2D v3 = v1 + v2;
18     v1.print(); v2.print(); v3.print();
19 }
```

Key Insight

Initializer List vs Body Assignment

Both forms below are semantically equivalent for primitives, but the initializer list form is preferred because it avoids the garbage-then-overwrite step:

```
1 // Body assignment form (less efficient for complex types)
2 vector2D(i32 x, i32 y) { this->x = x; this->y = y; }
3
4 // Initializer list form (direct construction -- preferred)
5 vector2D(i32 x, i32 y) : x{x}, y{y} {}
```

14.1.2 Returning by Value vs. by Reference in Arithmetic Operators

A common mistake when implementing `operator+` is returning a reference (`a&`) instead of a value (`a`). This causes *two* compiler errors simultaneously.

```
1 class a {
2     i32 b, c;
3 public:
4     a(i32 b, i32 c) : b{b}, c{c} {}
```

```

5
6     a operator+(const a& other) const {    // return by VALUE -- correct
7         return {this->b + other.b, this->c + other.c};
8     }
9     void print() { std::cout << b << " " << c << '\n'; }
10 };
11
12 void solve() {
13     a v1(1,2), v2(2,3);
14     a v3 = v1 + v2;
15     v3.print();
16 }

```

Common Mistake

Two Reasons `a& operator+()` Fails

- **Reason 1 — Rvalue binding:** The expression `{b + other.b, ...}` creates a nameless temporary (an *rvalue*). C++ forbids binding a non-const lvalue reference to an rvalue, so the compiler rejects it outright.
- **Reason 2 — Dangling reference:** Even if the compiler allowed it, the temporary is *destroyed* when `operator+` returns. The caller would receive a reference to a dead object (**dangling reference**), causing Undefined Behaviour.

Key Insight

Rule for Arithmetic vs. Compound-Assignment Operators

- `operator+`, `operator-`, `operator*` — return **by value**. They produce a new object and should never touch either operand.
- `operator+=`, `operator-=` — return **by reference** (`*this`). They mutate the left-hand object in place.

```

1 a& operator+=(const a& other) {
2     *this = *this + other;    // reuse operator+
3     return *this;             // return self by reference
4 }

```

Concept / Pattern

RVO Eliminates the Copy Penalty

Returning by value sounds expensive, but under `-O2/-O3` the compiler applies **Return Value Optimization (RVO)**: the result is constructed *directly* inside the caller's variable, so zero copying actually occurs at runtime.

14.1.3 I/O Operators: Why `operator<<` Needs `friend`

What are `std::ostream` and `std::istream`?

Before understanding *why* `friend` is needed, you need to know what these types actually are.

Performance / Detail

std::ostream — Output Stream

`std::ostream` is a class defined deep inside the C++ standard library (in `<ostream>`). It represents **any destination you can write text to** — a terminal, a file, a string buffer, etc. `std::cout` is simply a *global instance* of `std::ostream` that is pre-wired to your terminal.

The `<<` operator is a regular overload defined *on* `std::ostream`. The standard library already teaches it to handle `int`, `double`, `char`, `std::string`, etc. — but it knows **nothing** about your custom class.

Performance / Detail

std::istream — Input Stream

`std::istream` is the mirror: a class representing **any source you can read text from**. `std::cin` is a global `std::istream` instance pre-wired to your keyboard. The `>>` operator is again an overload on `std::istream` that knows built-in types — but not yours.

```
1 // Conceptually, the standard library looks like this internally:
2 class ostream {
3 public:
4     ostream& operator<<(int x)      { /* write int to terminal */ return *this; }
5     ostream& operator<<(double x) { /* write double */           return *this; }
6     ostream& operator<<(char x)   { /* write char */             return *this; }
7     // ... but NO overload for your custom class 'a'
8 };
9 extern ostream cout; // std::cout is just a global instance of this
```

The Left-Hand Side Problem

When you write `std::cout << obj`, the compiler rewrites this as a function call: `operator<<(std::cout, obj)`.

If `operator<<` were a *member* of your class `a`, the class object must be on the **left side**:

```
1 // If operator<< were a member of class 'a':
2 obj.operator<<(std::cout); // your object would be on the LEFT
3 // Which forces you to write it as:
4 obj << std::cout;         // BACKWARDS -- nobody writes this
```

But you want `std::cout << obj` (`cout` on the left). That means `operator<<` must belong to `std::ostream` — but **you cannot add members to a class you don't own**. The standard library is compiled code sitting in `/usr/lib`. You have no way to reach in and add a new overload to it.

The only exit: make `operator<<` a **free (global) function**:

```
1 // A free function -- no class owns it, takes any two arguments
2 std::ostream& operator<<(std::ostream& os, const a& obj);
3 //
4 //                ^                ^
5 //                LEFT (cout)      RIGHT (your object)
```

The Private Access Problem

A free function is an *outsider*. It cannot see **private** members of your class. If `b` and `c` are private, the free function gets a compile error:

```
1 std::ostream& operator<<(std::ostream& out, const a& other) {
2     out << other.b; // ERROR: 'int a::b' is private within this context
3     return out;
4 }
```

The fix is **friend**. Declaring it **friend** inside the class is a one-time VIP pass: the function *stays* a free function syntactically, but the compiler grants it read access to all private members.

```

1 class a {
2     i32 b, c; // private
3 public:
4     a(i32 b, i32 c) : b{b}, c{c} {}
5
6     // friend: still a free function, but private access granted
7     friend std::ostream& operator<<(std::ostream& out, const a& other) {
8         out << '(' << other.b << " " << other.c << ')' << '\n';
9         return out; // return stream ref -- enables chaining
10    }
11    friend std::istream& operator>>(std::istream& in, a& other) {
12        in >> other.b >> other.c; // fills private members directly
13        return in;
14    }
15 };
16
17 void solve() {
18     a v1(1,2), v2(2,3);
19     std::cout << v1 << v2; // prints both via chaining
20     std::cin >> v1; // reads into private members
21 }

```

Key Insight

Why Return the Stream by Reference?

`operator<<` returns `std::ostream&` (a reference to the *same* stream object) so calls can be **chained**:

```

1 std::cout << v1 << v2 << v3;
2 // Parsed left-to-right as:
3 ((std::cout << v1) << v2) << v3;
4 // Each << returns the same cout, which the next << uses as its left operand.

```

Return void or by value and chaining breaks immediately.

Concept / Pattern

The Full Chain of Reasoning

1. `std::ostream/std::istream` are standard library classes — **you cannot add members to them**.
2. `cout << obj` needs `cout` on the left, so `operator<<` must belong to `ostream` or be a free function — **member of your class is not an option**.
3. A free function is an outsider that cannot see `private` members — **a plain free function breaks encapsulation**.
4. `friend` gives a free function private access while keeping it outside the class — **the only clean solution**.

Concept / Pattern

I/O Operator Checklist

- Declare as `friend` inside the class.
- First parameter: `std::ostream&` (output) or `std::istream&` (input) — **non-const reference** (streams are stateful).
- Second parameter: `const T&` for output; `T&` (mutable) for input.
- Always **return** the stream reference to support chaining.

14.1.4 The Spaceship Operator <=> and operator==

C++20 introduced the three-way comparison operator (<=) which generates all six comparison operators (<, >, <=, >=, ==, !=) from a single definition.

```

1  class a {
2      i32 b, c;
3  public:
4      a() = default;
5      a(i32 b, i32 c) : b{b}, c{c} {}
6
7      // Three-way comparison: compares cross-products (slope ordering)
8      auto operator<=>(const a& rhs) const {
9          return (rhs.b * this->c) <=> (rhs.c * this->b);
10     }
11
12     // = default: compiler generates member-wise == for free
13     bool operator==(const a& rhs) const = default;
14
15     friend std::ostream& operator<<(std::ostream& out, const a& other) {
16         out << '(' << other.b << " " << other.c << ') ' << '\n';
17         return out;
18     }
19     friend std::istream& operator>>(std::istream& in, a& other) {
20         in >> other.b >> other.c;
21         return in;
22     }
23 };
24
25 void solve() {
26     a v1(1,2), v2(2,3);
27     std::cout << v1 << v2;
28     std::cout << (v1 > v2 ? 1 : 0) << '\n';
29 }

```

Concept / Pattern

auto Return Type for <=>

The return type of `operator<=>` is usually deduced as `auto`. The actual category is one of: `std::strong_ordering`, `std::weak_ordering`, or `std::partial_ordering`, depending on the types being compared.

14.1.5 Custom Container Classes

customVector2D — 2D Grid with Subscript Operator

A 2D matrix wrapper that overloads `operator>>` for bulk input and uses C++23 multi-dimensional `operator[]` for indexed access.

```

1  class customVector2D {
2      std::vector<std::vector<i32>>> v;
3      i32 r, c;
4  public:
5      customVector2D(i32 r, i32 c) : r{r}, c{c} {
6          v.resize(r, std::vector<i32>(c));
7      }
8
9      friend std::istream& operator>>(std::istream& in, customVector2D& other) {
10         for (i32 i = 0; i < other.r; i++)
11             for (i32 j = 0; j < other.c; j++)
12                 in >> other.v[i][j];
13         return in;
14     }
15 }

```

```

16 // C++23: multi-index operator[]
17 i32 operator[](const i32 r, const i32 c) { return v[r][c]; }
18 };
19
20 void solve() {
21     i32 r, c; std::cin >> r >> c;
22     customVector2D v{r, c};
23     std::cin >> v;
24     std::cout << v[1, 3] << '\n';
25 }

```

Common Mistake

C++23 Multi-Dimensional operator[

] operator[] (i32 r, i32 c) with multiple parameters requires C++23 (-std=c++23). In C++20 and earlier, use operator()(i32 r, i32 c) (function-call operator) instead. Most online judges still use C++17/20.

Generic Template Containers: customVector1D, customVector2D & customVector3D

The template versions generalise the containers to work with any type T, making them reusable for i32, i64, double, custom structs, etc. customVector2D is built on top of customVector1D, and customVector3D on top of both.

```

1 template <typename T>
2 class customVector1D{
3     std::vector<T> v;
4     i32 n;
5 public:
6     customVector1D() : n{0} {}
7     customVector1D(i32 n, T val){
8         this->n = n;
9         v.resize(n, val);
10    }
11    customVector1D(i32 n){
12        this->n = n;
13        v.resize(n);
14    }
15    friend std::istream& operator>>(std::istream& in, customVector1D& other){
16        i32 n = other.n;
17        for(i32 i = 0; i<n; i++) {
18            in >> other.v[i];
19        }
20        return in;
21    }
22    void force_back(T val){
23        v.push_back(val);
24        ++n;
25    }
26    auto begin() { return v.begin(); }
27    auto end()   { return v.end(); }
28    auto begin() const { return v.begin(); }
29    auto end()   const { return v.end(); }
30    T& back() { return v.back(); }
31    i32 size() const { return n; }
32    auto rbegin() { return v.rbegin(); }
33    auto rend()   { return v.rend(); }
34    void pop_back(){
35        n--;
36        v.pop_back();
37    }
38    friend std::ostream& operator<<(std::ostream& out, customVector1D& other) {
39        for(auto &i : other.v) out << i << " ";

```

```

40         std::cout<<'\\n';
41         return out;
42     }
43     customVector1D& operator=(const customVector1D& other){
44         if(this == &other) return *this;
45         n = other.n;
46         v = other.v;
47         return *this;
48     }
49     T& operator[](const i32 ind){
50         return v[ind];
51     }
52 };
53
54 template <typename T>
55 class customVector2D{
56     customVector1D<customVector1D<T>>> v;
57     i32 r, c;
58 public:
59     customVector2D() = default;
60     customVector2D(i32 r){
61         this->r = r;
62         this->c = 0;
63         v = customVector1D<customVector1D<T>>>(r, customVector1D<T>(c));
64     }
65     customVector2D(i32 r, i32 c){
66         this->r = r;
67         this->c = c;
68         v = customVector1D<customVector1D<T>>>(r, customVector1D<T>(c));
69     }
70     customVector2D(i32 r, i32 c, T val) {
71         this->r = r;
72         this->c = c;
73         v = customVector1D<customVector1D<T>>>(r, customVector1D<T>(c, val));
74     }
75     friend std::istream& operator>>(std::istream& in, customVector2D& other){
76         i32 r = other.r;
77         i32 c = other.c;
78         for(i32 i = 0; i<r; i++)
79             for(i32 j = 0; j<c; j++)
80                 in >> other.v[i][j];
81         return in;
82     }
83     friend std::ostream& operator<<(std::ostream& out, customVector2D& other){
84         i32 r = other.r;
85         i32 c = other.c;
86         for(i32 i = 0; i<r; i++){
87             for(i32 j = 0; j<c; j++) out << other.v[i][j] <<" ";
88             std::cout<<'\\n';
89         }
90         std::cout<<'\\n';
91         return out;
92     }
93     customVector1D<T>& operator[](const i32 r) { return v[r]; }
94     auto begin() { return v.begin(); }
95     auto end() { return v.end(); }
96     T& operator[](const i32 r, const i32 c){ return v[r][c]; }
97 };
98
99 template <typename T>
100 class customVector3D{
101     customVector1D<customVector1D<customVector1D<T>>>> v;
102     i32 k, r, c;
103 public:
104     customVector3D() = default;
105     customVector3D(i32 k, i32 r, i32 c){

```

```

106     this->k = k; this->r = r; this->c = c;
107     v = customVector1D<customVector1D<customVector1D<T>>>(k,
108         customVector1D<customVector1D<T>>>(r, customVector1D<T>(c)));
109 }
110 customVector3D(i32 k, i32 r, i32 c, T val){
111     this->k = k; this->r = r; this->c = c;
112     v = customVector1D<customVector1D<customVector1D<T>>>(k,
113         customVector1D<customVector1D<T>>>(r, customVector1D<T>(c, val)));
114 }
115 friend std::istream& operator>>(std::istream& in, customVector3D& other){
116     for(i32 l = 0; l<other.k; l++)
117         for(i32 i = 0; i<other.r; i++)
118             for(i32 j = 0; j<other.c; j++)
119                 in >> other.v[l][i][j];
120     return in;
121 }
122 T& operator[](const i32 k, const i32 r, const i32 c){ return v[k][r][c]; }
123 };

```

14.2 Containers: `std::vector`, `std::array` and Safe Iteration

14.2.1 `emplace_back()` vs `push_back()`

Both append an element to the end of a container, but they differ fundamentally in *where* the object is constructed.

`push_back()`: Construct Then Move

```

1 std::vector<User> users;
2
3 // 1. A temporary User("Alice", 22) is constructed on the stack.
4 // 2. push_back allocates space, then MOVES the temporary into the vector.
5 // 3. The temporary is destroyed.
6 users.push_back(User("Alice", 22));

```

`emplace_back()`: Construct In-Place (C++11)

```

1 // emplace_back forwards the raw arguments directly to the vector's heap memory.
2 // The constructor is called ONCE, right inside the vector. No temporary, no move.
3 users.emplace_back("Bob", 25);

```

Concept / Pattern

`push_back` vs `emplace_back` at a Glance

Property	<code>push_back(obj)</code>	<code>emplace_back(args...)</code>
Takes	An already-constructed object	Raw constructor arguments
Steps	Construct temp → move → destroy	One construction in-place
Best for	Existing variables	Constructing new objects in-place
Explicit ctors	Respects <code>explicit</code>	Can bypass <code>explicit</code> (careful)

Common Mistake**emplace_back Can Bypass explicit**

emplace_back uses direct-initialization, meaning it can accidentally trigger constructors marked explicit:

```
1 std::vector<std::vector<int>> matrix;
2 matrix.emplace_back(10); // Compiles! Creates a vector of size 10 -- may be unintended.
3 // matrix.push_back(10); // Fails to compile -- the explicit protection kicks in.
```

Key Insight**Decision Rule**

- Passing **raw constructor arguments** to create a new object? $\Rightarrow$ `emplace_back`.
- Passing an **already-existing variable**? $\Rightarrow$ `push_back` (intent is clearer).
- Passing a **primitive** (int, double)? $\Rightarrow$ No difference; `push_back` is fine.

14.2.2 reserve(), capacity(), size() and resize()**The Three Concepts**

```
1 std::vector<int> v;
2 v.reserve(6); // Allocates raw memory for 6 ints -- NO elements created
3
4 v.push_back(10);
5 v.push_back(20);
6 v.push_back(30);
7
8 v.size(); // = 3 (elements actually alive)
9 v.capacity(); // = 6 (raw slots allocated on the heap -- 3 unused)
```

Concept / Pattern**Why reserve() Matters for Performance**

When a vector runs out of capacity during `push_back`, it triggers a **reallocation**: allocate a new larger block (usually 2 $\times$), move every element across, destroy the old block. This is $O(N)$ and can happen many times in a loop.

```
1 // Without reserve: potentially log(N) reallocations
2 std::vector<int> v;
3 for (int i = 0; i < 1000; ++i) v.push_back(i);
4
5 // With reserve: exactly 1 allocation, every push_back is O(1)
6 std::vector<int> v2;
7 v2.reserve(1000);
8 for (int i = 0; i < 1000; ++i) v2.push_back(i);
```

reserve() vs resize()

Property	<code>v.reserve(100)</code>	<code>v.resize(100)</code>
Changes <code>capacity()</code> ?	Yes (at least 100)	Yes (at least 100)
Changes <code>size()</code> ?	No	Yes (becomes 100)
Constructs elements?	Zero	Yes (default-constructs new ones)
Add data after via...	<code>push_back()</code>	Direct indexing <code>v[i]</code>

Common Mistake**reserve() + Index Access = UB**

After `v.reserve(100)`, writing `v[5] = 10` is **Undefined Behaviour**. The memory is allocated but no objects exist at those slots yet. Only use `push_back/emplace_back` after `reserve`.

14.2.3 std::array vs C-Style Array

Both allocate a contiguous, fixed-size block directly on the stack. Their memory layout and assembly output are **identical** — zero overhead for `std::array`.

```
1 // Both: exactly 20 contiguous bytes (5 * 4) on the stack
2 int      c_arr[5]          = {10, 20, 30, 40, 50};
3 std::array<int, 5> cpp_arr = {10, 20, 30, 40, 50};
```

The Pointer Decay Problem (C-style only)

```
1 void printC(int arr[]) {
2     // Compiler sees: void printC(int* arr)
3     // sizeof(arr) == 8 (pointer size) -- the array size is LOST
4 }
5
6 void printCpp(const std::array<int, 5>& arr) {
7     std::cout << arr.size(); // Works: type carries the size
8 }
```

Value Semantics

```
1 int a[3] = {1,2,3};
2 int b[3];
3 // b = a;    // ERROR: C arrays cannot be assigned
4
5 std::array<int,3> x = {1,2,3};
6 std::array<int,3> y;
7 y = x;        // Works: deep copy of all elements
```

Concept / Pattern**std::array Feature Summary**

- Retains size at all times — `.size()` works correctly after passing to a function.
- Full STL iterator support: works with `std::sort`, `std::find`, range-for.
- Optional bounds checking: `arr.at(i)` throws `std::out_of_range` on bad index.
- Can be copied/assigned/returned from functions — C-style arrays cannot.
- Zero runtime overhead over C-style arrays.

14.2.4 Erasing from std::vector Inside a Loop**The Trap: Range-Based Loop + erase()**

When you use a range-based `for` loop, the compiler translates it into a standard iterator loop behind the scenes. Crucially, it evaluates and caches the `.end()` iterator **exactly once** before the loop begins — and never rechecks it.

```
1 void solve() {
2     std::vector<int> v = {1, 2, 3, 4};
3     for (auto& i : v) {
4         if (i == 2) {
```



```

5         v.erase(v.begin() + 1); // erases the element '2'
6     }
7     std::cout << i << " ";
8 }
9 }

```

Example

1 3 4 4

Step-by-Step Memory Mechanics

The loop caches a fixed distance of **4 steps** from the initial vector size. It is hard-wired to run 4 times, completely oblivious to the fact that `erase` changed the vector's size.

1. **Iteration 1 (index 0):** `i` references index 0 → value 1. Condition false. Prints 1.
2. **Iteration 2 (index 1):** `i` references index 1 → value 2. Condition true. `erase` removes 2 and shifts all trailing elements left. Memory becomes [1, 3, 4, ?]. Because `i` is a **reference** (`auto& i`), it looks directly at index 1, which now holds 3. Prints 3.
3. **Iteration 3 (index 2):** Loop advances to index 2, which now holds 4. Prints 4.
4. **Iteration 4 (index 3):** The loop executes its cached 4th step and reads index 3. Because of the element shift, index 3 now contains **residual data** — the old 4 left behind in physical memory. Prints 4.

Changing to Copy Capture Changes the Output

Removing the `&` and capturing by value (`auto i`) changes the second digit:

```

1 void solve() {
2     std::vector<int> v = {1, 2, 3, 4};
3     for (auto i : v) { // copy -- no &
4         if (i == 2) {
5             v.erase(v.begin() + 1);
6         }
7         std::cout << i << " ";
8     }
9 }

```

Example

1 2 4 4

Concept / Pattern

Why the Second Digit Changes from 3 to 2

- **Reference (`auto& i`):** `i` is a direct window into the memory slot. When `erase` shifts elements, the data underneath the window changes from 2 to 3 instantly — so it prints 3.
- **Copy (`auto i`):** At the start of iteration 2 the compiler takes a snapshot: `i = 2`. Even after `erase` shifts the vector, the local copy is isolated and unaffected — so it still prints 2.
- Both forms still walk into residual memory on the 4th step and print 4.

Hardware Reality: Why Residual Data Exists

When you erase an element, the computer does **not** clear or zero the physical memory address.

- **Logical tracker:** The vector updates its internal bookkeeping (`size()`) to say “we own 3 slots now.”
- **Physical reality:** The bytes at the 4th memory slot (index 3) remain completely untouched. The old value 4 sits there as residual data until another operation overwrites it.

Because the range-based loop relies on its cached `end` pointer rather than rechecking `size()`, it blindly walks into that unreclaimed 4th slot and reads whatever leftover data is there — which happens to be 4.

Correct Solutions

```

1 // (A) C++20 -- safest; library handles all iterator bookkeeping
2 std::erase_if(v, [](int x) { return x == 2; });
3
4 // (B) Index-based loop -- re-evaluates v.size() every iteration
5 for (size_t i = 0; i < v.size(); ++i) {
6     if (v[i] == 2) {
7         v.erase(v.begin() + i);
8         --i; // step back: element that shifted into i is still unseen
9     }
10 }
```

Common Mistake

Never Call `erase()` Inside a Range-Based `for`

The range-`for` caches `end` once and trusts it for the entire loop. Any `erase`, `insert`, or `push_back` that invalidates iterators or changes the size turns the remaining iterations into Undefined Behaviour. Always use `std::erase_if` or an explicit index loop when removing elements.

14.3 Memory: Stack, Heap, `new/delete` and Function Pointers

14.3.1 Stack vs. Heap

Performance / Detail

Quick Analogy

Stack = your desk: instant access, very limited space, cleaned up automatically when you finish. **Heap** = a giant warehouse: huge space, but you must book a slot, remember its address, and return it yourself when done.

Stack (Automatic Memory)

```

1 void f() {
2     int x = 42;                // 4 bytes on the stack frame of f()
3     int arr[100];              // 400 bytes on the stack
4     std::array<int, 5> a;      // 20 bytes on the stack
5     // When f() returns, ALL of the above are instantly reclaimed.
6 }
```

- **Allocation:** The CPU just moves the stack pointer — effectively free.
- **Size limit:** Typically 1–8 MB. Exceed it ⇒ **Stack Overflow**.
- **Lifetime:** Tied to the enclosing function scope.

Heap (Dynamic Memory)

```

1 void f() {
2     // 'ptr' (8 bytes) lives on the STACK.
3     // The 400 bytes it points to live on the HEAP.
4     int* ptr = new int[100]{};
5
6     // If you forget this, the 400 heap bytes are NEVER freed -> memory leak!
7     delete[] ptr;
8 }
```

- **Allocation:** The OS searches for a free block — slower, involves bookkeeping.
- **Size:** Limited only by physical RAM / virtual memory (gigabytes).
- **Lifetime:** Controlled by you. `delete/delete[]` to free.

Concept / Pattern

The Hybrid — Pointer on Stack / Data on Heap

```
1 int* arr = new int[100]{};
2 // ^                               -- 8-byte pointer variable lives on the stack
3 // ~~~~~                           -- 400 bytes of ints live on the heap
4 // When the function exits, the 8-byte pointer is destroyed.
5 // The 400 bytes on the heap are NOT freed automatically.
6 // You must call delete[] arr; before the pointer goes out of scope.
```

Property	Stack	Heap
Structure	LIFO, contiguous	Fragmented pool
Allocation	Automatic (CPU)	Manual / dynamic (OS)
Speed	Blazing fast	Slower (bookkeeping)
Size limit	~1–8 MB	Gigabytes (up to RAM)
Lifetime	Function scope	Until <code>delete</code>
Risk	Stack Overflow	Memory Leak

14.3.2 Handling new Failures

When `new` cannot allocate memory (e.g. you request terabytes), it throws `std::bad_alloc` by default. Two ways to handle it:

```
1 // Method A: try/catch
2 void solve() {
3     try {
4         i32* arr = new i32[100000000000000LL];
5         for (size_t i = 0; i < 10000000000; i += 1000) {
6             arr[i] = 0;
7         }
8         delete[] arr;
9     } catch (const std::bad_alloc& e) {
10        std::cout << "well, fuck you\n";
11    }
12 }
13
14 // Method B: std::nothrow -- returns nullptr instead of throwing
15 void solve2() {
16     i32* ptr = new (std::nothrow) i32[99999999999999LL];
17     if (ptr == nullptr) {
18         std::cout << "fuck you too\n";
19     }
20 }
```

Example

Output: Method A
well, fuck you

Example

Output: Method B
fuck you too

Key Insight**Use Smart Pointers Instead of Raw new/delete**

In modern C++, prefer `std::unique_ptr` and `std::shared_ptr` over raw `new/delete`. They automatically call `delete` when they go out of scope, making memory leaks impossible by construction.

```
1 auto arr = std::make_unique<int[]>(100); // freed automatically
```

14.3.3 Function Pointers

Functions are compiled into machine code and loaded into the **code segment** of memory. Because they have a fixed address, you can store that address in a pointer and call the function through it at runtime.

Syntax and Basic Usage

```
1 int add(int x, int y) { return x + y; }
2 int mul(int x, int y) { return x * y; }
3
4 // Declaration: return_type (*ptr_name)(param_types)
5 int (*op)(int, int) = nullptr;
6
7 op = add;
8 std::cout << op(5, 3) << '\n'; // 8 -- implicit call syntax
9
10 op = mul;
11 std::cout << (*op)(5, 3) << '\n'; // 15 -- explicit dereference (identical assembly)
```

Common Mistake**Parentheses Are Mandatory**

`int (*ptr)(int, int)` declares a pointer to a function returning `int`.

`int *ptr(int, int)` declares a *function* returning `int*` — completely different!

Cleaner Syntax: using Alias and std::function

```
1 // C++11 type alias -- much more readable
2 using MathOp = int (*)(int, int);
3 MathOp op = add;
4
5 // std::function -- type-erased wrapper, accepts lambdas + functors too
6 #include <functional>
7 std::function<int(int, int)> f = mul;
8 std::cout << f(4, 5) << '\n'; // 20
```

Practical Use: Callbacks

```
1 bool ascend (int a, int b) { return a < b; }
2 bool descend(int a, int b) { return a > b; }
3
4 void bubbleSort(std::vector<int>& v, bool (*cmp)(int, int)) {
5     for (size_t i = 0; i < v.size(); ++i)
6         for (size_t j = 0; j < v.size()-1; ++j)
7             if (cmp(v[j], v[j+1])) // the callback decides the order
8                 std::swap(v[j], v[j+1]);
9 }
10
11 void solve() {
12     std::vector<int> v = {4, 1, 3, 5, 2};
13     bubbleSort(v, ascend); // sort ascending
```

```

14     bubbleSort(v, descend);    // sort descending
15 }

```

Concept / Pattern

Function Pointer Summary

- A function pointer stores the **address** of compiled code in the code segment.
- The signature (**return type + parameter types**) must match exactly.
- Use **using** aliases or **std::function** to avoid unreadable raw-pointer syntax.
- Function pointers are the foundation of **callbacks**, strategy patterns, and **vtables** (runtime polymorphism under the hood).

14.4 Lambda Functions

A **lambda function** (also called a *closure*) is an anonymous, inline function object defined at the point where it is needed. Introduced in C++11, lambdas eliminate the need for separate named functions or legacy function pointers when passing short, context-aware logic to algorithms.

Performance / Detail

Lambda Anatomy

```

1  auto greet = [capture_clause](params) -> return_type {
2      // body
3  };
4  // ~~~~~
5  // 1. Capture list    2. Params    3. Return type (often omitted; compiler deduces it)

```

- **Capture clause** [] — controls which local variables are visible inside the lambda.
- **Parameters** () — normal function parameters.
- **Body** {} — the code to execute.

14.4.1 Why Lambdas are Superior to Function Pointers

Reason A: Locality of Code

Before lambdas, passing a custom comparison rule to `std::sort` required a separate function far from the call site. Lambdas keep the logic right where it is used:

```

1  // Sort pairs by X ascending, break ties by Y descending
2  std::vector<std::pair<int,int>> pts = {{1,5},{2,3},{1,2}};
3  std::sort(pts.begin(), pts.end(), [](const auto& a, const auto& b) {
4      if (a.first != b.first) return a.first < b.first;
5      return a.second > b.second;
6  });

```

Reason B: Capture Clause (State Awareness)

Standard function pointers are blind to local variables. Lambdas can capture them effortlessly:

```

1  int threshold = 42;
2  // [threshold] pulls a copy of 'threshold' inside the lambda
3  int cnt = std::count_if(v.begin(), v.end(), [threshold](int x) {
4      return x > threshold;
5  });

```

This is impossible with a raw function pointer because you cannot inject `threshold` through `std::count_if`'s fixed signature.

Reason C: Performance — Faster than Function Pointers

Common Mistake

Lambdas beat raw function pointers

When you pass a function *pointer* to `std::sort`, the compiler sees a runtime address and cannot inline the comparison. When you pass a **lambda**, the compiler knows the exact closure type at compile-time and inlines the body entirely — zero call overhead. Lambdas are essentially anonymous structs with `operator()` defined, evaluated 100% at compile-time.

14.4.2 Lambdas are Immutable by Default (mutable)

When you capture a variable **by value**, the captured copy is implicitly `const` inside the lambda body. You can read it but not write to it:

```
1 int x = 10;
2 auto f = [x]() {
3     // x = 20; // ERROR: assignment of read-only variable 'x'
4     return x + 1;
5 };
```

To allow modifying the captured copy (without affecting the original), add the `mutable` keyword after the parameter list:

```
1 int x = 10;
2 auto f = [x]() mutable {
3     x = 20;      // OK: modifies the INTERNAL copy only
4     return x;    // returns 20
5 };
6 std::cout << f(); // 20
7 std::cout << x;   // still 10 -- original is untouched
```

Concept / Pattern

`mutable` only affects value-captured variables

- Value captures (`[x]`, `[=]`) → `const` by default; add `mutable` to allow writes to the internal copy.
- Reference captures (`[&x]`, `[&]`) → always writable (you're modifying the original, not a copy).
- `mutable` does **not** affect the external variable — it only unlocks writes to the lambda's own internal copy.

14.4.3 IIFE: Immediately Invoked Function Expression

An **IIFE** (Immediately Invoked Function Expression) is a lambda that is *defined and called in the same expression*. The giveaway is a pair of `()` trailing immediately after the closing brace of the lambda.

Syntax

```
1 // Normal lambda: defined here, called separately
2 auto myLambda = []() { return 42; };
3 int result = myLambda();           // called later
4
5 // IIFE: defined AND called in one expression
6 int result = []() { return 42; }();
7 //
8 // trailing () invokes it immediately
```

The Superpower: Complex `const` Initialization

The primary motivation for IIFEs is initializing `const` variables that require complex logic (branching, loops). Normally you cannot make such a variable `const` because you must declare it first and assign inside the branches:

```
1 // WITHOUT IIFE -- forced to leave x mutable:
2 int x;
3 if (condition)
4     x = computeA();
5 else
6     x = computeB();
7 // x can be accidentally modified below -- risky!
```

With an IIFE, the entire block executes, returns a value, and initializes `const x` in one clean shot:

```
1 // WITH IIFE -- x is now truly const:
2 const int x = [&]() {
3     if (condition) return computeA();
4     return computeB();
5 }(); // <-- invoked immediately; result assigned to const x
```

Benefits of the IIFE Pattern

Key Insight

Why IIFEs are Useful

1. **Enforces immutability** — variables that need complex initialization can still be declared `const`, preventing accidental mutation later.
2. **Clean scope** — any temporary variables needed only for computing `x` stay trapped inside the lambda body and do not pollute the surrounding function.
3. **Readability** — complex initialization logic is visually grouped right next to the variable definition rather than scattered across several lines.

```
1 // Practical example: initializing a const string based on runtime input
2 const std::string mode = [&]() -> std::string {
3     if (argc > 1 && argv[1] == std::string("--fast")) return "fast";
4     if (argc > 1 && argv[1] == std::string("--safe")) return "safe";
5     return "default";
6 }();
7 // 'mode' is const from this point forward; cannot be accidentally overwritten
```

14.4.4 The Capture Clause: All 6 Forms

The capture clause `[]` is the control panel of a lambda. It dictates which local variables from the surrounding scope are pulled in, and whether they are passed by value (copied) or by reference.

Concept / Pattern

Complete Capture Reference

Syntax	Meaning
[]	Captures nothing. Lambda is isolated like a global function.
[x]	Captures x by value (read-only copy, snapshot at definition time).
[&x]	Captures x by reference (direct alias, changes are visible).
[=]	Implicitly captures every used local variable by value.
[&]	Implicitly captures every used local variable by reference.
[&, x]	Default reference; capture x by value as an exception.
[=, &x]	Default value; capture x by reference as an exception.
[v = std::move(x)]	Init-capture (C++14): move x into new lambda-local v .

Form 1: Capture Nothing []

```

1 void solve() {
2     int x = 10;
3     auto print_hello = []() {           // cannot see 'x'
4         std::cout << "Hello World\n";
5     };
6     print_hello();
7 }
```

Form 2: Capture by Value [x]

Makes a **read-only snapshot** of **x** at the moment the lambda is defined. Later changes to the original variable do NOT affect the lambda.

```

1 void solve() {
2     int target = 42, tax = 5;
3     auto check = [target, tax](int current) {
4         return current == (target + tax);
5     };
6     target = 100;                      // does NOT affect lambda's internal copy
7     std::cout << check(47);           // 1 (true): still uses 42 + 5
8 }
```

Common Mistake

Value-captured variables are const by default

Inside the lambda body, **target** and **tax** are implicitly **const**. You cannot assign to them unless you add the **mutable** keyword after the parameter list: [...]() mutable { ...}.

Form 3: Capture by Reference [&x]

Creates a **direct alias** to the original variable. Modifications inside the lambda affect the original, and vice versa.

```

1 void solve() {
2     int total_sum = 0;
3     auto add_to_sum = [&total_sum](int value) {
4         total_sum += value;           // modifies the original variable
5     };
6     add_to_sum(10);
7     add_to_sum(20);
8     std::cout << total_sum;          // 30
9 }
```


Form 4: Capture All by Value [=]

The compiler scans the lambda body and automatically copies every local variable it finds:

```

1 void solve() {
2     int a = 2, b = 3, c = 4;
3     auto calc = [=]() { return a * b + c; };
4     std::cout << calc();           // 10
5 }

```

Form 5: Capture All by Reference [&]

The most common form in competitive programming, especially for recursive DFS lambdas that need access to the entire local context:

```

1 void solve() {
2     int nodes = 5;
3     std::vector<int> depth(nodes, 0);
4     std::vector<std::vector<int>> adj(nodes);
5
6     auto dfs = [&](auto&& self, int u, int p) -> void {
7         for (auto v : adj[u]) {
8             if (v != p) {
9                 depth[v] = depth[u] + 1;
10                self(self, v, u);
11            }
12        }
13    };
14    dfs(dfs, 0, -1);
15 }

```

Form 6: Mixed / Hybrid Captures

Establish a default rule, then specify exceptions per variable:

```

1 void solve() {
2     int state = 0, config = 100;
3     std::vector<int> data = {1, 2, 3};
4
5     // Default: everything by reference. Exception: 'config' by value (read-only snapshot)
6     auto process = [&, config]() {
7         state = 1;           // OK: captured by reference
8         // config = 200;           // ERROR: config is const (captured by value)
9         data.push_back(config); // OK: data is by reference
10    };
11 }

```

14.4.5 Init-Capture / Generalized Capture (C++14)

Starting in C++14, you can **create brand-new variables** directly inside the capture bracket and initialize them with any expression — including `std::move`. This is the only way to move a non-copyable object (like `std::unique_ptr` or a large vector) into a lambda:

```

1 void solve() {
2     std::vector<int> heavy_data(1'000'000, 42);
3
4     // 'v' is a brand-new variable that OWNS the data (no copy, no reference needed)
5     auto worker = [v = std::move(heavy_data)]() {
6         std::cout << "size: " << v.size() << "\n";
7     };
8     worker();
9     // heavy_data is now empty -- resources were moved into the lambda
10 }

```

14.4.6 Captures and Value Categories: The Golden Rule

Concept / Pattern

How to read any capture clause instantly

- **Ampersand & present** $\Rightarrow$ binds strictly to **lvalues**. A reference cannot hook onto a nameless temporary. Example: `[&x]`, `[&]`.
- **No ampersand** $\Rightarrow$ makes a copy, readable from **either** lvalue or rvalue, but the internal variable is an lvalue inside the body. Example: `[x]`, `[=]`.
- **Init-capture with = inside brackets** $\Rightarrow$ evaluates an arbitrary expression (can be an rvalue). Use `std::move` to transform an lvalue into an rvalue and move it in. Example: `[v = std::move(big)]`.

Capture form	Binds to	Memory effect
<code>[&x]</code> or <code>[&]</code>	Strictly lvalues	No copy. Pointer/alias to stack variable.
<code>[x]</code> or <code>[=]</code>	Lvalues or rvalues	New read-only stack copy inside closure.
<code>[x = std::move(y)]</code>	Lvalue $\rightarrow$ rvalue	Steals resources; transfers ownership into lambda.

14.4.7 What is `std::function`?

`std::function` (from `<functional>`) is a **type-safe, polymorphic callable wrapper**. It can store, copy, and invoke *any* callable target that matches its declared signature — including regular functions, lambdas, function pointers, and custom structs with `operator()`.

Performance / Detail

`std::function` Syntax

```

1 #include <functional>
2
3 // Declare a std::function that takes (int, int) and returns int
4 std::function<int(int, int)> op;
5
6 // Assign a regular function
7 int add(int a, int b) { return a + b; }
8 op = add;
9 std::cout << op(3, 4);    // 7
10
11 // Re-assign to a lambda
12 op = [](int a, int b) { return a * b; };
13 std::cout << op(3, 4);    // 12
14
15 // Re-assign to a capturing lambda
16 int bias = 10;
17 op = [bias](int a, int b) { return a + b + bias; };
18 std::cout << op(3, 4);    // 17

```

How `std::function` Achieves Polymorphism: Type Erasure

Every lambda has a **unique, compiler-generated type**. This means you cannot store two different lambdas in the same `auto` variable — they are incompatible types. `std::function` solves this by erasing the concrete type behind a uniform interface:

```

1 // ERROR: 'f' cannot be re-assigned; auto locks to one specific lambda type
2 auto f = [](int x) { return x + 1; };
3 // f = [](int x) { return x * 2; }; // Compile error: different type!
4

```

```

5 // OK: std::function accepts any lambda with matching signature
6 std::function<int(int)> g = [](int x) { return x + 1; };
7 g = [](int x) { return x * 2; }; // Fine: reassign to a different lambda

```

Under the hood, `std::function` stores the callable in an internal buffer. If the callable is small enough (typically $\leq 16\text{--}32$ bytes), it lives on the stack. Otherwise, it falls back to a **heap allocation**.

Primary Use Cases

- **Heterogeneous callable storage** — when you need a single container (e.g., `std::map`, `std::vector`) to hold callables of completely different types.
- **Callback APIs** — when a function must accept any callable at runtime (e.g., an event system, a timer scheduler).
- **Recursive lambdas** (C++11/14 era) — since a plain `auto` lambda cannot name itself, wrapping in `std::function` gives it a stable, capturable type.

```

1 // Heterogeneous callable map: impossible with 'auto', easy with std::function
2 std::map<std::string, std::function<void()>> actions;
3 actions["jump"] = []() { std::cout << "Jump!\n"; };
4 actions["duck"] = []() { std::cout << "Duck!\n"; };
5
6 // Call by key at runtime:
7 actions["jump"](); // Jump!

```

Common Mistake

`std::function` Has Runtime Cost

Invoking a `std::function` requires an indirect call through an internal function pointer (similar to a virtual dispatch). The compiler **cannot inline** the body, and large captures may trigger a heap allocation. Never use it in tight loops or hot paths where performance matters. Prefer `auto` or `auto&& self` there.

14.4.8 Recursive Lambdas: `std::function` vs. `auto&& self`

Method 1: `std::function` (C++11)

Wrapping a lambda in `std::function` gives it a concrete, stable type it can name itself by:

```

1 void solve() {
2     std::function<int(int)> fib = [&](int n) {
3         if (n <= 1) return n;
4         return fib(n-1) + fib(n-2); // can call 'fib' because it's captured by ref
5     };
6     std::cout << fib(10); // 55
7 }

```

Trade-off: `std::function` uses **type erasure** — it stores the lambda behind a pointer, possibly heap-allocates state, and calls through a virtual-dispatch-like indirection. Every invocation pays a runtime overhead.

Method 2: `auto&& self` (C++23 — Deducing this)

Pass the lambda as its own first argument. The compiler deduces the exact closure type at compile-time and generates fully inlined code:

```

1 auto fib = [](auto&& self, int n) -> int {
2     if (n <= 1) return n;
3     return self(self, n-1) + self(self, n-2);
4 };
5 std::cout << fib(fib, 10); // 55 -- zero runtime overhead

```

`auto&& self` is a **forwarding (universal) reference**. The `&&` ensures that no matter how the lambda object is invoked (lvalue, rvalue, const), the exact value category is forwarded with zero unnecessary copies.

Concept / Pattern

Why `auto&& self` beats `std::function` for recursion

- **Zero heap allocation** — purely stack-resident closure.
- **Fully inlineable** — compiler sees concrete type, can unroll and optimize.
- **Exact speed of a global function** — no indirect dispatch.
- Use `dfs(dfs, start, -1)` at call sites (pass the lambda as its own first arg).

```

1 // Competitive programming DFS with auto&& self:
2 void solve() {
3     int n; std::cin >> n;
4     std::vector<std::vector<int>>> adj(n + 1);
5     // ... read edges ...
6     std::vector<int> depth(n + 1, 0);
7
8     auto dfs = [&](auto&& self, int u, int p) -> void {
9         for (int v : adj[u]) {
10             if (v != p) {
11                 depth[v] = depth[u] + 1;
12                 self(self, v, u);
13             }
14         }
15     };
16     dfs(dfs, 1, -1);
17 }
```

14.4.9 `std::function` vs. `auto`: A Detailed Comparison

Concept / Pattern

`auto` vs `std::function` Reference Table

Property	<code>auto</code>	<code>std::function<></code>
Type resolution	Compile-time (static)	Runtime (polymorphic)
Size in memory	= size of captured vars (0 if empty)	32–48 bytes (internal buffer + metadata)
Heap allocation	Never (stack-only)	Yes, if captured state overflows internal buffer
Execution speed	Blazingly fast; compiler inlines body	Slower; indirect pointer jump per call
Re-assignability	One exact lambda type only	Can be reassigned to any compatible callable

Memory Profile Example

```

1 int counter = 0;
2 auto f_auto = [&counter](int x) { return x + counter; };
3 std::function<int(int)> f_func = [&counter](int x) { return x + counter; };
4 // f_auto --> exactly 8 bytes (one reference pointer)
5 // f_func --> 32-48 bytes (internal vtable/manager/invoke metadata)
```

Decision Rule: When to Use Which

Key Insight

auto vs std::function

- Use **auto** everywhere you can: sorting comparators, filter predicates, recursive DFS, local helpers. Zero overhead, fully inlinable.
- Use **std::function** only when you need to store heterogeneous callables (different lambdas, function pointers, or functors) in a single container (e.g., an event/action map):

```
1 std::map<std::string, std::function<void()>> actions;
2 actions["jump"] = []() { do_jump(); };
3 actions["duck"] = []() { do_duck(); };
4 // 'auto' is impossible here: every lambda has a distinct type.
```

14.4.10 Type Erasure: The Hidden Cost of std::function

Type erasure is what makes **std::function** polymorphic. It hides the concrete type of the stored callable behind a uniform interface. The overhead breaks down into three components:

1. **Heap allocation** — when the captured state is too large for **std::function**'s internal small-buffer (typically 16–32 bytes), it falls back to **new**, adding an allocator round-trip and a pointer chase on every call.
2. **Indirect dispatch** — every **operator()** call goes through a function pointer stored in an internal “manager” structure (similar to a virtual dispatch), defeating branch-prediction and preventing inlining.
3. **Type information overhead** — **std::function** must store enough metadata to correctly copy, move, and destroy the callable, even if you never do any of those operations.

Performance / Detail

Summary: Lambda Concepts Map

- **Lambda** = anonymous closure struct with **operator()** inline.
- **Capture by value** **[x]** = const snapshot. Use **mutable** to write.
- **Capture by reference** **[&x]** = live alias; dangling risk if lambda outlives scope.
- **Init-capture** **[v=expr]** = C++14; moves heavy objects into lambda ownership.
- **Recursive via std::function** = flexible but slow (type erasure overhead).
- **Recursive via auto&& self** = C++23 preferred; zero-cost, fully inlined.
- **auto storage** = fastest, immovable single type.
- **std::function storage** = polymorphic, re-assignable, but 3–10× slower per call.

14.5 Object Relations: Composition, Aggregation, Association & Inheritance

In C++, classes can relate to each other in four fundamental ways that differ in **ownership**, **lifetime coupling**, and **how tightly** the types are bound together.

Concept / Pattern

The Four Object Relations at a Glance

Relation	Ownership	Lifetime coupling	How it looks in code
Composition	Full (owns)	Child dies with parent	Member <i>by value</i> inside class
Aggregation	None (borrows)	Child survives parent	Member as <i>raw pointer</i> or reference
Association	None	Fully independent	Passed as parameter, no storage
Inheritance	Structural	Welded into one object	<code>class Child : public Parent</code>

14.5.1 Composition — “Has-a” with Ownership

The composed object is **owned**: it is stored *by value* as a member. When the owner is destroyed, the member is destroyed automatically with it — they share the same lifetime.

```

1  class Slave {
2      std::string s;
3  public:
4      Slave(std::string t) : s{t} {}
5      ~Slave() {
6          std::cout << "Sex Slave " << s << " : My master turned me off\n";
7      }
8  };
9  class Master {
10     std::string s;
11     Slave slave;           // owned by value -- composition
12 public:
13     Master(std::string s, std::string t) : s{s}, slave{t} {}
14     ~Master() {
15         std::cout << "Kinky Master " << s << " : Fuck it, I am done\n";
16     }
17 };
18 void solve() {
19     Master master{"A", "B"};
20 } // master goes out of scope here

```

Example

Output

Kinky Master A : Fuck it, I am done
Sex Slave B : My master turned me off

Key Insight

Why This Order?

Destructors run in **reverse construction order**. The `Master` is constructed first (which constructs `Slave` inside it), so `Master`'s destructor fires first, then `Slave`'s. The slave can only be freed after the master is done — it was owned by the master.

14.5.2 Aggregation — “Has-a” without Ownership

There is no ownership. The aggregating class holds a **pointer or reference** to an existing object. When the aggregator is destroyed, the pointed-to objects *survive* — their lifetime is managed externally.

```

1  class Chick {

```

```

2     i32 n;
3 public:
4     Chick(i32 n) : n{n} {}
5     ~Chick() {
6         std::cout << "Chick " << n << " : I am free from that harem\n";
7     }
8 };
9 class Harem {
10     std::vector<Chick*> harem;    // pointers only -- no ownership
11 public:
12     Harem() = default;
13     void add(Chick* ch) { harem.push_back(ch); }
14     ~Harem() { std::cout << "Harem broken\n"; }
15 };
16 void solve() {
17     Chick* ch1 = new Chick(1);
18     Chick* ch2 = new Chick(2);
19     {
20         Harem harem;
21         harem.add(ch1);
22         harem.add(ch2);
23     }    // harem destroyed here -- chicks are NOT deleted
24     delete ch1;
25     delete ch2;
26 }

```

Example

Output

```

Harem broken
Chick 1 : I am free from that harem
Chick 2 : I am free from that harem

```

Common Mistake

Aggregation Means Manual Lifetime Management

Because the aggregator does not own the objects, you are responsible for deleting them. Use `std::unique_ptr` (composition) or `std::shared_ptr` (shared ownership) to make this automatic in production code.

14.5.3 Association — “Uses-a” with No Storage

Fully independent objects that can *interact* but neither owns nor stores the other. The relationship is temporary — established through a function parameter or local variable, not a member.

```

1 class Girl {
2     i32 n;
3 public:
4     Girl(i32 n) : n{n} {}
5     ~Girl() { std::cout << "Girl " << n << " got dumped\n"; }
6     i32 getName() { return n; }
7 };
8 class Boy {
9     i32 n;
10 public:
11     Boy(i32 n) : n{n} {}
12     ~Boy() { std::cout << "Boy " << n << " got dumped\n"; }
13     void ask(Girl& g) {                                // Girl passed by reference, NOT stored
14         std::cout << "Boy " << n << " asks Girl " << g.getName()
15             << ", Fet's Luck and the girl agrees\n";
16     }
17 };

```

```

18 void solve() {
19     Girl g(1);
20     Boy b(2);
21     b.ask(g);
22 }

```

Example

Output

Boy 2 asks Girl 1, Fet's Luck and the girl agrees
 Boy 2 got dumped
 Girl 1 got dumped

Performance / Detail

How to Tell Association Apart from Aggregation

- **Aggregation:** The class holds a `Girl*` or `std::vector<Girl*>` as a *member variable* — it stores a reference to the object between calls.
- **Association:** The class only receives a `Girl&` as a *function parameter* — the connection exists only for the duration of that one call.

14.5.4 Inheritance — Tightest Coupling

Inheritance is the tightest relationship: the child class **physically contains** the parent's data inside the same memory block. They are not two separate objects; they are one combined object.

```

1  class Parent {
2  protected:
3      i32 money;
4  public:
5      Parent(i32 money) : money{money} {}
6  };
7  class Child : public Parent {
8  public:
9      Child(i32 val) : Parent(val) {}          // must initialize the Parent sub-layer
10     void getMoney(i32 val) {
11         if (money > val) {
12             std::cout << "Ok\n";
13             money -= val;
14         } else {
15             std::cout << "No\n";
16         }
17     }
18 };
19 void solve() {
20     Child ch(1500);
21     ch.getMoney(500);
22     ch.getMoney(1000);
23 }

```

Example

Output

Ok
 No

Why Must the Child Initialize the Parent?

This feels counterintuitive at first. The answer is a hardware fact:

Concept / Pattern

One Combined Memory Block

When you write `Child ch(500)`, the CPU allocates **one single, contiguous block of bytes**. The `Parent` member variables (`money`) and the `Child` member variables are physically welded together inside that block.

Because they are part of the same object, **you cannot instantiate a `Child` without also instantiating its `Parent` layer**. The `Child` constructor is the only entry gate for incoming data, so it must explicitly forward the required values to the `Parent` via the initializer list: `Child(i32 val) : Parent(val) {}`.

If C++ did not enforce this, the `Parent` layer would contain uninitialized garbage bytes, corrupting the object's state before the first line of `Child` code even runs.

Think of it like a house with a basement: you don't call the basement directly; you call the `House` constructor and hand it the blueprints. The house constructor internally handles pouring the basement (`Parent`) before building the walls (`Child`).

14.5.5 `std::initializer_list` for Custom Containers

By adding an `std::initializer_list` constructor to a custom container, you can initialize it with a brace-enclosed list of values directly:

```
1 // Without initializer_list constructor:
2 customVector1D<i32> v{1, 2}; // calls size constructor -> vector of size 1, value 2!
3
4 // After adding:
5 //   customVector1D(std::initializer_list<T> list) : ... {}
6 customVector1D<i32> v{1, 2}; // now creates vector {1, 2} -- two elements
```

Common Mistake

Brace Initialization Prefers `initializer_list`

Once you add an `initializer_list` constructor, the compiler **always** prefers it when you use `{}` syntax — even if another constructor would be a better match. To explicitly call the size constructor, use **parentheses**: `customVector1D<i32> v(5, 0)` creates a vector of 5 zeros.

Chapter 15

Advanced OOP: Virtual Dispatch and Inheritance Patterns

15.1 Advanced OOP: virtual, Vtables, Inheritance Patterns & Operator Overloading

15.1.1 What is Polymorphism?

At its core, **Polymorphism** (from Greek, meaning "many forms") is the ability in object-oriented programming to present the same interface for differing underlying forms (data types or classes).

In C++, it allows you to write generic code that can interact with objects of different derived classes through a common base class pointer or reference. Instead of writing separate functions for `Cat`, `Dog`, and `Bird`, you write a single function that accepts an `Animal` reference. The magic of polymorphism ensures that the correct, specific behavior for each animal is executed at runtime.

This decoupling allows software architectures to be highly extensible: you can add entirely new types of `Animals` to the codebase later, and the existing polymorphic code will automatically work with them without needing modification.

There are two main types of polymorphism in C++:

- **Compile-Time Polymorphism (Static Binding):** Achieved through function overloading and templates. Fast, but rigid.
- **Runtime Polymorphism (Dynamic Binding):** Achieved through inheritance and `virtual` functions. Flexible, but incurs a slight performance cost due to Vtable lookups.

Let's explore how Runtime Polymorphism works mechanically under the hood.

```
1  class Animal{
2  protected:
3      std::string s;
4      Animal(std::string_view s) : s{s}{}
5  public:
6      const std::string &getName() {return s;}
7      std::string_view speak(){return "??????";}
8  };
9  class Cat : public Animal{
10 public:
11     Cat(std::string n) : Animal{n}{}
12     std::string_view speak() const{return "MEow";}
13 };
14 void solve() {
15     Cat cat("Miku");
16     std::cout<<cat.speak()<<'\n';
17     Animal &ref = cat;
18     std::cout<<ref.speak()<<'\n';
```

19 }

Example

MEow ???????

Why? `Animal& ref` uses **static binding** at compile time, always calling `Animal::speak()` regardless of the actual `Cat` object. Base pointers cannot see derived overrides without `virtual`.

Solution: Mark the function `virtual` — this switches to **dynamic binding**, resolving the call at runtime based on the actual object type. Note: `virtual` is incompatible with `constexpr/constexpr`, but fully compatible with `const`.

```

1  class Animal{
2  protected:
3      std::string s;
4      Animal(std::string_view s) : s{s}{}
5  public:
6      const std::string &getName() const {return s;}
7      virtual std::string_view speak() const {return "???????";}
8      virtual ~Animal() = default;
9  };
10 class Cat : public Animal{
11 public:
12     Cat(std::string n) : Animal{n}{}
13     std::string_view speak() const override {return "Meow";}
14 };
15 void solve() {
16     Cat cat("Miku");
17     std::cout<<cat.speak()<<'\n';
18     Animal &ref = cat;
19     std::cout<<ref.speak()<<'\n';
20     std::cout<<ref.getName()<<'\n';
21 }

```

Example

Meow Meow Miku

With matching `const` qualifiers and `virtual` on the base, all three calls resolve correctly via the `Cat` Vtable — true **runtime polymorphism**. Let's break down exactly what each of these keywords (`virtual`, `override`, `const`, and `virtual Animal()`) is doing behind the scenes using your updated code as our master blueprint.

15.1.2 1. What `virtual` does here (The Runtime Detour)

Without `virtual`, C++ uses **static binding**. It looks at the type of the reference (`Animal& ref`) and hard-codes a jump straight to `Animal::speak()`.

By marking `virtual std::string_view speak() const` in the base class, you are telling the compiler to switch to **dynamic binding**.

- **The Vtable Allocation:** The moment the compiler sees `virtual`, it builds a hidden table of function pointers for both `Animal` and `Cat` called a **Virtual Table (Vtable)**.
- **The Pointer Injection:** When you create `Cat cat("Miku");`, the compiler sneaks a hidden pointer called a `vp_ptr` right at the very front of your object's memory block. This pointer targets the `Cat` Vtable.
- **The Resolution:** When your code hits `ref.speak()`, the CPU pauses. Instead of jumping blindly to the `Animal` class code, it follows `ref` down to the actual object in memory, grabs the hidden `vp_ptr`, looks inside the `Cat` Vtable, and finds the address for `Cat::speak()`. That's how `ref.speak()` prints `Meow` instead of `???????`.

15.1.3 2. Why you *had* to change `Animal::speak()` to `const`

In C++, `const` is a core part of a function's physical signature, just like its name or its parameters. To the compiler, these two functions are completely unrelated:

- `std::string_view speak();` → A function that is allowed to modify the object.
- `std::string_view speak() const;` → A completely distinct, read-only function.

If the base class version doesn't have `const`, but the child version does, **they don't match**. The compiler will assume that `Cat` has just created a brand-new, independent function that happens to share the same name, leaving the original non-`const` virtual function un-overridden.

When you call `ref.speak()`, since `ref` only knows about the non-`const` version, it wouldn't even see the child's `const` version. Synchronizing the `const` qualifier ensures they share the exact same slot in the Vtable map.

15.1.4 3. What `override` does (The Compiler Bodyguard)

The `override` keyword doesn't change how your code executes at runtime. It is a compile-time safety checker designed specifically to catch signature mistakes like the missing `const` you encountered!

When you append `override` to `Cat::speak()`, you are giving a strict command to the compiler:

"I am intending to override a virtual function from my parent class. If I made a typo in the name, forgot a `const`, or messed up the parameter types, flag it immediately and halt compilation."

Without `override`, if you made a signature mistake, the code would compile silently, but at runtime, polymorphism would fail and it would execute the parent's code instead. It transforms a frustrating runtime tracking bug into a clear compile-time error.

15.1.5 4. What the Virtual Destructor (`virtual Animal() = default;`) does here

Right now, in your `solve()` function, everything sits cleanly on the stack frame:

```
1 Cat cat("Miku");
```

When `solve()` exits, the stack unrolls, and the compiler safely cleans up the `Cat` layer first and the `Animal` layer second automatically. Here, a virtual destructor isn't strictly altering the output.

However, imagine if someone uses your classes later on the heap like this:

```
1 Animal* ptr = new Cat("Miku");
2 // ... use the object ...
3 delete ptr; // The Danger Zone
```

If `Animal()` were **not** marked as `virtual`, the compiler would use static binding on the pointer type (`Animal*`). It would execute *only* the `Animal` destructor, completely skipping the `Cat` destructor.

Even though `Cat` doesn't manage raw heap arrays right now, skipping a destructor breaks object lifecycles. By marking `virtual Animal() = default;`, you guarantee that if a developer calls `delete ptr` on a base pointer, the Vtable will route the cleanup sequence correctly: triggering `Cat()` first, which then cascades down to cleanly finish `Animal()`.

`override` is **100% a compile-time safety checker**. It adds zero overhead to your binary — no extra bytes, no machine code at runtime. Think of it as an insurance policy for your Vtable: it verifies that every intended override actually matches a virtual function in the base class.

15.1.6 The Silent Bug: What happens *without* `override`?

Before `override` was introduced in C++11, if you made a tiny typo while trying to override a parent function, the compiler wouldn't say a word. It would assume you were intentionally creating a **brand-new function** inside the child class.

Look at this dangerous example:

```

1 class Animal {
2 public:
3     virtual void speak() const { std::cout << "???\n"; }
4 };
5
6 class Cat : public Animal {
7 public:
8     // INTENT: Override speak()
9     // MISTAKE: Typo in the name! "speek" instead of "speak"
10    void speak() const { std::cout << "Meow\n"; }
11 };

```

This code compiles flawlessly. But look at what happens when you try to use polymorphism:

```

1 void solve() {
2     Cat my_cat;
3     Animal& ref = my_cat;
4
5     ref.speak(); // Expecting "Meow", but outputs: ???
6 }

```

Because `speak()` is treated as an entirely separate function, the `Animal` Vtable slot for `speak()` never gets updated with the child's logic. Finding this bug in a massive codebase can take hours because there are no compilation errors or warnings; the program simply behaves incorrectly at runtime.

15.1.7 How `override` Acts as Your Bodyguard

When you add the `override` keyword, you are explicitly making a claim to the compiler: *"I promise this function matches a virtual function in my parent class."*

```

1 class Cat : public Animal {
2 public:
3     // The compiler now actively cross-checks your claim
4     void speak() const override { std::cout << "Meow\n"; }
5 };

```

The moment the compiler sees `override`, it scans the `Animal` base class blueprint. It looks for a virtual function named `speak` that is `const` and takes no arguments.

When it doesn't find one, it immediately slams the brakes and throws a loud compilation error:

```
error: void Cat::speak() const marked override, but does not override
```

It catches your typo before your code can even finish building!

15.1.8 3 Sneaky Mistakes `override` Will Catch

Besides typos, `override` guards against three other signature mismatches:

1. **const Mismatch** — Base: `virtual void speak();` vs. Child: `void speak() const override;` → **Blocked.**
2. **Parameter Type Mismatch** — Base: `virtual void do_damage(double);` vs. Child: `void do_damage(int) override;` → **Blocked.** (`int` $\neq$ `double`)
3. **Return Type Mismatch** — Base: `virtual int get_id();` vs. Child: `long long get_id() override;` → **Blocked.**

15.1.9 Rule of Thumb

In modern C++, always write `override` when redefining a virtual function. It forces the compiler to do the verification at compile-time, making your runtime Vtable routing bulletproof.

15.1.10 The final Keyword

final is the companion to **override**. Where **override** says “*I am overriding this*”, **final** says “*nothing can override this further*”. It has two forms:

```

1 // 1. final on a CLASS -- no class may inherit from Cat
2 class Cat final : public Animal {
3 public:
4     std::string_view speak() const override { return "Meow"; }
5 };
6 // class Kitten : public Cat {}; // ERROR: cannot derive from final class
7
8 // 2. final on a FUNCTION -- no derived class may override this specific function
9 class Cat : public Animal {
10 public:
11     std::string_view speak() const override final { return "Meow"; }
12 };

```

Concept / Pattern

final and Devirtualization

final gives the compiler a compile-time guarantee that no subclass will ever change the function. It can therefore:

- Resolve the virtual call **statically** (no Vtable lookup).
- **Inline** the function body directly at the call site.

This is called **devirtualization** and can give significant performance gains in hot loops. A class or function marked **final** effectively opts back into the performance of a non-virtual call.

Common Mistake

final is a Commitment

Use **final** only when you are certain the hierarchy ends there. Marking a class **final** prevents any future extension, which can reduce flexibility in large codebases.

15.1.11 How Vtables Work Under the Hood

To understand **Vtables** (**Virtual Tables**), we need to look at how C++ manages memory for polymorphic types.

By default, C++ matches function calls at compile-time based on the declared type of your pointer or reference. But when you mark a function as **virtual**, you tell the C++ compiler: “*Hey, do not look at the type of the pointer. Wait until runtime, find the actual object in memory, and execute its version of the function.*”

To achieve this magic without sacrificing performance, the compiler implements a two-part mechanism: the **Vtable** and the **vptr**.

15.1.12 1. The Blueprint: What is a Vtable?

A **Vtable** is a static, read-only array of function pointers generated by the compiler at compile-time.

- There is exactly **one Vtable per class** that contains virtual functions.
- Each entry in the array holds the raw physical memory address of a virtual function implementation.

If you have a base class **Animal** with two virtual functions, its Vtable looks like an array of two function pointers. If a derived class **Cat** inherits from **Animal** and overrides only one of those functions, the compiler generates a *separate* Vtable for **Cat**. Inside **Cat**’s Vtable, the overridden function slot is updated to point to **Cat**’s implementation, while the un-overridden slot continues to point back to the base **Animal** implementation.

15.1.13 2. The Anchor: What is a `vp`tr?

A Vtable belongs to the *class blueprint*, not individual objects. So, how does an object instance find its corresponding Vtable at runtime?

The moment you add even a single virtual function to a class, the compiler secretly modifies the physical layout of that class. It injects a hidden member variable called a **vp**tr (**virtual pointer**) right at the very beginning of the object's memory block.

Whenever an object instance is constructed in RAM:

1. The constructor runs and sets its hidden **vp**tr to target its class's specific Vtable.
2. If it's a `Cat` object, its **vp**tr points to the `Cat` Vtable.
3. If it's a raw `Animal` object, its **vp**tr points to the `Animal` Vtable.

15.1.14 3. Step-by-Step Runtime Dispatch Trace

Let's look at exactly what happens inside the CPU when you execute a polymorphic call through a base reference:

```
1 Cat cat("Miku");
2 Animal& ref = cat;
3
4 ref.speak(); // <--- Dynamic binding triggers here!
```

Instead of jumping straight to a hardcoded function address, the CPU performs a fast 3-step lookup at runtime:

1. **Follow the Reference:** The CPU goes to the memory address where `ref` points. Because of our reference rule, it finds the fully intact `Cat` object sitting on the stack.
2. **Grab the `vp`tr:** The CPU reads the hidden bytes at the start of the object block to access the **vp**tr.
3. **Index the Vtable:** The **vp**tr leads the CPU straight to the `Cat` Vtable array. It looks up the slot reserved for `speak()`, extracts the memory address of `Cat::speak()`, and jumps execution directly to the "Meow" logic.

This entire detour takes just a fraction of a nanosecond, which is why C++ can achieve true runtime flexibility while maintaining incredibly high runtime speeds.

```
1 class Animal{
2 protected:
3     std::string s;
4     Animal(std::string_view s) : s{s}{}
5 public:
6     const std::string &getName() const {return s;}
7     virtual std::string_view speak() {return "??????";}
8     virtual std::string_view hello() {return "locked";}
9     virtual ~Animal() = default;
10 };
11 class Cat : public Animal{
12 public:
13     Cat(std::string n) : Animal{n}{}
14     std::string_view hello() {return "unlocked";}
15     std::string_view speak() override {return "Meow";}
16 };
17 //override is just a safety measure
18 void solve() {
19     Cat cat("Miku");
20     std::cout<<cat.speak()<<'\n';
21     Animal &ref = cat;
22     std::cout<<ref.hello()<<'\n';
23     std::cout<<ref.speak()<<'\n';
24     std::cout<<ref.getName()<<'\n';
25 }
```


Notice there is no `override` keyword on `hello()` in `Cat` — yet it printed `unlocked`. That is because **virtualness is inherited downwards implicitly**: any function with a matching signature automatically occupies the correct Vtable slot.

Let's trace exactly how the compiler built the Vtables for your code to see why this happened.

15.1.15 How the Vtables Are Built (Slot-by-Slot)

Because `Animal` has three virtual functions (including the destructor), the compiler reserves **3 slots** in the Vtable for `Animal`, and a matching **3 slots** for `Cat`.

Here is exactly what those two Vtables look like in your compiled binary:

The Animal Vtable Layout

Slot Index	Function Slot	Points To Physical Address Of...
[0]	<code>Animal()</code>	<code>Animal::Animal()</code> (Base Destructor)
[1]	<code>speak()</code>	<code>Animal::speak()</code> (Returns "??????")
[2]	<code>hello()</code>	<code>Animal::hello()</code> (Returns "locked")

The Cat Vtable Layout

When `Cat` inherits from `Animal`, the compiler copies the structure of the `Animal` Vtable and checks which functions the child re-implemented with a matching name and signature:

Slot Index	Function Slot	Points To Physical Address Of...
[0]	<code>Animal()</code>	<code>Cat::Cat()</code> (Automatically updated child destructor)
[1]	<code>speak()</code>	<code>Cat::speak()</code> (Updated because of explicit override)
[2]	<code>hello()</code>	<code>Cat::hello()</code> (Implicitly updated!)

Because `Cat` defined a function with the exact same name (`hello`), same return type (`std::string_view`), and same parameters (none) as the virtual `hello()` in `Animal`, the compiler automatically hijacked Slot [2] in `Cat`'s Vtable and forced it to point straight to `Cat::hello()`.

15.1.16 Tracing `ref.hello()` at Runtime

When your code executes this sequence:

```
1 Animal &ref = cat;
2 std::cout << ref.hello() << '\n';
```

1. The CPU looks at `ref` and finds the physical `cat` object on the stack frame.
2. It reads `cat`'s hidden `vp_ptr`, which points directly to the `Cat` Vtable.
3. It looks up Slot [2] (the slot designated for `hello`).
4. Because that slot was overwritten by the compiler during setup, the CPU pulls out the address for `Cat::hello()` and jumps straight to it, printing `unlocked`.

15.1.17 Why `override` Is Mandatory for Safety

This behavior is exactly why the `override` keyword was invented in C++11.

Writing `override` is **optional** for functionality, but **mandatory for safety**. Because C++ implicitly overrides functions with matching signatures behind your back, you can accidentally override a parent function without meaning to, or fail to override one because of a hidden signature mismatch (like a missing `const`).

If you want to make your code clear and self-documenting, adding the `explicit` tag ensures anyone reading it knows exactly which Vtable slots are being remapped:

```

1 class Cat : public Animal {
2 public:
3     Cat(std::string n) : Animal{n}{}
4     std::string_view hello() override { return "unlocked"; } // Explicitly clear!
5     std::string_view speak() override { return "Meow"; }
6 };

```

15.1.18 Virtual Calls vs. Inlining

Virtual calls are the primary enemy of compiler inlining — one of the largest hidden costs of runtime polymorphism. Here is why, and how the compiler fights back with **Devirtualization**.

What Is Inlining?

Inlining is a crucial optimization performed by the compiler. When a function is simple, the compiler physically copy-pastes the body of that function directly into the place where it was called, completely eliminating the overhead of a function call.

Consider this non-virtual example:

```

1 struct Calculator {
2     int add(int a, int b) { return a + b; } // Non-virtual
3 };
4
5 void solve() {
6     Calculator calc;
7     int result = calc.add(5, 10);
8 }

```

The compiler can see *exactly* what `calc.add()` does at compile-time. Instead of generating machine instructions to push variables onto the stack, jump to the function's memory address, and return back, it simply rewrites your binary code to look like this:

```

1 void solve() {
2     int result = 5 + 10; // Perfectly inlined! Zero function call overhead.
3 }

```

Why Virtual Calls Kill Inlining

Now, let's look at your polymorphic code using the `Animal` pointer context:

```

1 void execute_speak(Animal* ptr) {
2     std::cout << ptr->speak() << '\n'; // Can this be inlined?
3 }

```

When the compiler is building the binary for `execute_speak`, it looks at `ptr->speak()`.

- At compile-time, **the compiler has no idea what `ptr` is pointing to**. It could be a `Cat`, a `Dog`, or a raw `Animal`.
- Because it cannot predict the object type, it cannot copy-paste the function body into place.

Instead, it generates a runtime lookup: *go to `ptr`, read the hidden `vptr`, jump to the `Vtable`, look up the slot, jump to that address* — no inlining possible.

Because inlining requires knowing **exactly what code to copy at compile-time**, the runtime uncertainty of a **virtual** call completely shuts down this optimization.

The Structural Cost of Lost Inlining

Losing out on inlining doesn't just mean you pay the tiny penalty of a function pointer jump. It triggers a domino effect of lost optimizations:

- **Instruction Cache Misses:** The CPU has to jump to a completely different sector of memory to execute the child function body, breaking the linearity of the CPU instruction pipeline.
- **Dead-End Optimizations:** When a function is inlined, the compiler can perform loops unrolling, constant propagation, and vectorization based on the copy-pasted code context. A virtual function call acts as an absolute wall that stops the compiler from looking ahead.

How the Compiler Fights Back: Devirtualization

Modern optimizing compilers (like GCC and Clang with `-O2` or `-O3`) try to bypass this restriction using a clever optimization trick called **Devirtualization**.

If the compiler can definitively prove the exact type of an object at compile-time, it will bypass the Vtable entirely, turn the virtual call back into a standard static call, and inline it anyway.

Case A: Explicit Local Type (Inlined!)

```
1 void solve() {
2     Cat cat("Miku");
3     // The compiler knows 100% that 'cat' is a Cat object.
4     // It bypasses the Vtable, resolves it to Cat::speak(), and INLINES "Meow".
5     std::cout << cat.speak() << '\n';
6 }
```

Case B: The final Keyword (Inlined!)

If you mark a child class or a specific override with the `final` keyword, you tell the compiler: *"No other class can ever inherit from this or override this function downstream."*

```
1 class Cat final : public Animal { // 'final' means the lineage ends here
2 public:
3     std::string_view speak() const override { return "Meow"; }
4 };
5
6 void execute(Cat* cat_ptr) {
7     // Even through a pointer, the compiler knows no subclass can override Cat::speak().
8     // It safely devirtualizes and INLINES the code!
9     std::cout << cat_ptr->speak() << '\n';
10 }
```

Concept / Pattern

Inlining Summary

Virtual calls **prevent inlining** whenever the object type is ambiguous at compile-time. In hot loops, prefer compile-time polymorphism (templates, if `constexpr`, CRTP).

15.1.19 Four Rules of Safe Polymorphic Code

Rule 1: Virtual Only Works Through Pointers or References

This rule comes down to a concept called **Object Slicing** and how memory allocation works.

If you assign a child object directly to a base object variable **by value** (instead of using a pointer or reference), the compiler forces the child to fit into the smaller memory footprint of the parent. It literally "slices off" the child portion, along with its virtual capabilities.

```

1 void solve() {
2     Cat cat("Miku");
3
4     // BY VALUE (Slicing happens!)
5     Animal normal_var = cat;
6     std::cout << normal_var.speak() << '\n'; // Prints: "???????" (Virtual fails!)
7
8     // BY REFERENCE
9     Animal& ref = cat;
10    std::cout << ref.speak() << '\n';          // Prints: "Meow" (Virtual works!)
11 }

```

Why? When you write `Animal normal_var = cat;`, the compiler allocates space on the stack specifically for a raw `Animal` object. An `Animal` object's hidden `vptr` points strictly to the `Animal` Vtable. The compiler copies over the parent variables from `cat` into `normal_var` and throws away everything that made it a `Cat`.

Pointers (`Animal*`) and references (`Animal&`) avoid this because they do not alter or move the underlying memory. They just act as window frames pointing to the original, fully intact `Cat` object sitting elsewhere on the stack or heap, allowing you to read its hidden `vptr` and access the `Cat` Vtable.

Rule 2: Virtualness Propagates Downward Implicitly

This means that once a function is marked `virtual` at the top of the inheritance family tree, **it stays virtual all the way down the lineage**, even if you omit the `virtual` or `override` keywords in the child classes.

```

1 class Animal {
2 public:
3     virtual void speak() const; // Explicitly virtual
4 };
5
6 class Cat : public Animal {
7 public:
8     void speak() const; // IMPLICITLY virtual! (Even without keywords)
9 };
10
11 class Kitten : public Cat {
12 public:
13     void speak() const override; // Still overrides perfectly from Cat/Animal!
14 };

```

Why? When the compiler builds the Vtable layout for `Animal`, it marks the slot for `speak()` as a dynamic entry. When `Cat` inherits from `Animal`, it inherits that exact layout. Even if you don't write the word `virtual` inside `Cat`, the compiler knows that `speak()` occupies a dynamic Vtable slot.

Therefore, if a new class `Kitten` inherits from `Cat`, `Kitten` can still override `speak()`, and it will still dynamically dispatch perfectly through an `Animal` pointer.

Rule 3: Virtualness Does Not Propagate Upward

This means that **inheritance flows downwards, not upwards**. If a function is a standard, normal function in the parent class, but you suddenly decide to make it `virtual` inside a child class, it does *not* retroactively make the parent class function virtual.

```

1 class Animal {
2 public:
3     void sleep() const { std::cout << "Zzz\n"; } // Normal function
4 };
5
6 class Cat : public Animal {
7 public:
8     virtual void sleep() const { std::cout << "Cat nap\n"; } // Virtual starts here!
9 };

```

If you have an `Animal` reference pointing to a `Cat`, the `Animal` class profile has absolutely no entry for a virtual `sleep()` function in its Vtable layout.

```

1 void solve() {
2     Cat cat;
3     Animal& ref = cat;
4     ref.sleep(); // Static binding! Prints: "Zzz" (Ignores the child's virtual function)
5 }

```

Virtual dispatch can only look downstream toward specialized implementations; it cannot look upstream to rewrite how the parent base class processes code.

Rule 4: Never Call Virtual Functions from Constructors or Destructors

This is one of the most critical safety rules in C++. In C++, **an object is built from the bottom-up (Parent first) and destroyed from the top-down (Child first)**. While a parent constructor is currently running, the child layer **does not exist yet** in memory. While a parent destructor is running, the child layer **has already been destroyed**.

Therefore, calling a virtual function inside a constructor or destructor will bypass the Vtable and execute the *parent's* version of the function, completely ignoring the child override.

```

1 class Animal {
2 public:
3     Animal() {
4         // DANGER: Trying to use polymorphism during birth
5         std::cout << "Born: " << speak() << '\n';
6     }
7     virtual std::string_view speak() const { return "???"; }
8 };
9
10 class Cat : public Animal {
11 public:
12     Cat() : Animal() {}
13     std::string_view speak() const override { return "Meow"; }
14 };
15
16 void solve() {
17     Cat c;
18 }

```

Example

Born: ???

Why ??? instead of Meow?

When `Cat c;` begins initializing, it immediately runs the `Animal` constructor first. At that precise millisecond, the `Cat` constructor hasn't executed, its custom member variables aren't initialized, and its hidden `vp_ptr` is still pointing strictly to the `Animal` Vtable.

Because the object is technically still just a raw `Animal`, C++ locks down dynamic dispatch and safely resolves the call to `Animal::speak()`. If it mistakenly tried to call `Cat::speak()`, the function might try to read uninitialized `Cat` variables, leading to instant undefined behavior or a crash.

15.1.20 Abstract Classes vs. Interface Classes

Abstract Classes and **Interface Classes** both use **pure virtual functions**, but serve very different design roles.

Abstract Class: The Partial Blueprint

An **Abstract Class** is a class designed to capture shared attributes and common behavior for a family of related objects. It acts as a *partial blueprint* it can contain fully implemented member functions and structural state variables alongside un-implemented placeholders.

You turn a regular class into an abstract class by declaring at least one **Pure Virtual Function**. You do this by appending `= 0` to the function signature.

```

1 #include <iostream>
2 #include <string>
3
4 // ABSTRACT CLASS: Represents a conceptual category
5 class Asset {
6 protected:
7     std::string ticker; // State Variable: Allowed in abstract classes!
8     double price;
9
10 public:
11     Asset(std::string t, double p) : ticker{t}, price{p} {}
12
13     // 1. Concrete Function: Shared implementation across all child types
14     void printDetails() const {
15         std::cout << "Asset: " << ticker << " | Price: $" << price << "\n";
16     }
17
18     // 2. Pure Virtual Function: Mandates specialized downstream implementation
19     virtual double calculateRisk() const = 0;
20
21     virtual ~Asset() = default;
22 };

```

Core Rule:

You cannot instantiate an abstract class. Writing `Asset my_asset("BTC", 60000);` will trigger an immediate compiler error. It can *only* exist as a base reference or pointer (`Asset*`) pointing to a fully completed, concrete child class (like `Stock` or `Option`) that has implemented the `calculateRisk()` logic.

Interface Class: The Strict Contract

An **Interface Class** is a hyper-restricted, pure subset of an abstract class. It represents a strict **architectural contract** that dictates pure capability or behavior (*"What an object can do"*), with absolutely zero regard for state data or implementation details (*"How it is stored or done"*).

In C++, an interface class is defined by adhering to two strict structural rules:

1. Every single member function must be a **pure virtual function**.
2. It must contain **no member variables** (no state tracking).

```

1 #include <string_view>
2
3 // INTERFACE CLASS: Represents a capability/contract
4 class IRenderable {
5 public:
6     // Pure behavior contract
7     virtual void draw() const = 0;
8     virtual std::string_view getLayerName() const = 0;
9
10     // Interfaces MUST have a virtual destructor to prevent partial deletion memory leaks
11     virtual ~IRenderable() = default;
12 };

```

Key Insight

Naming Convention

In professional C++, interfaces are universally prefixed with `I` (e.g., `IRenderable`, `IOrderManager`) to signal that the class contains no state or concrete logic.

Abstract Class vs. Interface: The Definitive Comparison

To see how they diverge structurally and philosophically, look at their technical requirements side-by-side:

Feature	Abstract Class	Interface Class
Language Feature	Native structural class layout.	Achieved by restricting abstract
Pure Virtual Functions	Must have at least one .	All member functions must be
Concrete Functions	Allowed . Can have complete functions with logic blocks.	Banned . Cannot contain any l
Member Variables	Allowed . Can track state variables (<code>int</code> , <code>string</code> , vectors).	Banned . Cannot hold structur
Relationship Model	Models an "Is-A" relationship (e.g., <i>Gold</i> is an <i>Asset</i>).	Models a "Can-Do" capability
Constructor	Allowed . Requires a constructor to initialize its state variables.	Banned . Has no state variable

When to Use Which?

Use an Abstract Class when eliminating duplicate code across a family of related classes:

If you are writing three classes that share 80% of the exact same internal structural layout and require identical logic helper functions, bundle that matching code up into an Abstract Class. The children inherit the completed foundational architecture for free and only write custom overrides for the remaining 20%.

Use an Interface Class for loose coupling and safe multiple inheritance:

If you want to decouple systems completely, use an Interface. For example, if you are building an automated trading execution pipeline, you can write an `IExecutionEngine` interface.

Your main framework code can interact purely with `IExecutionEngine&` references. Whether the underlying implementation is a `BinanceEngine`, a `BybitEngine`, or a local `BacktestEngine`, your core logic remains completely unchanged because it relies on the abstract behavioral contract, not a specific, hardcoded data layout.

Furthermore, because interfaces contain no data variables, **inheriting from multiple interfaces is completely safe**. It completely bypasses the dreaded "Diamond Problem" of multiple inheritance because there are no duplicate parent variables to conflict with each other in memory!

```
1 // A concrete class can inherit from one structural base class,
2 // but fulfill as many pure interface capability contracts as it needs!
3 class CryptoAsset : public Asset, public IRenderable, public ISerializable {
4     // ... implements all contracts cleanly with zero memory layout conflicts
5 };
```

15.1.21 Multiple Inheritance and the Diamond Problem

Multiple Inheritance allows a single derived class to inherit from **more than one parent base class** simultaneously.

While it is an incredibly powerful architectural tool, it is also one of the most controversial features in object-oriented programming because it introduces complex physical memory layouts and a famous structural trap called **The Diamond Problem**.

The Basic Model: Combining Two Unrelated Parents

In its simplest, cleanest form, multiple inheritance is used to combine completely distinct sets of behaviors or attributes into a single unified object.


```

1 #include <iostream>
2 #include <string>
3
4 // Parent Class 1
5 class Logger {
6 public:
7     void log(const std::string& msg) {
8         std::cout << "[LOG]: " << msg << "\n";
9     }
10 };
11
12 // Parent Class 2
13 class Transmitter {
14 public:
15     void transmit() {
16         std::cout << "Broadcasting data packets over network...\n";
17     }
18 };
19
20 // Child Class inheriting from BOTH parents cleanly
21 class SmartRadio : public Logger, public Transmitter {
22 public:
23     void operate() {
24         log("Radio powering up"); // From Logger
25         transmit();               // From Transmitter
26     }
27 };
28
29 void solve() {
30     SmartRadio radio;
31     radio.operate();
32 }

```

15.1.22 2. What it looks like in Memory

When you instantiate an object that Uses multiple inheritance, the C++ compiler flattens the parents side-by-side into **one contiguous block of physical RAM**.

```

1 [ SmartRadio Object Memory Block ]
2 [ Logger Layer ][ Transmitter Layer ][ SmartRadio Layer ]
3 [ (Bytes for L) ][ (Bytes for T) ][ (Bytes for SR) ]
4 \-----/
5 Allocated Contiguously in RAM

```

Because the memory blocks sit side-by-side, casting a `SmartRadio*` pointer to a `Transmitter*` pointer actually causes the CPU to perform a tiny pointer adjustment (adding a byte offset) so it points exactly to the start of the `Transmitter` data sub-block.

15.1.23 3. The Fatal Flaw: The Diamond Problem

Multiple inheritance works beautifully until your inheritance hierarchy branches out into a diamond shape.

Imagine you have a base class `Device`. Two classes, `Copier` and `Printer`, inherit from `Device`. Then, a specialized class called `MultifunctionMachine` inherits from both `Copier` and `Printer`.

```

1 [ Device ] (Contains member variable 'id')
2 /       \
3 [ Copier ] [ Printer ]
4 \       /
5 [ MultifunctionMachine ]

```


15.1.24 The Broken Code Layout

```

1  class Device {
2  public:
3      int id;
4  };
5
6  class Copier : public Device {};
7  class Printer : public Device {};
8
9  // The Trap: Inheriting from both
10 class MultifunctionMachine : public Copier, public Printer {
11 public:
12     void showId() {
13         // COMPILER ERROR: "reference to 'id' is ambiguous"
14         // std::cout << id << '\n';
15     }
16 };

```

15.1.25 Why does this fail?

Because `Copier` inherits from `Device`, it gets its own copy of the `id` variable. Because `Printer` also inherits from `Device`, it *also* gets its own separate copy of the `id` variable.

When `MultifunctionMachine` combines both paths, its physical memory block contains **two separate instances of the `Device` base layer**:

When you type `id` inside the child class, the compiler looks at the layout, sees two completely different variables named `id` (one inside the `Copier` lane and one inside the `Printer` lane), gets confused, throws an **ambiguity error**, and halts compilation.

The Solution: Virtual Inheritance

To fix this architectural flaw, C++ introduces **Virtual Inheritance**. By adding the `virtual` keyword when inheriting from the topmost base class, you instruct the compiler: *"Ensure only one single, shared instance of this base class ever exists in the final child memory layout."*

```

1  #include <iostream>
2
3  class Device {
4  public:
5      int id = 999;
6  };
7
8  // FIX: Marking inheritance as virtual
9  class Copier : virtual public Device {};
10 class Printer : virtual public Device {};
11
12 // The Diamond resolves perfectly now
13 class MultifunctionMachine : public Copier, public Printer {
14 public:
15     void showId() {
16         // Compiles perfectly! Only one 'id' exists now.
17         std::cout << "Device ID: " << id << '\n';
18     }
19 };
20
21 void solve() {
22     MultifunctionMachine mfm;
23     mfm.showId();
24 }

```

15.1.26 How Virtual Inheritance Works Under the Hood

When you use `virtual public Device`, the compiler restructures the object layout. It shifts the shared `Device` layer all the way to the absolute end of the complete object block.

Instead of embedding `Device` directly inside `Copier` and `Printer`, it embeds a hidden pointer called a **Virtual Base Pointer (vbptr)** into them. These pointers point downstream to the single, shared `Device` block at the bottom, eliminating duplication entirely.

15.1.27 5. Modern Best Practice: Mixin Interfaces

Because managing virtual base pointers, offsets, and constructor ordering trees can become incredibly messy, modern software architecture strongly restricts multiple inheritance to **Interface Classes**.

If your parent classes are pure interfaces (containing absolutely no member variables and only pure virtual functions), **the Diamond Problem is physically impossible**. There are no duplicate parent variables to conflict with each other in memory, meaning you can safely combine as many interface capability contracts as your architecture requires!

```
1 // Clean, professional engineering design:
2 class CryptoTrader : public Asset, public ISerializable, public IRenderable {
3     // 1 structural parent state layer + multiple pure behavior contracts
4 };
```

You are on the right track conceptually, but they are actually **two completely different hidden systems** running on different pointers!

While both use dynamic offsets to handle runtime uncertainty, they solve two entirely separate architectural problems.

Here is the exact distinction:

- **Virtual Functions** handle *behavioral* uncertainty (Which function logic do I execute?). They use a **vptr** to find a **Vtable**.
- **Virtual Inheritance** handles *structural* uncertainty (Where do the base variables physically live?). They use a **vbptr** to find a **Vtable / Offset Map**.

15.1.28 1. Side-by-Side Comparison of the Hidden Pointers

If you combine both features into a single class hierarchy, a single object can actually contain **both** hidden pointers at the same time:

```
1 class b {
2 public:
3     int data;
4     virtual void speak() {} // Virtual function
5 };
6
7 class a : virtual public b {}; // Virtual inheritance
```

If you look inside the physical memory layout of an object of class `a`, the compiler has injected two distinct navigation tools:

```
1 [          Object of Class a Memory Block          ]
2 [ a component ][ ... ][          Shared Base Component b          ]
3 [ hidden vbptr ] -----> [ hidden vptr ][          int data          ]
4 |                     |
5 |                     v
6 v                     Points to: Vtable (Array of function pointers)
7 Points to: Offset Map
8 (Tells CPU where 'b' lives)
```

15.1.29 2. The Functional Differences

To see how they operate independently under the hood, look at what happens when the CPU processes your code:

The `vptr` (Virtual Function Pointer)

- **When it triggers:** When you call a function like `ptr->speak()`.
- **The Destination:** It leads the CPU to an array of **executable machine code addresses** (The Vtable).
- **The Goal:** It determines *what logic to run* because a child class might have overridden the function body.

The `vbptr` (Virtual Base Pointer)

- **When it triggers:** When you read or write to a variable like `obj.data`.
- **The Destination:** It leads the CPU to an **integer offset value** inside a structure map.
- **The Goal:** It determines *where the memory variables are located*. Because multiple inheritance shifted `b` to the very end of the shared memory block, the CPU needs the `vbptr` just to find the physical coordinates of data before it can read or write to it.

15.1.30 Summary Rule of Thumb

Think of them as two different maps used by the compiler:

- **Virtual Functions (`vptr`)** → A map of **Functions** (Addresses of code blocks).
- **Virtual Inheritance (`vbptr`)** → A map of **Variables** (Offsets of physical byte locations).

They look similar in syntax because they both use the word `virtual`, but one manages your code's *logic flow* while the other manages your object's *physical shape in RAM*. You are looking for **`dynamic_cast`** (with a slight typo at the end!).

In C++, `dynamic_cast` is a specialized, type-safe casting operator used exclusively for **Downcasting** in polymorphic inheritance hierarchies. It allows you to safely ask the question at runtime: “*Hey, is this generic `Animal*` pointer actually pointing to a specific `Cat*` object right now?*”

Unlike other casts (like `static_cast` or C-style casts) which blindly force a conversion at compile-time, `dynamic_cast` **checks the hidden Vtable details at runtime** to verify if the conversion is valid.

15.1.31 1. The Core Syntax and How It Works

Imagine you have your standard `Animal` and `Cat` setup. You are holding a base reference or pointer, and you want to safely convert it downstream to invoke a child-specific capability.

```

1 #include <iostream>
2 #include <string>
3
4 class Animal {
5 public:
6     virtual ~Animal() = default; // CRITICAL: Dynamic cast REQUIRES at least one virtual
        function!
7 };
8
9 class Cat : public Animal {
10 public:
11     void meow() const { std::cout << "Meow!\n"; }
12 };
13
14 class Dog : public Animal {
15 public:
16     void bark() const { std::cout << "Woof!\n"; }
17 };

```

Here is how you use `dynamic_cast` inside your execution block:

```

1 void pet_animal(Animal* animal_ptr) {
2     // Attempting to safely cast downstream to a Cat pointer
3     Cat* cat_ptr = dynamic_cast<Cat*>(animal_ptr);
4
5     if (cat_ptr != nullptr) {
6         // SUCCESS: The underlying object is definitively a Cat
7         cat_ptr->meow();
8     } else {
9         // FAIL: 'animal_ptr' was actually a Dog or raw Animal
10        std::cout << "Cast failed! This animal is not a cat.\n";
11    }
12 }
13
14 void solve() {
15     Cat c;
16     Dog d;
17
18     pet_animal(&c); // Outputs: Meow!
19     pet_animal(&d); // Outputs: Cast failed! This animal is not a cat.
20 }

```

15.1.32 2. Pointers vs. References: The Error Handling Catch

`dynamic_cast` behaves differently depending on whether you are converting a pointer or a reference, because a reference in C++ cannot be set to `nullptr`.

Casting Pointers (Target*)

If the cast fails at runtime, `dynamic_cast` returns `nullptr`. This is incredibly clean because you can use a simple `if` condition to handle the failure gracefully.

Casting References (Target&)

If you pass a reference and the conversion is physically impossible, it cannot return `nullptr`. Instead, it throws a runtime exception: `std::bad_cast`. You must wrap it in a `try-catch` block to prevent your program from crashing.

```

1 void pet_reference(Animal& animal_ref) {
2     try {
3         Cat& cat_ref = dynamic_cast<Cat&>(animal_ref);
4         cat_ref.meow();
5     } catch (const std::bad_cast& e) {
6         std::cout << "Caught bad_cast: Object is not a Cat!\n";
7     }
8 }

```

15.1.33 3. How does it check types at runtime? (RTTI)

`dynamic_cast` is powered by a C++ subsystem called **RTTI (Run-Time Type Information)**.

Remember how virtual functions use a hidden `vp_ptr` inside the object to point to a class Vtable? The compiler embeds an extra hidden metadata pointer inside that Vtable layout that points to a **type_info object descriptor block**.

When your CPU executes `dynamic_cast<Cat*>(animal_ptr)`:

1. It reads the `vp_ptr` of the object passed.
2. It follows it to the Vtable and extracts the RTTI descriptor tracking block.
3. It checks if the type descriptor matches or derives from the `Cat` metadata profile.

4. If it matches, it computes the correct memory address offset and returns the pointer. If it doesn't match, it returns `nullptr`.

15.1.34 4. The Golden Rules of `dynamic_cast`

- **Must be Polymorphic:** You can *only* use `dynamic_cast` on classes that have at least one virtual function (typically the virtual destructor). Without a Vtable, there is no RTTI tracking block embedded in memory, and your code will fail to compile.
- **Performance Cost:** Because it performs a string/metadata pointer comparison chain at runtime through the Vtable layout, it is significantly slower than compile-time alternatives like `static_cast`. Use it selectively in performance-critical tracking environments.

Covariant return types represent a very neat, clean exception to the strict signature-matching rules of C++ virtual functions.

Normally, when you override a virtual function in a child class, the return type must match the parent class signature **exactly**. However, if a virtual function returns a **pointer or a reference to a class**, C++ allows the child class override to return a pointer or reference to a **more specific, derived class** instead.

This allows you to preserve specific type safety when working with polymorphic factories or cloners, without needing messy `dynamic_cast` operators later.

15.1.35 1. The Code Blueprint: A Polymorphic Factory

The classic use case for covariant return types is a cloning mechanism (often called the Prototype Design Pattern) or a factory.

Let's look at how your `Animal` and `Cat` setup implements this cleanly:

```

1  #include <iostream>
2  #include <string>
3  #include <memory>
4
5  class Animal {
6  public:
7      virtual ~Animal() = default;
8
9      // Base function returns a pointer to the Base class
10     virtual Animal* clone() const = 0;
11     virtual void speak() const = 0;
12 };
13
14 class Cat : public Animal {
15 private:
16     std::string name;
17 public:
18     Cat(std::string n) : name{n} {}
19
20     // COVARIANT RETURN TYPE: Returning Cat* instead of Animal*!
21     // This is 100% legal because Cat inherits from Animal.
22     Cat* clone() const override {
23         return new Cat(*this);
24     }
25
26     void speak() const override {
27         std::cout << name << " says: Meow!\n";
28     }
29
30     void purr() const {
31         std::cout << name << " purrs...\n";
32     }
33 };

```

15.1.36 2. The Payoff: Eliminating Type Slicing and Casts

Look at how much cleaner your application layer becomes depending on how you instantiate the object:

Case A: Working through the Base Pointer (Standard Polymorphism)

If you are treating your object generally, it works exactly like standard dynamic binding:

```
1 void duplicate_and_sound(const Animal* original) {
2     // Returns an Animal* because 'original' is an Animal tracking pointer
3     Animal* copy = original->clone();
4     copy->speak();
5     delete copy;
6 }
```

Case B: Working directly with the Derived Class (The Covariant Benefit)

If you are already working with a concrete `Cat` object, calling `.clone()` returns a specific `Cat*` directly to you. **You don't lose the object's identity.**

```
1 void solve() {
2     Cat specific_cat("Miku");
3
4     // Without covariant return types, clone() would return an Animal*,
5     // forcing you to use dynamic_cast<Cat*> just to access Cat-specific methods.
6     Cat* cloned_cat = specific_cat.clone();
7
8     cloned_cat->speak(); // From Animal layer
9     cloned_cat->purr();  // Cat-specific method works directly with NO casting!
10
11     delete cloned_cat;
12 }
```

How Covariant Returns Work Under the Hood

When you compile this code, the compiler updates the Vtable slot for `clone()` inside the `Cat` class profile to point to `Cat::clone()`.

At runtime, the CPU performs the standard Vtable pointer jump. However, behind the scenes, if a base reference (`Animal& ref`) invoked the call, the compiler automatically injects a tiny pointer adjustment offset if the memory alignment layout of the `Cat` sub-block differs from the `Animal` layer. It silently handles the upstream casting safety automatically.

15.1.37 4. The Strict Constraints of Covariance

To use covariant return types, your code must satisfy three absolute rules:

1. **Must be Pointers or References:** The return types must be `Base*` / `Derived*` or `Base&` / `Derived&`. It does **not** work by value (e.g., returning a raw `Animal` vs `Cat`) because that would trigger compile-time object slicing.
2. **Valid Inheritance Path:** The derived return type must be publicly and unambiguously inherited from the base return type.
3. **No Smart Pointers (Raw Only):** Covariant return types **do not work natively** with templates like `std::unique_ptr<Animal>` and `std::unique_ptr<Cat>`. Because `std::unique_ptr<T>` is a distinct wrapper class rather than a raw reference pointer, the compiler treats them as entirely unrelated types, and the override will fail to compile.

Operator overloading inside derived (child) classes is an area where C++ object-oriented design and value semantics collide. It can introduce subtle bugs like **member variable amnesia** (forgetting to copy the parent's data) or **object slicing** if not handled with care.

Here is the complete guide to handling operator overloading safely within an inheritance hierarchy.

15.1.38 1. The Core Trap: The Assignment Operator (operator=)

When you write a derived class, C++ automatically generates a default assignment operator for you. This default operator is smart: it automatically calls the parent class's assignment operator first, and then copies the child's unique variables.

However, the moment you need to write a **custom** `operator=` in the derived class (for example, if the child manages a raw heap pointer), **you inherit the responsibility of manually updating the parent layer.**

15.1.39 The Broken Way (Data Loss)

```

1 #include <iostream>
2 #include <string>
3
4 class Base {
5 protected:
6     std::string base_data;
7 public:
8     Base(std::string s) : base_data{s} {}
9 };
10
11 class Derived : public Base {
12 private:
13     int* child_ptr; // Needs custom deep-copy management
14 public:
15     Derived(std::string s, int val) : Base{s}, child_ptr{new int(val)} {}
16
17     ~Derived() { delete child_ptr; }
18
19     // CUSTOM OVERLOAD TRAP
20     Derived& operator=(const Derived& other) {
21         if (this == &other) return *this;
22
23         // CRITICAL BUG: We are copying the child pointer,
24         // but we completely forgot to copy 'base_data' from the Base layer!
25         *child_ptr = *other.child_ptr;
26
27         return *this;
28     }
29 };

```

15.1.40 The Correct Way: Explicit Base Invocation

To fix this, you must explicitly invoke the parent class's assignment operator inside your child implementation. You do this by casting your reference or calling it directly via the scope resolution operator (`Base::`):

```

1 Derived& operator=(const Derived& other) {
2     if (this == &other) return *this;
3
4     // 1. Explicitly force the Base class components to copy slice-by-slice
5     Base::operator=(other);
6
7     // 2. Safely handle the custom child component logic
8     *child_ptr = *other.child_ptr;
9
10    return *this;
11 }

```

15.1.41 2. Stream Insertion (operator<<) and Slicing

Stream operators (like `std::ostream& operator<<()`) cannot be member functions because their left-hand operand is `std::cout`, not your object. Therefore, they must be implemented as non-member **friend** functions.

This introduces a massive issue with polymorphism. **Stream operators cannot be virtual.** If you pass a derived object to a function expecting a base reference, C++ will statically bind to the base stream operator and slice away the child's output data.

15.1.42 The Problem: Static Binding of Free Functions

```

1 class Base {
2 public:
3     friend std::ostream& operator<<(std::ostream& os, const Base& b) {
4         return os << "Base: " << b.base_data;
5     }
6 };
7
8 class Derived : public Base {
9     int child_val;
10 public:
11     friend std::ostream& operator<<(std::ostream& os, const Derived& d) {
12         return os << "Derived: " << d.base_data << ", " << d.child_val;
13     }
14 };
15
16 void print_object(const Base& obj) {
17     // TRAP: Because operator<< is a static friend function,
18     // it always binds to Base, completely bypassing Derived's overload!
19     std::cout << obj << '\n';
20 }

```

15.1.43 The Elegant Solution: The Virtual Helper Pattern

To achieve polymorphic printing while maintaining normal operator syntax, you route the non-virtual `operator<<` through a hidden virtual helper function (often named `print` or `toString`).

```

1 #include <iostream>
2 #include <string>
3
4 class Base {
5 protected:
6     std::string base_data;
7 public:
8     Base(std::string s) : base_data{s} {}
9
10    // 1. The Virtual Engine
11    virtual void print(std::ostream& os) const {
12        os << "Base: " << base_data;
13    }
14
15    // 2. The Global Operator Contract (Binds once at the root)
16    friend std::ostream& operator<<(std::ostream& os, const Base& b) {
17        b.print(os); // Delegates to the dynamic Vtable lookup!
18        return os;
19    }
20    virtual ~Base() = default;
21 };
22
23 class Derived : public Base {
24 private:
25     int child_val;
26 public:
27     Derived(std::string s, int v) : Base{s}, child_val{v} {}
28
29    // 3. Child simply overrides the printing logic cleanly
30    void print(std::ostream& os) const override {
31        os << "Derived: " << base_data << " | Child Value: " << child_val;
32    }
33 };

```



```

34
35 void print_object(const Base& obj) {
36     std::cout << obj << '\n'; // Triggers Vtable -> prints the derived layout perfectly!
37 }
38
39 void solve() {
40     Derived d("Miku", 39);
41     print_object(d); // Outputs: Derived: Miku / Child Value: 39
42 }

```

15.1.44 3. Comparison Operators (operator==, operator<)

Overloading comparison operators in inheritance hierarchies introduces a structural dilemma: **What happens if you try to compare a Cat to a Dog if both derive from Animal?**

If you try to make `operator==` virtual, the parameter signature must match *exactly*. This forces your child class to accept a generic reference to the Base class, requiring a `dynamic_cast` check before any real work can be done.

```

1  class Animal {
2  public:
3      virtual bool operator==(const Animal& other) const = 0;
4  };
5
6  class Cat : public Animal {
7  private:
8      int whiskers;
9  public:
10     bool operator==(const Animal& other) const override {
11         // 1. Check if the incoming animal is actually a Cat
12         const Cat* other_cat = dynamic_cast<const Cat*>(&other);
13         if (!other_cat) return false; // Not a cat, so they can't be equal
14
15         // 2. Compare the specific data safely
16         return this->whiskers == other_cat->whiskers;
17     }
18 };

```

15.1.45 Design Best Practice

Because type-checking inside comparisons is slow and architecturally messy, professional C++ guidelines generally recommend avoiding polymorphic comparisons altogether.

Instead, perform comparisons strictly on identical concrete types locally, or use a distinct virtual state evaluation identifier (like checking a `getTypeId()` or unique scalar attribute hash inside the base class).

Chapter 16

std::pair, std::tuple, and std::variant

16.1 std::pair

Concept / Pattern

```
1 #include <utility>
2
3 std::pair<int, std::string> p = {42, "hello"};
4 p.first;    // 42
5 p.second;   // "hello"
6
7 auto p2 = std::make_pair(1, 3.14); // type-deduced pair<int, double>
8
9 // Structured binding -- cleanest way to unpack
10 auto [num, str] = p;
11
12 // Very common: map::insert returns pair<iterator, bool>
13 auto [it, inserted] = my_map.insert({"key", 99});
14 if (inserted) std::cout << "new entry added\n";
```

16.2 std::tuple: N-ary Pair

```
1 #include <tuple>
2
3 auto t = std::make_tuple(1, 3.14, std::string("hi")); // tuple<int, double, string>
4
5 // Access by index (compile-time constant required)
6 std::get<0>(t);    // 1
7 std::get<1>(t);    // 3.14
8 std::get<2>(t);    // "hi"
9
10 // Access by type (must be unique)
11 std::get<std::string>(t); // "hi"
12
13 // Cleanest: structured bindings
14 auto [n, d, s] = t;
15
16 // Tie: unpack into existing variables (older style)
17 int x; double y; std::string z;
18 std::tie(x, y, z) = t;
19 std::tie(x, std::ignore, z) = t; // skip the double
20
21 // Size and type queries
22 constexpr int N = std::tuple_size_v<decltype(t)>; // 3
23 using T1 = std::tuple_element_t<1, decltype(t)>; // double
```

16.3 `std::variant`: Type-Safe Union

`std::variant<A, B, C>` holds exactly one value of type A, B, or C. Unlike a raw union, it always knows which type is active. Great for sum types and state machines.

```

1  #include <variant>
2
3  std::variant<int, double, std::string> v = 42;
4  v = 3.14;      // now holds double
5  v = "hello";   // now holds string
6
7  // Check active type
8  std::holds_alternative<std::string>(v); // true
9  v.index();     // 2 (index in the type list)
10
11 // Access -- throws std::bad_variant_access if wrong type
12 std::get<std::string>(v);    // "hello"
13
14 // Safe access via pointer (no throw)
15 if (auto* s = std::get_if<std::string>(&v))
16     std::cout << *s;
17
18 // Visit: calls a callable on whatever type is active
19 std::visit([](auto& val) { std::cout << val; }, v);
20
21 // Pattern-matching style with overloaded visitor
22 template<class... Ts> struct overloaded : Ts... { using Ts::operator()...; };
23 std::visit(overloaded{
24     [](int n)          { std::cout << "int: "    << n; },
25     [](double d)       { std::cout << "double: " << d; },
26     [](std::string& s) { std::cout << "string: " << s; },
27 }, v);

```

When to Use

Use `std::variant` when the set of possible types is fixed at compile time (parsers, AST nodes, state machines, error-or-value returns). For “any type at runtime”, use `std::any` – but that’s rarely needed in interview problems.

Chapter 17

STL Containers

17.1 Container Overview

Performance / Detail

Container	Access	Best used for
<code>std::vector<T></code>	$O(1)$ random	Default sequence. Fast append, random access.
<code>std::array<T,N></code>	$O(1)$ random	Fixed-size, stack-allocated. Prefer over C arrays.
<code>std::deque<T></code>	$O(1)$ random	Fast push/pop at both ends. Slower than vector.
<code>std::list<T></code>	$O(N)$	Fast insert/delete anywhere; poor cache locality.
<code>std::forward_list<T></code>	$O(N)$	Singly-linked list. More memory-efficient than list.
<code>std::map<K,V></code>	$O(\log N)$	Sorted key-value pairs. Keys are unique.
<code>std::multimap<K,V></code>	$O(\log N)$	Sorted, allows duplicate keys.
<code>std::set<T></code>	$O(\log N)$	Sorted unique elements.
<code>std::multiset<T></code>	$O(\log N)$	Sorted, allows duplicates.
<code>std::unordered_map<K,V></code>	$O(1)$ avg	Hash map. Fastest key lookup. No ordering.
<code>std::unordered_set<T></code>	$O(1)$ avg	Hash set. Fastest membership test.
<code>std::stack<T></code>	LIFO	Wrapper around deque. push/pop/-top.
<code>std::queue<T></code>	FIFO	push/pop/front/back.
<code>std::priority_queue<T></code>	$O(\log N)$	Max-heap by default.

17.2 `std::map` and `std::unordered_map`

Concept / Pattern

```

1 #include <map>
2 #include <unordered_map>
3
4 // std::map: sorted by key (red-black tree), O(log N)
5 std::map<std::string, int> scores;
6 scores["Alice"] = 90;
7 scores["Bob"] = 80;
8 scores.insert({"Carol", 70});           // insert a pair
9 scores.emplace("Dave", 85);           // in-place construction
10
11 // Access
12 scores["Alice"];           // 90 (inserts with value 0 if missing!)
13 scores.at("Alice");       // 90 (throws if key missing)
14
15 // Safe lookup
16 if (scores.count("Eve")) { /* Eve exists */ }
17 if (auto it = scores.find("Eve"); it != scores.end()) {
18     std::cout << it->second; // it->first=key, it->second=value
19 }
20
21 // Iteration: always in sorted order
22 for (const auto& [name, score] : scores)
23     std::cout << name << ": " << score << '\n';
24
25 // Erase
26 scores.erase("Bob");
27
28 // std::unordered_map: hash table, O(1) average
29 std::unordered_map<std::string, int> fast_scores;
30 fast_scores["Alice"] = 90; // same interface, but NO guaranteed order

```

Common Mistake

`map["key"]` does **two things**: looks up "key" AND inserts it with a default value (0 for int) if it didn't exist. Use `map.find()` or `map.count()` for safe read-only lookup.

17.3 `std::set` and `std::unordered_set`

```

1 #include <set>
2 #include <unordered_set>
3
4 std::set<int> s = {5, 3, 1, 4, 2};
5 // Elements are always sorted: {1, 2, 3, 4, 5}
6
7 s.insert(6);           // insert
8 s.erase(3);           // remove
9 bool has = s.count(4) > 0; // membership check
10 bool has2 = s.contains(4); // C++20, cleaner
11
12 // Set intersection / union / difference using std::algorithm
13 std::set<int> a = {1, 2, 3, 4};
14 std::set<int> b = {3, 4, 5, 6};
15 std::set<int> intersection;
16 std::set_intersection(a.begin(), a.end(), b.begin(), b.end(),
17     intersection.begin());
18 // intersection = {3, 4}
19
20 // unordered_set: O(1) average, no ordering guarantee
21 std::unordered_set<std::string> seen;

```

```

22 seen.insert("Alice");
23 seen.count("Alice"); // 1 (exists)

```

17.4 std::stack, std::queue, std::priority_queue

```

1  #include <stack>
2  #include <queue>
3
4  // Stack (LIFO)
5  std::stack<int> stk;
6  stk.push(1); stk.push(2); stk.push(3);
7  stk.top(); // 3 (peek, doesn't remove)
8  stk.pop(); // removes 3
9  stk.size(); // 2
10
11 // Queue (FIFO)
12 std::queue<int> q;
13 q.push(1); q.push(2); q.push(3);
14 q.front(); // 1 (peek front)
15 q.back(); // 3 (peek back)
16 q.pop(); // removes 1
17
18 // Priority Queue (max-heap by default)
19 std::priority_queue<int> pq;
20 pq.push(5); pq.push(1); pq.push(3);
21 pq.top(); // 5 (largest element)
22 pq.pop(); // removes 5
23
24 // Min-heap: use greater<int> as comparator
25 std::priority_queue<int, std::vector<int>, std::greater<int>> min_pq;
26 min_pq.push(5); min_pq.push(1);
27 min_pq.top(); // 1 (smallest)

```

17.5 std::deque: Double-Ended Queue

```

1  #include <deque>
2  std::deque<int> dq = {1, 2, 3};
3  dq.push_front(0); // {0, 1, 2, 3}
4  dq.push_back(4); // {0, 1, 2, 3, 4}
5  dq.pop_front(); // {1, 2, 3, 4}
6  dq.pop_back(); // {1, 2, 3}
7  dq[1]; // 2 (random access like vector)

```


Chapter 18

Iterators and Algorithms

18.1 Iterator Categories

Iterators are the glue between containers and algorithms. Think of them as generalized pointers.

Performance / Detail

Category	Supported operations & example containers
Input	Read-only, single-pass forward. (istream_iterator)
Output	Write-only, single-pass forward. (ostream_iterator)
Forward	Read/write, multi-pass, one direction. (forward_list)
Bidirectional	Forward + - operator. (list, map, set)
Random Access	All + it+n, it[n], it1 < it2. (vector, deque, array)
Contiguous (C++17)	Random access + contiguous memory guarantee. (vector, array)

```
1 std::vector<int> v = {1, 2, 3, 4, 5};
2
3 // begin/end: iterator to first element / one past last
4 auto it = v.begin(); // points to v[0]
5 *it; // 1 (dereference)
6 ++it; // advance to v[1]
7 it += 2; // random-access only: jump by 2 (now at v[3])
8 *it; // 4
9
10 // rbegin/rend: reverse iterators
11 for (auto rit = v.rbegin(); rit != v.rend(); ++rit)
12     std::cout << *rit; // 5 4 3 2 1
13
14 // cbegin/cend: const iterators (read-only)
15 for (auto it = v.cbegin(); it != v.cend(); ++it)
16     std::cout << *it; // can't modify through const iterator
17
18 // Iterator utilities
19 #include <iterator>
20 auto it2 = std::next(v.begin(), 3); // advance 3 steps
21 auto it3 = std::prev(v.end(), 1); // go back 1 step
22 auto dist = std::distance(v.begin(), v.end()); // 5
23 std::advance(it, 2); // advance in place
```

18.2 std::algorithm: The 20 You Need to Know

Concept / Pattern

```

1 #include <algorithm>
2 #include <numeric>
3 std::vector<int> v = {5, 3, 1, 4, 2};
4
5 // Sorting
6 std::sort(v.begin(), v.end()); // {1,2,3,4,5}
7 std::sort(v.begin(), v.end(), std::greater<int>{}); // {5,4,3,2,1}
8 std::stable_sort(v.begin(), v.end()); // preserves order of equal elements
9
10 // Searching
11 auto it = std::find(v.begin(), v.end(), 3); // iterator to 3, or end()
12 bool found = (it != v.end());
13
14 // Binary search (requires sorted range)
15 bool exists = std::binary_search(v.begin(), v.end(), 3);
16
17 auto lb = std::lower_bound(v.begin(), v.end(), 3); // first >= 3
18 auto ub = std::upper_bound(v.begin(), v.end(), 3); // first > 3
19
20 // Counting / Checking
21 int cnt = std::count(v.begin(), v.end(), 3);
22 int cnt2 = std::count_if(v.begin(), v.end(), [](int x){ return x > 3; });
23 bool all = std::all_of(v.begin(), v.end(), [](int x){ return x > 0; });
24 bool any = std::any_of(v.begin(), v.end(), [](int x){ return x > 4; });
25 bool none = std::none_of(v.begin(), v.end(), [](int x){ return x < 0; });
26
27 // Transformation
28 std::transform(v.begin(), v.end(), v.begin(), [](int x){ return x * 2; });
29 std::replace(v.begin(), v.end(), 3, 99); // replace all 3s with 99
30 std::replace_if(v.begin(), v.end(), [](int x){ return x < 3; }, 0);
31
32 // Copying / Moving
33 std::vector<int> dest(v.size());
34 std::copy(v.begin(), v.end(), dest.begin());
35 std::copy_if(v.begin(), v.end(), dest.begin(), [](int x){ return x % 2 == 0; });
36 std::move(v.begin(), v.end(), dest.begin()); // moves elements
37
38 // Removing
39 // std::remove doesn't actually erase -- use erase-remove idiom
40 v.erase(std::remove(v.begin(), v.end(), 3), v.end());
41 v.erase(std::remove_if(v.begin(), v.end(), [](int x){ return x > 3; }), v.end());
42
43 // Numeric
44 int sum = std::accumulate(v.begin(), v.end(), 0); // sum
45 int prod = std::accumulate(v.begin(), v.end(), 1, std::multiplies<int>{});
46 auto [mn, mx] = std::minmax_element(v.begin(), v.end()); // {min_iter, max_iter}
47 int mn2 = std::min_element(v.begin(), v.end())->value; // wrong syntax, use below
48 int min_val = *std::min_element(v.begin(), v.end());
49 int max_val = *std::max_element(v.begin(), v.end());
50
51 std::vector<int> prefix(v.size());
52 std::partial_sum(v.begin(), v.end(), prefix.begin()); // prefix sums
53 std::iota(v.begin(), v.end(), 0); // fill with 0, 1, 2, ..., N-1
54
55 // Permutations / Shuffling
56 std::reverse(v.begin(), v.end());
57 std::rotate(v.begin(), v.begin() + 2, v.end()); // rotate left by 2
58 std::shuffle(v.begin(), v.end(), std::mt19937{std::random_device{}()});
59 std::next_permutation(v.begin(), v.end()); // next lexicographic permutation

```

Key Insight

The **erase-remove idiom** is so common it has its own C++20 shorthand:

```
1 // C++20: std::erase and std::erase_if (no need for the idiom!)
2 std::erase(v, 3); // erase all 3s
3 std::erase_if(v, [](int x){ return x % 2 == 0; }); // erase all evens
```


Chapter 19

Error Handling and Exceptions

19.1 Why Exceptions?

C++ error handling comes in two flavours: **return codes** (like returning `-1` or `nullptr`) and **exceptions**. Return codes are easily ignored and clutter every call site. Exceptions force the caller to deal with the problem or explicitly propagate it.

Performance / Detail

Strategy	When to use
Return codes / <code>std::optional</code>	Performance-critical paths, predictable failures (e.g. “not found”), C-compatible APIs.
Exceptions	Truly exceptional, unexpected failures (e.g. file not found, network down, out of memory).
<code>std::expected</code> (C++23)	Best of both: typed error return without the overhead of exceptions.

19.2 try / catch / throw

Concept / Pattern

```

1 #include <stdexcept>
2 #include <iostream>
3
4 void risky(int x) {
5     if (x < 0)
6         throw std::invalid_argument("x must be non-negative");
7     if (x > 1000)
8         throw std::out_of_range("x is too large: " + std::to_string(x));
9 }
10
11 int main() {
12     try {
13         risky(-5);
14     }
15     catch (const std::invalid_argument& e) {    // most specific first!
16         std::cerr << "Bad arg: " << e.what() << '\n';
17     }
18     catch (const std::out_of_range& e) {
19         std::cerr << "Range: " << e.what() << '\n';
20     }
21     catch (const std::exception& e) {          // catch-all for std exceptions
22         std::cerr << "Error: " << e.what() << '\n';
23     }
24     catch (...) {                              // catch absolutely everything
25         std::cerr << "Unknown error!\n";
26     }
27 }

```

Common Mistake

Always catch by **const reference** (`const std::exception&`), never by value. Catching by value slices the exception object — you lose the derived type's information and waste a copy.

19.3 The Standard Exception Hierarchy

Performance / Detail

All standard exceptions inherit from `std::exception` and provide a `what()` method returning a C-string description.

```

1 std::exception
2     std::logic_error           // programming bugs detectable before runtime
3     std::invalid_argument     // wrong argument value
4     std::out_of_range         // index/value out of valid range
5     std::length_error         // exceeds max allowed size
6     std::domain_error         // math domain error
7 std::runtime_error            // errors only detectable at runtime
8     std::overflow_error       // arithmetic overflow
9     std::underflow_error      // arithmetic underflow
10    std::range_error           // result out of representable range
11 std::bad_alloc                // new/malloc failed (out of memory)
12 std::bad_cast                 // dynamic_cast failed on references
13 std::bad_optional_access      // dereferencing empty optional

```

19.4 Custom Exceptions

```

1 // Inherit from std::runtime_error for custom exceptions
2 class DatabaseError : public std::runtime_error {
3     int error_code;
4 public:
5     DatabaseError(int code, const std::string& msg)
6         : std::runtime_error(msg), error_code(code) {}
7
8     int code() const { return error_code; }
9 };
10
11 try {
12     throw DatabaseError(404, "Table not found");
13 }
14 catch (const DatabaseError& e) {
15     std::cerr << "DB error " << e.code() << ": " << e.what() << '\n';
16 }

```

19.5 noexcept: Promising No Exceptions

`noexcept` tells the compiler this function will never throw. If it does throw, `std::terminate()` is called immediately — no stack unwinding.

```

1 void swap_fast(int& a, int& b) noexcept {
2     int tmp = a; a = b; b = tmp;           // these can't throw anyway
3 }
4
5 // noexcept(expr): conditionally noexcept based on a compile-time bool
6 template <typename T>
7 void swap_generic(T& a, T& b) noexcept(std::is_nothrow_move_constructible_v<T>) {
8     T tmp = std::move(a);
9     a = std::move(b);
10    b = std::move(tmp);
11 }

```

Key Insight

Always mark move constructors and move assignment operators `noexcept`. `std::vector` checks this at compile time: if your move constructor might throw, it falls back to slow copies when reallocating the array. This can silently turn $O(1)$ moves into $O(N)$ copies.

19.6 Exception Safety Guarantees

Performance / Detail

When writing functions, you should document (and uphold) one of these guarantees:

Guarantee	What it means
No-throw	The function never throws. Marked <code>noexcept</code> . Best for destructors, moves, swaps.
Strong	If an exception is thrown, the program state is exactly as it was before the call (commit-or-rollback).
Basic	If an exception is thrown, no resources are leaked and the program is in a valid (but possibly changed) state. The minimum acceptable guarantee.
None	If an exception is thrown, anything can happen. Avoid this.

19.7 RAII: The Connection Between Exceptions and Resource Safety

When an exception is thrown, the stack **unwinds**: all local objects in scope between the **throw** and the matching **catch** have their destructors called. This is why RAII works so well with exceptions.

Concept / Pattern

```
1 void process_file(const std::string& path) {
2     std::ifstream file(path);      // RAII: file opened in constructor
3     if (!file) throw std::runtime_error("Cannot open: " + path);
4
5     // ... do work that might throw ...
6     might_throw();
7
8     // file.close() is called automatically in ~ifstream(),
9     // whether we exit normally OR via an exception.
10    // No resource leak, ever.
11 }
```

This is why you should **never** use raw **new/delete** directly — use **unique_ptr** so heap memory is automatically freed even when exceptions fly.

19.8 std::expected (C++23): Typed Error Returns

std::expected<T, E> holds either a value of type **T** (success) or an error of type **E** (failure). Zero overhead, no exception cost, typed errors.

```
1 #include <expected>
2 #include <string>
3
4 enum class ParseError { InvalidFormat, Overflow, Empty };
5
6 std::expected<int, ParseError> parse_int(std::string_view s) {
7     if (s.empty()) return std::unexpected(ParseError::Empty);
8     try { return std::stoi(std::string(s)); }
9     catch (...) { return std::unexpected(ParseError::InvalidFormat); }
10 }
11
12 auto result = parse_int("42");
13 if (result)           // check if it has a value
14     std::cout << *result;    // dereference to get the value
15 else
16     std::cout << "Error code: " << (int)result.error();
17
18 // Or use value_or for a default:
19 int val = parse_int("abc").value_or(-1); // -1 on error
```

When to Use

Use **std::expected** when: the failure is a normal, expected part of the API (like parsing user input), you need to propagate typed error info without exceptions, or you're in a **noexcept** context. Use exceptions for truly exceptional, unrecoverable conditions.

Chapter 20

Move Semantics and Value Categories

20.1 Value Categories: Lvalues, Rvalues, and References

In modern C++, every expression has a type (e.g., `int`, `std::string`) and a **value category**. The two primary value categories are **lvalues** and **rvalues**. Understanding these is crucial for mastering move semantics and perfect forwarding.

20.1.1 Lvalues vs. Rvalues

Concept / Pattern

The Simple Definition

- **lvalue** (locator value): An expression that points to a specific memory location. It has a persistent identity. *Rule of thumb: If you can take its address with `&`, it's an lvalue.*
- **rvalue** (read value): A temporary, ephemeral value that does not have a persistent memory address. It is usually evaluated and then immediately destroyed.

```
1 int x = 10;           // 'x' is an lvalue (it has a memory address). '10' is an rvalue (temporary)
2
3 int* p = &x;          // OK: taking the address of an lvalue.
4 // int* q = &10;      // ERROR: cannot take the address of an rvalue.
5
6 x = 20;               // OK: lvalues can appear on the left side of an assignment.
7 // 20 = x;           // ERROR: rvalues cannot appear on the left side.
```

What is Implicitly an Lvalue?

- **Variables:** Any named variable (e.g., `int x`, `std::string s`), even if it's declared as an rvalue reference (`T&& v`). If it has a name, it's an lvalue!
- **Dereferences:** `*ptr` evaluates to the lvalue it points to.
- **Array subscripts:** `arr[3]` is an lvalue.
- **Functions returning lvalue references:** Functions returning `T&` (e.g., `std::cout << std::map::operator[]`).
- **String literals:** `"hello"` is an lvalue of type `const char[6]`. (It lives in the read-only data segment).

What is Implicitly an Rvalue?

- **Literals:** `42`, `3.14`, `true`, `'a'` (except string literals).
- **Arithmetic/Logical results:** `x + y`, `x && y`.
- **Functions returning by value:** `int f()`, `std::string f()`. The returned temporary is an rvalue.
- **Lambdas:** `[]()` creates a temporary closure object (rvalue).

20.1.2 Lvalue References (&) vs. Rvalue References (&&)

References act as aliases. C++11 introduced rvalue references to allow binding to temporary objects, which enables **move semantics** (stealing resources instead of copying).

```

1  int x = 5;
2
3  // 1. Lvalue Reference (T&)
4  int& l_ref = x;           // OK: binds to lvalue
5  // int& l_ref2 = 5;       // ERROR: cannot bind lvalue reference to rvalue
6
7  // 2. Const Lvalue Reference (const T&) -- The Universal C++98 Binder
8  const int& c_ref = x;     // OK: binds to lvalue
9  const int& c_ref2 = 5;    // OK: extends the lifetime of the temporary rvalue '5'
10
11 // 3. Rvalue Reference (T&&) -- Introduced in C++11
12 // int&& r_ref = x;       // ERROR: cannot bind rvalue reference to lvalue
13 int&& r_ref2 = 5;         // OK: binds to rvalue temporary

```

Common Mistake

If It Has a Name: It's an Lvalue!

Inside a function that takes an rvalue reference (`void f(std::string&& s)`), the parameter `s` **has a name**. Therefore, inside the function body, `s` itself is treated as an **lvalue**, even though its type is “rvalue reference”. To pass it on as an rvalue, you must cast it again using `std::move(s)`.

20.1.3 Converting Lvalues to Rvalues: `std::move`

`std::move` does not actually move anything. It is merely a compile-time cast (specifically `static_cast<T&&>`) that unconditionally turns an lvalue into an rvalue. This tells the compiler: “*I am done using this object; you may steal its resources.*”

```

1  std::string a = "Hello";
2  std::string b = a;           // COPY: 'a' is an lvalue.
3  std::string c = std::move(a); // MOVE: 'a' is cast to an rvalue. Resources are stolen by '
   c'.
4  // 'a' is now in a valid but unspecified state (usually empty).

```

20.1.4 Reference Collapsing and Forwarding References

In template metaprogramming, it is common to take a template parameter as `T&&`. Depending on what you pass in, `T` might be deduced as a reference type itself. When references are composed (e.g., a reference to a reference), C++ uses **reference collapsing rules**.

Concept / Pattern

The Reference Collapsing Rule

If a type resolves to a reference to a reference, the references collapse to an lvalue reference (&) unless BOTH are rvalue references (&&).

- `& + & → &`
- `& + && → &`
- `&& + & → &`
- `&& + && → &&`

Forwarding References (Universal References)

A template parameter written as `T&&` is *not* just an rvalue reference. It is a **forwarding reference**. It can bind to *anything* (lvalues or rvalues) and preserves the value category of the argument via reference collapsing.

```
1 template <typename T>
2 void wrapper(T&& arg) { // T&& is a forwarding reference here
3     // Use std::forward to preserve the lvalue/rvalue nature of 'arg'
4     process(std::forward<T>(arg));
5 }
6
7 int x = 10;
8 wrapper(x); // Passed lvalue: T deduces as int&. T&& collapses to int&.
9 wrapper(20); // Passed rvalue: T deduces as int. T&& collapses to int&&.
```

Key Insight

std::move vs. std::forward

- Use **std::move()** when you have a specific variable and want to unconditionally cast it to an rvalue to allow moving.
- Use **std::forward<T>()** inside templates taking forwarding references (T&&) to conditionally cast to an rvalue *only* if the original argument was an rvalue.

Chapter 21

Templates and Compile-Time Programming

21.1 Template Processing: Compilation Mechanics & Metaprogramming

Templates in C++ are evaluated 100% during compilation. They leave behind absolutely no performance footprints or execution logic steps inside the final running app.

21.1.1 From Blueprint to Instance

1. Parsing: A generic template is not a concrete function; it is an abstract blueprint or instructions card. If a template is never explicitly used or triggered, the compiler strips it out completely.

2. Instantiation: When you supply real data types to a template call (e.g., `add(5, 5)`), the compiler intercepts the line, clones the blueprint, and writes a standalone, concrete version into the code pool for that specific combination.

```
1 template <typename T, typename U>
2 struct Pair { T first; U second; };
3
4 Pair<int, double> p1; // Compiler creates background class: Pair_int_double
5 Pair<string, int> p2; // Compiler creates background class: Pair_string_int
```

21.1.2 Trade-Offs

- Advantages: Zero Runtime Cost. Generic abstractions operate at exactly the same speed as dedicated handwritten custom instances.
- Disadvantages: Increased Compilation Time and Code Bloat. Generating dozens of physical variants for diverse configurations causes the final compiled binary file size to inflate.

21.1.3 Template Metaprogramming (TMP)

Because templates run purely while compiling, they can handle recursive calculations so the runtime app doesn't have to process them:

```
1 template<int N> struct Factorial { static const int value = N * Factorial<N - 1>::value; };
2 template<> struct Factorial<0> { static const int value = 1; };
3
4 int val = Factorial<5>::value; // Evaluated at compile-time to a static 120 literal constant
5 !
```

21.2 Polymorphism: Compile-Time vs. Runtime (vtable)

Polymorphism means providing a single unified interface to process diverse data types. C++ separates this execution path into static and dynamic bindings.

21.2.1 Compile-Time Polymorphism (Static Binding)

Uses Function Overloading or Templates. The compiler resolves the target call signature by checking source types during compilation.

- Code Example: Overloading `print(int)` and `print(double)`.
- Cost: Zero runtime performance impact. Fast execution paths are hardcoded directly into assembly jumps.

21.2.2 Runtime Polymorphism (Dynamic Binding)

Uses Inheritance and `virtual` functions. The execution target remains unresolved until the app runs, allowing behavior to adapt based on runtime input.

21.2.3 Mechanics of the Virtual Lookup Table (vtable)

When a class introduces a `virtual` function, the compiler stops creating explicit jumps in memory. Instead:

1. It builds a static index directory array for that specific class structure called the Virtual Table (`vtable`), tracking correct memory function pointers.
 2. Every instantiated object receives a hidden, internal object pointer member called the virtual table pointer (`_vptr`).
 3. During runtime, calling a virtual function causes the CPU to pause, follow the object's `_vptr` to the class `vtable`, search for the index, and then redirect to the address.
- Cost: Slower execution because of secondary memory jump lookups, minor metadata RAM inflation per instance, and potential CPU instruction cache misses.

Common Mistake

vtable Performance Tax

Every virtual function call forces the CPU to: (1) load the object's `_vptr`, (2) look up the function pointer in the `vtable` array, then (3) perform an indirect jump. This indirect jump cannot be inlined or predicted as reliably as a direct call, which can cause CPU branch-predictor misses and instruction-cache thrash in tight loops.

Rule of thumb: prefer compile-time polymorphism (templates/`if constexpr`) in hot paths; reserve `virtual` for extensibility-critical interfaces.

21.3 Templates + `if constexpr`: Compile-Time Specialisation

Combining Templates with `if constexpr` yields a paradigm called Compile-Time Specialization. Instead of writing split functions or deploying runtime tests, a single function template can alter its internal assembly lines completely based on input types.

21.3.1 Example: Iterator Advancement

```

1  template <typename Iterator>
2  void advance_iterator(Iterator& it, int n) {
3      if constexpr (std::is_same_v<typename std::iterator_traits<Iterator>::iterator_category,
4                      std::random_access_iterator_tag>) {
5          it += n; // Compiles down to an O(1) single speed step for vector iterators
6      } else {
7          for (int i = 0; i < n; ++i) ++it; // Compiles down to an O(n) step loop for lists
8      }
9  }
```

If you pass a vector iterator, the compiler confirms the `if constexpr` branch is true, generates *only* the `it += n` assembly line, and completely erases the `for` loop from that path.

21.4 `constexpr`: Immediate Functions & Zero-Runtime Footprints

Introduced in C++20, `constexpr` guarantees that a function can ONLY be executed at compile time. It completely blocks any runtime fallback.

21.4.1 `constexpr` vs. `constexpr`

- `constexpr`: "Run this calculation while compiling if the arguments are constants. If they are runtime variables, quietly step back and turn this into a normal runtime function."
- `constexpr`: "Run this calculation while compiling. If you cannot because an input is a dynamic runtime variable, immediately fail compilation and halt."

```

1 constexpr int square(int x) { return x * x; }
2
3 int main() {
4     int r1 = square(5); // Legal. Pre-calculated to 25.
5     int input; std::cin >> input;
6     int r2 = square(input); // HARD COMPILER ERROR! No runtime fallback allowed.
7 }
```

Immediate functions are perfect for compile-time string format validation (`std::format`), pre-baking massive look-up arrays, or securing safety bounds before deployment.

21.5 The Casting Nuclear Options: `const_cast` & `reinterpret_cast`

These two casts exist outside the normal type-safety system. They do not convert data — they *lie to the compiler* about what a piece of memory is. Use them only when you have an ironclad reason and you fully understand what the hardware will do.

21.5.1 `const_cast` — Stripping the Constness Label

What It Actually Does

`const_cast` does **not** modify any memory. It only changes the *type of the pointer or reference* the compiler uses to access the memory. Think of it as peeling a “DO NOT WRITE” sticker off a box — the sticker was just a label. Whether writing is actually safe depends entirely on whether the box was ever truly read-only in hardware.

```

1 const int x = 42;
2 // x lives at, say, address 0x7fff1000
3 // The compiler treats &x as type: const int*
4 // No pointer of type int* can legally point here
5
6 int* p = const_cast<int*>(&x);
7 // const_cast changes the TYPE of the pointer: const int* --> int*
8 // The address stored in p is STILL 0x7fff1000
9 // The memory itself has not changed at all -- only the label has
```

Concept / Pattern

What the Compiler Sees

```

1 // Before const_cast:
2 //   &x has type const int* --> writing via this pointer: COMPILE ERROR
3
4 // After const_cast<int*>(&x):
5 //   p has type int* --> writing via p: compiles fine
6 //   But the UNDERLYING OBJECT x is still const
7 //   Writing *p = 99 is Undefined Behaviour if x was declared const

```

The Legitimate Use Case

The only *safe* use of `const_cast` is calling a legacy C API that takes a non-const pointer but **provably never writes through it**:

```

1 // Old C library function -- badly declared, but internally read-only
2 void legacy_print(char* str); // should be const char*, but isn't
3
4 void modern_code() {
5     const std::string msg = "hello";
6
7     // Safe: we KNOW legacy_print won't write, it just reads
8     legacy_print(const_cast<char*>(msg.c_str()));
9
10    // The string's memory is NOT const-allocated (it's heap),
11    // so writing would not crash -- but we know it won't write anyway.
12 }

```

The Dangerous Case: Truly Const Objects

```

1 void solve() {
2     const int val = 100; // val is a TRUE compile-time constant
3
4     int* p = const_cast<int*>(&val);
5     *p = 200; // UNDEFINED BEHAVIOUR -- writing to a const object
6
7     std::cout << val << '\n'; // prints ??? -- depends on optimisation
8     std::cout << *p << '\n'; // also ??? -- same memory, but...
9 }

```

Common Mistake

Why This Produces Bizarre Output Under -O2

When you declare `const int val = 100`, the compiler **bakes the value 100 directly into every use of `val`** at compile time (constant folding). It replaces `val` with the literal 100 in the generated assembly — it never emits a memory-load instruction for `val` at all.

The assembly generated at `-O2` effectively becomes:

```

1 // What -O2 compiles 'cout << val' to:
2 // mov esi, 100      <-- hardcoded literal, no RAM access
3 // call std::ostream::operator<<
4
5 // Meanwhile *p = 200 writes to RAM address of val,
6 // but that write is NEVER READ BACK -- the compiler already
7 // substituted the literal. So you see 100 printed, even though
8 // RAM holds 200. Classic UB: two realities coexist.

```


Key Insight**When const_cast IS Safe**

- The original object was declared **without** `const` (e.g. a non-const local variable passed to a function taking `const T&`). You can cast the `const` away and write through the pointer safely.
- The pointer is **never written through** (legacy API interop, read-only bridge).
- **Never** cast away `const` on a truly `const`-declared object and write through it.

21.5.2 reinterpret_cast — Raw Bit Reinterpretation**What It Actually Does**

`reinterpret_cast` tells the compiler: “Take the address stored in this pointer and pretend the bytes there are a completely different type.” **No conversion happens.** No bits are flipped. The same sequence of bits is read through a different lens.

```

1 float f = 1.0f;
2 // f in memory (IEEE 754): 0x3F800000
3 // Bit pattern: 0011 1111 1000 0000 0000 0000 0000 0000
4
5 int* p = reinterpret_cast<int*>(&f);
6 // The compiler now reads those SAME 4 bytes as if they were an int
7 // No conversion: bits are untouched
8
9 std::cout << *p << '\n';
10 // Prints: 1065353216 (which is 0x3F800000 in decimal)
11 // The float value 1.0 and the integer 1065353216 share identical bits

```

Concept / Pattern**Compiler’s Perspective: No Code Generated**

```

1 float f = 1.0f;
2 int* p = reinterpret_cast<int*>(&f);
3
4 // What the compiler emits in assembly:
5 //   lea rax, [f]      <-- load ADDRESS of f
6 //   mov [p], rax      <-- store same address in p
7 //
8 // There is NO conversion instruction. reinterpret_cast costs
9 // exactly ZERO machine instructions. It is purely a compile-time
10 // type-label change -- the hardware sees the same bytes regardless.

```

The Strict Aliasing Rule Violation

This is the hidden landmine. The compiler’s optimiser is *allowed to assume* that two pointers of different types **never point to the same memory**. This is the **Strict Aliasing Rule**.

```

1 float f = 3.14f;
2 int* p = reinterpret_cast<int*>(&f);
3
4 *p = 0; // write 0 through the int* alias
5
6 std::cout << f; // Under -O2: compiler may still think f == 3.14!
7 // It cached f in a register and assumes no int*
8 // could have modified it (strict aliasing).
9 // Result: reads the cached register, prints 3.14
10 // even though RAM now holds 0.

```

Common Mistake**Why Strict Aliasing Makes This Worse Than `const_cast`**

With `const_cast` UB, you at least get a predictable wrong answer. With strict aliasing UB, the compiler is free to reorder, eliminate, or duplicate the conflicting reads and writes in *any* way it sees fit. The bug may appear or vanish depending on which source line is nearby, which loop it is in, or which `-O` flag you use. It is **non-deterministic** by spec.

Legitimate Uses

```

1 // (1) Hardware register mapping (embedded / kernel code)
2 // A hardware register lives at fixed physical address 0xDEAD0000
3 volatile uint32_t* reg = reinterpret_cast<volatile uint32_t*>(0xDEAD0000);
4 *reg = 0x01; // write to hardware register directly
5
6 // (2) Byte-level serialisation: inspect any object as raw bytes
7 float f = 3.14f;
8 unsigned char* bytes = reinterpret_cast<unsigned char*>(&f);
9 for (int i = 0; i < 4; ++i)
10     printf("%02X ", bytes[i]); // prints the 4 IEEE 754 bytes of 3.14
11 // Note: char/unsigned char aliases are explicitly allowed by the standard
12
13 // (3) Type punning the safe way (C++20: use std::bit_cast instead!)
14 // std::bit_cast<int>(f) is the modern, defined-behaviour replacement

```

Key Insight**Use `std::bit_cast` in C++20 Instead**

C++20 introduced `std::bit_cast<To>(from)`, which does the same bit-reinterpretation as `reinterpret_cast` type-punning but is **well-defined behaviour** and works in `constexpr` contexts:

```

1 #include <bit>
2 float f = 1.0f;
3 int i = std::bit_cast<int>(f); // defined, no UB, constexpr-safe

```

Prefer `std::bit_cast` for type punning whenever C++20 is available.

Concept / Pattern**`const_cast` vs. `reinterpret_cast` at a Glance**

Property	<code>const_cast</code>	<code>reinterpret_cast</code>
Changes bits?	No — label only	No — address reused as-is
Changes type label?	Yes — adds/removes const	Yes — any pointer type
Safe when?	Original object is non-const	char/byte aliasing; hardware regs
UB trigger	Writing to truly const object	Strict aliasing violation
Assembly cost	Zero instructions	Zero instructions
C++20 replacement	—	<code>std::bit_cast</code> for punning

21.6 Undefined Behaviour & Constant Folding

Consider this execution snippet modifying a stack constant:

```

1 void solve(){
2     const int val = 100;
3     int* mod = const_cast<int*>(&val);
4     *mod = 200;
5     cout << val << '\n';

```

6 }

21.6.1 Why Does It Not Crash at -00?

Because `val` is a local stack variable without aggressive optimizations enabled (-00). The variable occupies a small slot in the current stack segment that the hardware keeps open for read/write access. The pointer forcefully edits the bits to 200, and the stream reads that address, printing 200.

21.6.2 The Trap: Constant Folding under -02

If you activate optimization flags like -02, the compiler applies constant folding. Since you promised `val` was `const`, the compiler replaces the variable with the hardcoded number 100 during compilation.

The optimized execution path transforms behind the scenes to:

```
1 void solve() {
2     // ... pointer writes 200 to stack RAM address ...
3     cout << 100 << '\n'; // Hardcoded token bypasses RAM read entirely!
4 }
```

Result: The console prints 100, even though the underlying memory location physically holds 200. This is the classic unpredictable behavior of code containing UB.

21.7 Compiler Optimisation Tiers: -00 through -03

Optimization flags change how the compiler transforms source files into assembly instructions.

21.7.1 -00: Zero Optimisation (Debug)

The default debug state. Translates code literally to match human-written lines step-for-step. Compiles quickly, but the execution speed is slow.

21.7.2 -02: Production Level

Applies aggressive, safe modifications to maximize execution speed without inflating binary size.

- Key passes: Constant Folding, Dead Code Elimination (deleting unused paths), and Instruction Scheduling (re-ordering machine commands to prevent CPU stalls).

21.7.3 -03: Aggressive Speed Performance

Turns on all -02 settings, plus radical performance transformations.

- Auto-Vectorization (SIMD): Restructures linear array tracking loops into single-cycle parallel vector commands using AVX or NEON processing hardware registers.
- Loop Unrolling: Duplicates small loop contents to remove the counter check overhead.
- Aggressive Inlining: Copy-pastes function blocks across call instances to clear frame jump delays.

21.7.4 Trade-offs of -03

Can cause code bloat (inflated executable size), ruins standard step-by-step debugging (variables show up as `<optimized out>`), and aggressively exposes hidden Undefined Behavior bugs.

Concept / Pattern

Compiler Flag Quick Reference

- `-O0` No optimisation. Literal translation. Use for debugging.
- `-O1` Basic safe optimisations (dead code, simple inlining).
- `-O2` Production level: constant folding, instruction scheduling, loop optimisation. **Default for release builds.**
- `-O3` Aggressive: SIMD vectorisation, loop unrolling, aggressive inlining. Can expose UB.
- `-Os` Optimise for *size* rather than speed.
- `-Og` Optimise for debuggability (GCC extension).

Chapter 22

Timing, Formatting, and Advanced Class Features

22.1 Basic Timing with `std::chrono`

```
1 #include <chrono>
2 using namespace std::chrono_literals;
3
4 // Timing a block of code
5 auto start = std::chrono::high_resolution_clock::now();
6 // ... code to measure ...
7 auto end = std::chrono::high_resolution_clock::now();
8 auto elapsed = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
9 std::cout << "Took: " << elapsed.count() << " ms\n";
10
11 // Sleep
12 std::this_thread::sleep_for(100ms); // sleep 100 milliseconds
13 std::this_thread::sleep_for(1s); // sleep 1 second
```

22.2 `std::format` (C++20): Type-Safe String Formatting

```
1 #include <format>
2
3 std::string s = std::format("Hello, {}!", "world"); // "Hello, world!"
4 std::string n = std::format("{} + {} = {}", 1, 2, 3); // "1 + 2 = 3"
5
6 // Width, alignment, padding
7 std::format("{:>10}", "hi"); // "          hi" (right-align, width 10)
8 std::format("{:<10}", "hi"); // "hi          " (left-align)
9 std::format("{:05}", 42); // "00042" (zero-pad)
10 std::format("{:.3f}", 3.14159); // "3.142" (3 decimal places)
11 std::format("{:x}", 255); // "ff" (hex)
12 std::format("{:#x}", 255); // "0xff" (hex with prefix)
13
14 // Print directly (avoids building a string first)
15 std::cout << std::format("value = {}\n", 42);
```

22.3 Class Template Argument Deduction (CTAD, C++17)

Before C++17 you had to spell out template arguments. CTAD lets the compiler deduce them.

```
1 // Old way (before C++17)
2 std::pair<int, double> p1(1, 3.14);
3 auto p2 = std::make_pair(1, 3.14); // workaround helper
4
```

```

5 // C++17 CTAD
6 std::pair p3(1, 3.14);           // deduced: pair<int, double>
7 std::vector v{1, 2, 3};         // deduced: vector<int>
8 std::array a{1, 2, 3};          // deduced: array<int, 3>
9 std::tuple t(1, "hi", 3.14);    // deduced: tuple<int, const char*, double>

```

22.4 The Rule of Zero

If your class uses RAII members (like `unique_ptr`, `vector`, `string`), don't write any of the Big Five — the compiler generates correct ones automatically.

Concept / Pattern

```

1 // AVOID: manually managing raw memory
2 class OldStyle {
3     int* data;
4 public:
5     OldStyle(int n) : data(new int[n]) {}
6     ~OldStyle()      { delete[] data; }
7     OldStyle(const OldStyle& o) { /* deep copy */ }
8     OldStyle& operator=(const OldStyle&) { /* ... */ return *this; }
9     OldStyle(OldStyle&&) noexcept { /* ... */ }
10    OldStyle& operator=(OldStyle&&) noexcept { /* ... */ return *this; }
11 };
12
13 // PREFER: use RAII wrappers -- Rule of Zero applies
14 class NewStyle {
15     std::vector<int> data; // manages its own memory
16 public:
17     NewStyle(int n) : data(n) {}
18     // Compiler generates all 5 correctly and efficiently -- nothing to write!
19 };

```

22.5 Aggregate Types and Aggregate Initialization

An **aggregate** is a class/struct with no user-declared constructors, no private/protected members, no virtual functions, and no inherited constructors.

```

1 struct Point { int x; int y; };
2 struct RGB   { float r, g, b; };
3
4 Point p = {3, 4};           // aggregate initialization
5 RGB c = {0.5f, 1.f, 0.f};
6 auto p2 = Point{.x=3, .y=4}; // C++20: designated initializers
7
8 static_assert(std::is_aggregate_v<Point>); // true

```

22.6 RAII Scope Guard: Cleanup on Any Exit

```

1 // A generic scope guard: runs cleanup function when it goes out of scope,
2 // regardless of whether the scope exits normally or via exception
3 template <typename F>
4 class ScopeGuard {
5     F cleanup;
6     bool active = true;
7 public:
8     explicit ScopeGuard(F f) : cleanup(std::move(f)) {}
9     ~ScopeGuard() { if (active) cleanup(); }
10    void dismiss() { active = false; } // cancel if work succeeded

```

```
11 };
12
13 void risky() {
14     FILE* f = fopen("data.txt", "r");
15     ScopeGuard guard([&]{ if (f) fclose(f); }); // guaranteed cleanup
16
17     if (/* error */) throw std::runtime_error("oops");
18     // file closed on any exit path -- no leak possible
19 }
```


Chapter 23

C++ Attributes

23.1 Attributes You Should Actually Use

Concept / Pattern

```
1 // [[nodiscard]]: warn if caller throws away the return value
2 [[nodiscard]] int compute() { return 42; }
3 compute(); // WARNING: ignoring return value ofnodiscard function
4
5 // C++20: add an explanatory message
6 [[nodiscard("Ignoring this leaks the resource!")]]
7 std::unique_ptr<int> allocate();
8
9 // [[maybe_unused]]: suppress "unused variable/parameter" warnings
10 [[maybe_unused]] int debug_info = 0;
11 void foo([[maybe_unused]] int flag) { }
12
13 // [[deprecated]]: marks something as old, warns callers
14 [[deprecated("Use new_api() instead")]]
15 void old_api() { }
16 old_api(); // WARNING: old_api is deprecated
17
18 // [[fallthrough]]: intentional fall-through in switch (see control flow chapter)
19 switch (x) {
20     case 1: do_thing(); [[fallthrough]];
21     case 2: do_other(); break;
22 }
23
24 // [[likely]] / [[unlikely]] (C++20): hint for branch prediction
25 if ([[likely]] x > 0) { /* fast path */ }
26 if ([[unlikely]] x < 0) { /* rare error */ }
27
28 // [[noreturn]]: function never returns (calls abort / throws unconditionally)
29 [[noreturn]] void fatal(std::string_view msg) {
30     std::cerr << msg;
31     std::abort();
32 }
```


Chapter 24

C++20 Features: Concepts and More

24.1 Concepts: Constrained Templates

Concepts let you say “this template only works if T satisfies these constraints”. They make template error messages human-readable instead of a wall of gibberish.

Concept / Pattern

```
1 #include <concepts>
2
3 // Define a concept: T must support +, -, *, /
4 template <typename T>
5 concept Arithmetic = std::is_arithmetic_v<T>;
6
7 // Use it to constrain a template
8 template <Arithmetic T>
9 T add(T a, T b) { return a + b; }
10
11 add(1, 2);           // OK: int satisfies Arithmetic
12 add(1.5, 2.5);       // OK: double satisfies Arithmetic
13 // add("a", "b");    // COMPILER ERROR: string doesn't satisfy Arithmetic
14                      // Error message is clear: "constraint not satisfied"
15
16 // Shorthand: requires clause
17 template <typename T> requires Arithmetic<T>
18 T multiply(T a, T b) { return a * b; }
19
20 // Built-in concepts from <concepts>
21 template <std::integral T> void foo(T); // only integer types
22 template <std::floating_point T> void bar(T); // only float/double
23 template <std::totally_ordered T> bool less_than(T a, T b) { return a < b; }
```

```
1 // Concept with requires expression (inline constraints)
2 template <typename T>
3 concept Printable = requires(T t) {
4     { std::cout << t } -> std::same_as<std::ostream&>;
5 };
6
7 template <Printable T>
8 void print(T val) { std::cout << val; }
```

24.2 std::span: Non-Owning View over Contiguous Data

`std::span<T>` is to arrays what `string_view` is to strings — a lightweight, non-owning view over contiguous memory. Works with `std::vector`, `std::array`, raw arrays, anything contiguous.

```

1 #include <span>
2
3 void process(std::span<int> data) { // works with any contiguous int container
4     for (int& x : data) x *= 2;
5     std::cout << data.size() << '\n';
6 }
7
8 std::vector<int> v = {1, 2, 3, 4, 5};
9 process(v); // works!
10
11 int arr[5] = {1, 2, 3, 4, 5};
12 process(arr); // works too!
13
14 // Subspan: view of a portion, O(1)
15 auto sub = std::span(v).subspan(1, 3); // view of v[1..3]

```

24.3 Three-Way Comparison: <=> (Spaceship)

```

1 #include <compare>
2
3 struct Point {
4     int x, y;
5     // Generates all 6 comparison operators automatically
6     auto operator<=>(const Point&) const = default;
7 };
8
9 Point a{1, 2}, b{3, 4};
10 if (a < b) { /* ... */ } // works now
11 if (a == b) { /* ... */ } // works now
12
13 // Returns a comparison category:
14 auto cmp = a <=> b;
15 if (cmp < 0) { /* a < b */ }
16 if (cmp == 0) { /* a == b */ }
17 if (cmp > 0) { /* a > b */ }
18
19 // std::strong_ordering: for types where == means identical
20 // std::weak_ordering: for types with equivalence (not identity)
21 // std::partial_ordering: for types where some pairs aren't comparable (NaN)

```

24.4 Designated Initializers (C++20)

```

1 struct Config {
2     int width = 800;
3     int height = 600;
4     bool fullscreen = false;
5     std::string title = "App";
6 };
7
8 // Initialize only the fields you want, in order
9 Config c{
10     .width = 1920,
11     .height = 1080,
12     .title = "My Game"
13     // fullscreen stays false (default)
14 };

```

Chapter 25

Multithreading and Concurrency

25.1 `std::thread`: Basic Threading

Concept / Pattern

```
1 #include <thread>
2
3 void task(int id) {
4     std::cout << "Thread " << id << '\n';
5 }
6
7 int main() {
8     std::thread t1(task, 1);
9     std::thread t2([]{ std::cout << "Lambda thread\n"; });
10
11     t1.join(); // wait for t1 to finish
12     t2.join(); // MUST join or detach before destructor runs
13
14     // t.detach(): let the thread run independently (fire-and-forget)
15     // after detach you can't join it
16 }
```

Common Mistake

If a `std::thread` object is destroyed without `join()` or `detach()`, `std::terminate()` is called immediately. Always join or detach. In C++20, `std::jthread` auto-joins on destruction — prefer it.

25.2 Mutexes and Locks

```
1 #include <mutex>
2
3 std::mutex mtx;
4 int shared = 0;
5
6 void safe_increment() {
7     std::lock_guard<std::mutex> lock(mtx); // RAII: locks now, unlocks at end of scope
8     ++shared;
9 }
10
11 // unique_lock: more flexible -- can be deferred, timed, manually unlock early
12 void flexible() {
13     std::unique_lock<std::mutex> lock(mtx);
14     ++shared;
15     lock.unlock(); // release early if needed
16     // do work without holding lock
```

```

17 }
18
19 // Lock two mutexes safely (no deadlock)
20 std::mutex m1, m2;
21 std::scoped_lock lk(m1, m2); // C++17: locks both atomically

```

25.3 std::atomic: Lock-Free Shared Variables

```

1 #include <atomic>
2
3 std::atomic<int> counter{0};
4
5 void increment() {
6     ++counter;           // atomic: thread-safe with no mutex
7     counter.fetch_add(5); // add 5, return old value
8     counter.load();       // atomic read
9     counter.store(0);     // atomic write
10    int old = counter.exchange(99); // set to 99, get old value back
11 }
12
13 // compare_exchange: foundation of lock-free algorithms
14 int expected = 5;
15 bool swapped = counter.compare_exchange_strong(expected, 10);
16 // If counter==5: sets to 10, returns true
17 // If counter!=5: sets expected=counter, returns false

```

25.4 std::async and std::future

```

1 #include <future>
2
3 // Run a task asynchronously, get the result via future
4 auto fut = std::async(std::launch::async, []() {
5     return 42; // computed in another thread
6 });
7
8 // do other work here while it runs...
9
10 int result = fut.get(); // blocks until ready, returns 42
11 // fut.get() also re-throws any exception the async task threw
12
13 // std::promise: manually signal a future from another thread
14 std::promise<int> prom;
15 std::future<int> future = prom.get_future();
16
17 std::thread t([&]{ prom.set_value(99); });
18 int val = future.get(); // 99
19 t.join();

```

25.5 std::condition_variable

```

1 std::mutex mtx;
2 std::condition_variable cv;
3 bool data_ready = false;
4
5 // Producer
6 void producer() {
7     { std::lock_guard lock(mtx); data_ready = true; }
8     cv.notify_one(); // wake one waiting thread (notify_all wakes all)
9 }
10
11 // Consumer

```

```
12 void consumer() {  
13     std::unique_lock lock(mtx);  
14     cv.wait(lock, []{ return data_ready; }); // wait until predicate is true  
15     // auto-handles spurious wakeups via the lambda  
16     std::cout << "Got data!\n";  
17 }
```


Chapter 26

Common Interview Patterns and Idioms

26.1 Copy-and-Swap for Assignment Operators

A clean, exception-safe way to implement assignment. The trick: take the argument *by value* (which handles both copy and move via overload resolution), then swap.

```
1 class Buffer {
2     std::unique_ptr<int []> data;
3     size_t size;
4 public:
5     Buffer(size_t n) : data(std::make_unique<int []>(n)), size(n) {}
6
7     // Copy constructor
8     Buffer(const Buffer& o) : data(std::make_unique<int []>(o.size)), size(o.size) {
9         std::copy(o.data.get(), o.data.get() + size, data.get());
10    }
11
12    // Move constructor
13    Buffer(Buffer&&) noexcept = default;
14
15    // Assignment: takes by value -- one function handles both copy and move
16    Buffer& operator=(Buffer other) noexcept {
17        std::swap(data, other.data);
18        std::swap(size, other.size);
19        return *this; // 'other' destructor frees the old data
20    }
21 };
```

26.2 Structured Bindings: Full Reference

```
1 // Works with: pairs, tuples, aggregates, arrays, map entries
2 auto [x, y] = std::pair{1, 2.0};
3
4 // Struct / aggregate decomposition
5 struct Point { int x, y, z; };
6 auto [a, b, c] = Point{1, 2, 3};
7
8 // Array decomposition
9 int arr[3] = {10, 20, 30};
10 auto [p, q, r] = arr;
11
12 // Map iteration -- most common interview use
13 std::map<std::string, int> m = {"a", 1}, {"b", 2};
14 for (const auto& [key, val] : m)
15     std::cout << key << ": " << val << '\n';
16
17 // const auto& doesn't copy
18 const auto& [cx, cy] = Point{1, 2, 3};
```

26.3 Useful `std::algorithm` Patterns for Interviews

```

1 // Erase-remove idiom (classic) -- also needed for lists
2 v.erase(std::remove(v.begin(), v.end(), target), v.end());
3 v.erase(std::remove_if(v.begin(), v.end(), pred), v.end());
4
5 // C++20 shorthand
6 std::erase(v, target);
7 std::erase_if(v, pred);
8
9 // Prefix sums (very common in DP and range queries)
10 std::vector<int> prefix(v.size() + 1, 0);
11 std::partial_sum(v.begin(), v.end(), prefix.begin() + 1);
12
13 // Fill with 0, 1, 2, ..., N-1
14 std::iota(v.begin(), v.end(), 0);
15
16 // Next permutation (permutation problems)
17 std::sort(v.begin(), v.end()); // start from smallest permutation
18 do {
19     // process permutation
20 } while (std::next_permutation(v.begin(), v.end()));
21
22 // Find with custom comparator
23 auto it = std::min_element(v.begin(), v.end(),
24     [](const auto& a, const auto& b){ return a.score < b.score; });

```